

π Tantum

alias Tempus Tantum

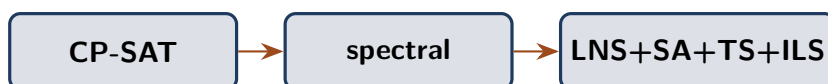
sistema di generazione orario scolastico

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”
Lucio Anneo Seneca, *Epistulae morales ad Lucilium*, I, 1

Manuale

Un metodo per l'orario delle scuole italiane

Da un'idea di Fernando Gargiulo, Giovanni Borghi,
Matteo Mariani, Stefano Bertozzi



Giovanni Borghi

3 maggio 2026

Repository: <https://github.com/gborghi/timetable>

Documentazione sorgente: docs/

Colophon

Questo volume è composto in EB Garamond per il corpo del testo e in Lato per i titoli, con Inconsolata per i blocchi di codice. La compilazione è stata eseguita con lualatex (TeX Live), biber per la bibliografia e makeindex per l'indice analitico. Il sorgente $\LaTeX$ risiede in docs/manual.tex con i capitoli in docs/manual/chapters/; il bibliografico è in docs/manual/bibliography.bib. La pipeline di build è in docs/build_manual.sh (Linux/macOS/Git Bash) e docs/build_manual.bat (Windows).

Tipografia: pacchetti ebgaramond, lato, inconsolata, microtype.

Layout: KOMA-Script scrbook, geometry, scrlayer-scrpage.

Editoriale: tcolorbox, lettrine, marginnote, epigraph, tikz, pgfplots.

Bibliografia e indici: biblatex backend biber, imakeidx.

Versione del repository corrispondente al commit visibile nel log git `log --oneline -- docs/`. Eventuali errori e suggerimenti possono essere segnalati come *issue* sul repository GitHub all'indirizzo <https://github.com/gborghi/timetable>.

Tutti i diritti del progetto sono di Giovanni Borghi. Documento prodotto a maggio 2026.

Indice

Elenco delle figure	9
Elenco delle tabelle	10
Prologo: come nasce questo libro	11
1 Il problema dell'orario scolastico	15
1.1 Quante combinazioni possibili	15
1.2 Vincoli rigidi e vincoli morbidi	16
1.3 Una breve storia delle idee	16
1.4 Le quattro famiglie di approcci	17
1.5 La filosofia di π Tantum	17
1.6 Le peculiarità del caso italiano	18
2 Una panoramica di πTantum	20
2.1 Cosa fa il sistema, in tre frasi	20
2.2 Tre layer in un unico repository	20
2.3 Diagramma a blocchi del sistema	21
2.4 Il flusso d'uso quotidiano	21
2.5 Quando il sistema non basta da solo	23
2.6 Le tecnologie scelte e perché	23
3 L'orario come oggetto strutturale	25
3.1 I docenti	25
3.2 Le classi	25
3.3 Le materie	26
3.4 Le aule	26
3.5 Gli indirizzi di studio	26
3.6 Gli studenti	27
3.7 I gruppi articolati	27
3.8 Le relazioni fra le entità	27
3.9 I vincoli ricorrenti del caso italiano	28
3.10 Cosa rende peculiare il sistema italiano	28
4 Installazione	30
4.1 Windows	30
4.2 Linux (Ubuntu, Debian, Fedora, Arch)	31
4.3 macOS (Intel + Apple Silicon)	31
4.4 Verifica installazione	32
4.5 Aggiornamento	33
4.6 Disinstallazione	33
5 Vincoli	34
5.1 I 5 stati / 4 kind	34
5.2 Matrici di disponibilità 5-stati	34
5.3 Vincoli logici (DNF)	35
5.4 Vincoli HARD a toggle (per classe)	36
5.5 Conflict detection	36
5.6 Tag aule – etichette libere	36
5.7 Preferenze di giorno libero del docente	37

5.8	Preferenze di giorno libero della classe	37
6	Workflow di ottimizzazione	38
6.1	Schema delle fasi	38
6.2	Phase A: assegnazione (cattedre)	38
6.3	Phase B: scheduling	38
6.4	Metaeuristiche	39
6.5	Phase 4: assegnazione aule	39
6.6	Drag-and-drop con preview live	39
6.7	Slot picker matriciale (Monitor)	40
6.8	Punteggio di graduatoria e criterio Anzianita	40
7	Guida UI	41
7.1	Funzionalità trasversali	41
7.2	Tab per tab	43
8	Launcher	45
8.1	Windows: start.bat	45
8.2	Linux / macOS: start.sh + stop.sh	45
9	CP-SAT: il motore che cerca, deduce, ritratta	46
9.1	Il problema concreto	46
9.2	Una storia in cinque tappe	46
9.3	Costruzione passo-passo: cosa serve sapere	48
9.4	Esempio mini-completo: 3 docenti, 2 classi	49
9.5	Generalizzazione alla scuola reale	51
9.6	Quando usarlo (e quando no)	51
9.7	Connessioni con il resto del sistema	52
10	Decomposizione spettrale: spezzare per regnare	53
10.1	Da dove viene l'idea	53
10.2	Costruzione passo-passo	53
10.3	Esempio mini-completo a mano	55
10.4	Generalizzazione: cosa cambia su scuola reale	55
10.5	Quando funziona, quando no	56
10.6	Connessioni	56
11	Metaeuristiche: scolpire una soluzione	58
11.1	La radice comune: local search	58
11.2	Simulated Annealing (SA): l'analogia con la metallurgia	58
11.3	Tabu Search (TS): il taccuino delle mosse vietate	59
11.4	Iterated Local Search (ILS): perturbare e ricominciare	59
11.5	Variable Neighborhood Search (VNS): cambiare obiettivo	60
11.6	Large Neighborhood Search (LNS): distruggi e ricostruisci	60
11.7	Adaptive LNS (ALNS): scolpire con scalpelli adattivi	61
11.8	Quale scegliere?	61
11.9	Esempio mini-completo: SA su 8 lezioni	61
11.10	Limiti e parametri da tarare	62
11.11	Connessioni	62
12	Hall pre-check: il livello di benzina prima di partire	63
12.1	Il teorema dei matrimoni	63
12.2	Dal teorema all'orario	63
12.3	Tre controlli pratici	64
12.4	Esempio mini-completo	65
12.5	L'integrazione nell'UI	65
12.6	Limiti e quando non basta	65
12.7	Approfondimento: l'algoritmo di Hopcroft-Karp	66

12.8	Connessioni	66
13	Lagrangian Relaxation e Column Generation	67
13.1	Rilassamento lagrangiano: dualizzare i ponti	67
13.2	Column Generation: il catalogo immobiliare	69
13.3	Connessioni	70
14	Monte Carlo: simulare per capire	71
14.1	La storia del nome	71
14.2	Sensitivity analysis: cosa misurare	71
14.3	Schema operativo	72
14.4	Quando è utile	72
14.5	Quando non basta	72
14.6	Connessioni	73
15	Analisi del grafo bipartito: vedere la struttura	74
15.1	Modularità: quanto è “a cluster” la scuola	74
15.2	Betweenness centrality: i “ponticelli” del sistema	75
15.3	Densità: quanto è “pieno” il grafo	75
15.4	Visualizzazione e interpretazione	75
15.5	Connessioni	76
16	Statistica applicata: leggere i risultati	77
16.1	Distribuzioni: vedere la forma dei dati	77
16.2	Correlazioni e regressioni	78
16.3	Chi-quadro: indipendenza	79
16.4	Visualizzazione interattiva	79
16.5	Connessioni	79
17	Il DSL generico: un linguaggio per i vincoli	80
17.1	Filosofia: “un parser, molti compilatori”	80
17.2	Grammatica (sintesi)	80
17.3	Parser e AST	81
17.4	Tre back-end	81
17.5	Estensione: aggiungere un built-in	82
17.6	Performance	82
17.7	Roadmap	82
17.8	Connessioni	83
18	Architettura	84
18.1	Stack	84
18.2	Diagramma a blocchi	84
18.3	Struttura cartelle	84
18.4	Lifecycle del backend	85
19	Modello dati	86
19.1	Famiglie di tabelle	86
19.2	Diagramma relazioni	87
19.3	Migrazioni idempotenti	87
19.4	Pickle dell’engine	87
20	DSL generico per i vincoli	88
20.1	Filosofia	88
20.2	Grammatica BNF	88
20.3	Sorgenti iterabili	89
20.4	Galleria di esempi	89
20.5	Errori comuni	93

21	Tecniche di ottimizzazione avanzate	94
21.1	Adaptive Large Neighborhood Search (ALNS)	94
21.2	Variable Neighborhood Search (VNS)	94
21.3	Hall's theorem pre-check	94
21.4	Column Generation (Dantzig-Wolfe)	94
21.5	Lagrangian Relaxation	94
22	Diagnostica statistica	95
22.1	Sensitivity Monte Carlo	95
22.2	Analisi bipartito	95
22.3	Correlazioni e regressioni	95
22.4	Distribuzioni	95
22.5	Run telemetry	95
23	API REST	96
23.1	Pattern CRUD	96
23.2	Esempi curl	96
23.3	Riferimento gruppi di endpoint	97
24	Estendere il sistema	98
24.1	Aggiungere una nuova tabella + CRUD	98
24.2	Migrazione idempotente	98
24.3	Aggiungere un nuovo tipo di vincolo	98
24.4	Aggiungere un nuovo step di ottimizzazione	99
24.5	Testing	99
A	Repository GitHub	100
B	Benchmark e risultati	101
B.1	I cinque profili	101
B.2	Pipeline end-to-end: tempi totali	101
B.3	Componenti dell'obiettivo SOFT	101
B.4	Hall pre-check: tempo trascurabile	102
B.5	Decomposizione spettrale: speedup misurato	102
B.6	Metaeuristiche: convergenza dell'obiettivo	103
B.7	Lettura dei risultati	103
B.8	Confronto cross-method	104
C	Lessons learned	105
C.1	Cosa ha funzionato	105
C.2	Cosa non ha funzionato (al primo tentativo)	106
C.3	Quello che ancora si può migliorare	107
C.4	Una riflessione finale	107
D	Glossario discorsivo dei metodi	108
D.1	CP-SAT (Constraint Programming over SAT)	108
D.2	Decomposizione spettrale	108
D.3	LNS (Large Neighborhood Search)	109
D.4	ALNS (Adaptive LNS)	109
D.5	SA (Simulated Annealing)	110
D.6	TS (Tabu Search)	110
D.7	ILS (Iterated Local Search)	110
D.8	VNS (Variable Neighborhood Search)	111
D.9	Hall's theorem pre-check	111
D.10	Lagrangian Relaxation con sub-gradient	111
D.11	Column Generation (Dantzig-Wolfe)	112
D.12	Monte Carlo Sensitivity	112
D.13	Modularità, betweenness, densità (analisi del bipartito)	112

D.14	Distribuzioni e istogrammi	II3
D.15	Correlazioni e regressioni	II3
D.16	DSL generico	II4
D.17	Stati 5-colori delle matrici	II4
D.18	Tag aule e tag studenti	II4
D.19	Run telemetry e visualizzazione live	II5
D.20	Lockati: lezioni che non si toccano	II5
E	Reference	II6
	Bibliografia	II7
	Indice analitico	II9

Elenco delle figure

2.1	Architettura a blocchi di π Tantum: il frontend parla al backend tramite REST, il backend persiste su SQLite e dialoga con il solver attraverso un convertitore esplicito basato su pickle, il solver invoca OR-Tools per le risoluzioni CP-SAT.	21
6.1	Pipeline a 4 fasi.	38
9.1	Il ciclo CP-SAT. Dopo aver propagato il modello iniziale, il solver alterna decisioni e propagazioni; ogni conflitto produce una clausola appresa che evita di ripetere lo stesso errore. . .	49
10.1	Esempio di grafo bipartito con due cluster naturali e un “ponte” debole. Il vettore di Fiedler assegna componenti di segno opposto ai due cluster.	55
18.1	Architettura a blocchi della webapp.	84
19.1	Relazioni rilevanti del modello dati (semplificato).	87
B.1	Tempo totale della pipeline (scala logaritmica). Si nota la salita lineare in scala log fino a huge, con il salto previsto su superhuge dovuto alla dimensione del Phase A.	102
B.2	Confronto del tempo di Phase B con e senza decomposizione spettrale, sui profili medio-grandi.	103
B.3	Convergenza di ALNS sul profilo huge: il miglioramento si concentra nei primi 1000–2000 step e poi satura, tipico delle metaeuristiche.	104

Elenco delle tabelle

B.1	Tempi della pipeline end-to-end per profilo. Phase A include assegnazione docenti–classi e CP-SAT su matching. Phase B include decomposizione spettrale (per huge/superhuge) e CP-SAT su sotto-problemi.	101
B.2	Componenti dell’obiettivo SOFT di Phase A. glib_pen=0 significa che ogni docente ha ricevuto il giorno libero di prima preferenza (tranne in medium, dove 1–3 docenti hanno preso la seconda preferenza per overlap di vincoli).	102
B.3	Tempo del pre-check di Hall in millisecondi. Trascurabile su tutti i profili.	102
B.4	Speedup misurato della decomposizione spettrale su Phase B. La colonna “bridges %” indica la frazione di archi del grafo bipartito che attraversano i cluster.	103
B.5	Tempi totali con diverse combinazioni di metodi. Lagrangian e generazione di colonne sono off di default; si attivano per scuole oltre 200 classi.	104

Prologo: come nasce questo libro

“Tutto, Lucilio, ci viene da altri; soltanto il tempo è nostro.”

Lucio Anneo Seneca, *Epistulae morales ad Lucilium*, I, 1

Quando un dirigente scolastico entra in carica, una delle prime preoccupazioni che lo aspettano è l'orario delle lezioni. Comporlo non è un compito secondario, e nemmeno una pratica burocratica da delegare alla segreteria con un sospiro: significa decidere chi insegna a chi, in quale aula, in quale momento della giornata, conciliando le esigenze di centinaia di docenti, le ore curricolari di ciascuna disciplina, le incompatibilità logistiche delle aule, le richieste personali (un professore che chiede un giorno libero per motivi familiari, una classe che ha attività fissate al mercoledì mattina) e le costrizioni del contratto nazionale di lavoro. Il problema sembra subito complicato, e in effetti lo è: i matematici l'hanno classificato come NP-difficile, una sigla che vuole dire che non esiste un algoritmo veloce in grado di trovare la soluzione ottimale al crescere delle dimensioni della scuola. Eppure le scuole un orario lo producono, ogni anno, e per molti decenni questo è stato fatto a mano da segretari pazienti che dedicavano settimane a incastrare ore e cattedre. Il sistema software che descriviamo in queste pagine, π Tantum, è nato per dare loro uno strumento moderno, scritto pensando proprio alle scuole italiane e alle loro convenzioni; nelle pagine che seguono cercheremo di raccontare il metodo che vi sta dietro, gli algoritmi che lo rendono possibile, e l'interfaccia che permette di usarlo senza dover essere informatici.

Il nome è una piccola ammissione di affetto per la lettera greca π , le cui due gambe verticali ricordano le due “T” di *Tempus Tantum*, che a loro volta vengono dal verso seneciano riportato in epigrafe. Il senso del verso è tutto in quella parola: *tantum*, “soltanto”, “nient'altro che”. Il tempo è l'unica cosa che davvero ci appartiene, e in una scuola italiana il tempo è un bene quotidiano sempre scarso. Lo scarso bene comune che π Tantum prova a organizzare è proprio questo: i 1800 minuti settimanali che ogni classe ha a disposizione, e che vanno spesi nel modo migliore per chi insegna e per chi impara. Da qui anche la scelta editoriale di accompagnare il logo del software con un richiamo classico, e di intitolare i capitoli con qualche citazione presa dalla letteratura tecnica e divulgativa che ne ha ispirato lo sviluppo.

Questo libro è a sua volta nato in modo simile a un orario scolastico: per piccoli incrementi, in periodi diversi dell'anno, e con la convinzione che alcune cose meritino di essere scritte in forma di volume e non di README di poche righe. Quando il software ha cominciato a essere usato sul serio in qualche scuola, mi sono accorto che mancava un testo che mettesse insieme tre cose che fino ad allora viaggiavano separate: il contesto del problema (perché gli orari sono difficili da fare), il metodo che π Tantum adotta per affrontarlo (le tecniche algoritmiche che usa) e le indicazioni operative per chi lo deve usare ogni giorno (le tab dell'interfaccia, i casi d'uso ricorrenti, le insidie da evitare). Non era un volume da scrivere in un weekend, ma mi sono detto che valeva la pena: di libri che spiegano à la fois la teoria del scheduling e la pratica della scuola italiana ce ne sono pochi, e probabilmente nessuno scritto in italiano. Le pagine che seguono sono il tentativo di colmare quella lacuna, con tutta l'imperfezione che un tentativo di questo tipo comporta.

A chi è rivolto

Mi sono accorto, scrivendo, che le persone che potrebbero prendere in mano questo libro non sono di un solo tipo. La prima categoria è quella di chi π Tantum lo dovrebbe usare per davvero: il coordinatore d'orario di un liceo o di un istituto tecnico, il vicepresidente che sta valutando se adottare il software per il prossimo anno scolastico, l'animatore digitale di una scuola che vuole capire se vale la pena suggerirlo al collegio. A loro le parti I e II del libro dovrebbero parlare in un italiano leggibile, senza formule, e con casi d'uso che assomigliano a quelli che incontrano davvero a settembre o in corso d'anno. Per loro ho cercato, dove possibile,

di ridurre il jargon tecnico al minimo necessario, e quando proprio non si poteva evitare ho provato a spiegare il termine appena introdotto in una nota in linea o in un riquadro a margine.

C'è poi una seconda categoria: gli studenti universitari di informatica, di ricerca operativa, o di tecnologie dell'educazione che possono trovare nel timetabling un campo di applicazione interessante perché tocca temi che nei loro corsi vedono in modo astratto. Per loro le parti III e IV sono pensate come un corso semestrale informale: ogni tecnica algoritmica viene introdotta partendo dal problema concreto, accompagnata dalla sua storia (il paper seminale, l'idea originaria, l'autore), spiegata con un esempio giocattolo che si può seguire mentalmente, e poi generalizzata al caso reale. Mi sono sforzato di non saltare i passaggi intermedi, perché è proprio lì che la maggior parte dei manuali tecnici perde i lettori meno esperti.

Una terza categoria è quella dei ricercatori di scheduling che cercano i dettagli implementativi specifici: quali heuristic, quali parametri, quali decomposizioni, quali benchmark sui profili di dimensioni reali. Per loro, oltre alle parti III e IV, la parte V raccoglie i risultati misurati sui cinque profili di test in cui π Tantum suddivide l'universo delle scuole italiane (small, medium, big, huge, superhuge), e gli appendici tecnici spiegano le scelte architettoniche. Un'ultima categoria, infine, è quella degli sviluppatori che vogliono estendere il sistema, aggiungere un nuovo tipo di vincolo, un nuovo metodo metaeuristico, o una nuova vista nel frontend: per loro la parte IV documenta il modello dati, le API REST, i punti di estensione del codice e l'organizzazione del repository.

Quattro categorie diverse, dunque, con bisogni diversi. Una delle scommesse di questo libro è che si possa servire tutte e quattro *con lo stesso registro narrativo*, cambiando ciò di cui si parla ma non il modo in cui se ne parla. Vediamo se la scommessa regge.

Perché un libro, oggi

In un'epoca in cui la documentazione tecnica si è ridotta a issue tracker, README di poche righe e tutorial di YouTube, scrivere un volume di qualche centinaio di pagine può sembrare un'impresa anacronistica. Lo faccio per tre ragioni distinte, ognuna delle quali pesa abbastanza da giustificare la fatica.

La prima ragione è che il timetabling scolastico è sotto-documentato in italiano. Esistono ottimi libri di constraint programming generale, come il *Handbook of Constraint Programming* di Rossi, Van Beek e Walsh (Rossi et al. 2006); esistono ottimi libri di metaeuristiche, come l'*Handbook of Metaheuristics* di Glover e Kochenberger (Glover e Kochenberger 2003); esistono ottimi articoli di review specifici sul timetabling scolastico, come la *Survey of Automated Timetabling* di Schaerf (Schaerf 1999) o l'aggiornamento di Burke e Petrovic (Burke e Petrovic 2007). Manca, però, un volume in lingua italiana che ti accompagni dal problema concreto di una scuola del nostro paese fino alla riga di codice del solver, spiegando *perché* si fanno certe scelte e *come* si modellano vincoli che non appaiono nei benchmark accademici: gli indirizzi di studio italiani, la divisione delle classi di concorso, le compresenze obbligatorie nei tecnici, i gruppi articolati che attraversano le classi di base. Questo libro vorrebbe essere quel volume.

La seconda ragione è che i metodi sotto il cofano di π Tantum sono idee belle che meritano di essere raccontate. Il teorema di Hall del 1935 (Hall 1935), scritto su poche pagine eleganti del *Journal of the London Mathematical Society*, è ancora oggi il fondamento matematico del nostro controllo di fattibilità. Il vettore di Fiedler del 1973 (Fiedler 1973) permette di spezzare scuole enormi in cluster ben separati semplicemente guardando l'algebra del grafo bipartito che lega classi e docenti. Il *simulated annealing* di Kirkpatrick del 1983 (Kirkpatrick et al. 1983) importa nella combinatoria un'idea presa in prestito dalla metallurgia: il metallo che si raffredda lentamente trova una configurazione cristallina ordinata, e il software che “si raffredda lentamente” trova soluzioni di alta qualità in un mare combinatorio altrimenti intrattabile. Sono storie che si possono raccontare bene, e raccontarle in italiano, con la giusta dose di dettaglio tecnico ma senza pretese specialistiche, mi sembra una piccola operazione culturale che vale la pena tentare.

La terza ragione è personale. Dietro questo software c'è una storia di frustrazioni vissute e di soddisfazioni guadagnate poco a poco. C'è l'esperienza di vedere coordinatori d'orario di scuole vicine passare le notti di agosto a sistemare matrici in Excel; c'è la frustrazione di non riuscire a spiegare loro, in un linguaggio semplice, *perché* certe combinazioni che chiedevano di provare non potevano funzionare (Hall, ancora una volta); c'è il piacere di vedere apparire la prima soluzione completa dopo qualche minuto di simulated annealing su un istituto di novanta classi che fino a quel momento non aveva mai chiuso un orario in tempo. Non posso

raccontare quelle storie nel dettaglio dei nomi e delle scuole, per discrezione di chi mi ha aiutato, ma il libro ne è impregnato in tutte le sue pagine.

Come leggere questo libro

Il volume è diviso in sei parti che si possono affrontare in ordine o saltando avanti, a seconda dell'interesse del lettore. La prima parte contiene questo prologo, una descrizione formale del problema del timetabling scolastico e una panoramica di alto livello del sistema π Tantum: chi non si è mai occupato di scheduling può leggerla per intero in due o tre ore di lettura tranquilla, e arrivare alla fine con un'idea chiara di che cosa si parlerà. La seconda parte descrive l'orario reale visto dal lato organizzativo, la struttura dei dati, i casi d'uso, e fa il tour dell'interfaccia utente: è quella che il coordinatore d'orario consulterà più spesso quando inizierà ad usare il sistema.

La terza parte è il cuore tecnico. Un capitolo per ogni metodo algoritmico, con la stessa struttura narrativa costante: si parte da un problema concreto, si racconta la storia dell'idea (l'autore, il paper seminale, il contesto in cui è nata), si costruisce il metodo passo dopo passo spiegando ogni concetto prima di usarlo, si lavora un esempio in miniatura che si può seguire mentalmente, si generalizza al caso reale, si discutono i limiti e i casi in cui il metodo non funziona, si suggeriscono i due o tre riferimenti bibliografici essenziali per chi voglia approfondire. È un'organizzazione ripetitiva di proposito: una volta capito lo schema sul primo capitolo (CP-SAT), gli altri si leggono più in scioltezza.

La quarta parte raccoglie gli approfondimenti tecnici strutturali: l'architettura del software, il modello dati con tutte le sue tabelle e relazioni, il riferimento delle API REST, i punti di estensione per chi voglia modificare π Tantum. La quinta parte raccoglie i benchmark misurati sui cinque profili di test, le tabelle e i grafici dei tempi di esecuzione, e una sezione di *lessons learned* in cui provo a fare un bilancio onesto di quello che ha funzionato e di quello che non ha funzionato. La sesta e ultima parte contiene un glossario discorsivo di tutti i termini tecnici introdotti nel libro, gli indici (analitico e dei nomi), la bibliografia completa e un'appendice di sviluppo con FAQ, comandi utili e troubleshooting.

Una nota sui rimandi incrociati: ho cercato di metterne abbastanza da rendere la lettura non lineare possibile (si trovano in tutti i punti del tipo “come abbiamo visto al cap. 9”) ma senza esagerare al punto da costringere il lettore a navigare avanti e indietro per seguire il filo. Se incontrate un termine tecnico che non ricordate dove era stato definito, l'indice analitico in fondo dovrebbe rimandarvi alla pagina giusta.

Convenzioni tipografiche

Trovate sparsi nel testo dei piccoli riquadri colorati con funzioni diverse, che servono a sospendere temporaneamente il filo del discorso per dare una definizione precisa, proporre un esempio numerico, o avvertire di un errore comune. Sono pensati come momenti di pausa: il lettore può leggerli o saltarli senza perdere il filo principale del capitolo. I principali tipi sono il riquadro *Intuizione* (azzurro), che propone la spiegazione informale di un'idea prima della definizione formale; il riquadro *Definizione* (verde scuro), che enuncia in modo preciso un termine tecnico; il riquadro *Esempio* (oro caldo), che svolge un caso numerico concreto, di solito su una scuola in miniatura; il riquadro *Attenzione* (rosso), che segnala limiti, errori comuni e cose da non fare; il riquadro *Nota storica* (terra di Siena), che racconta l'origine di un'idea, le persone che l'hanno proposta, l'anno in cui è stata pubblicata; il riquadro *Per approfondire* (indaco profondo), che suggerisce due o tre riferimenti bibliografici essenziali a fine sezione; il riquadro *Caso di studio* (oro su Siena), che ricostruisce una situazione realistica anche se fittizia per discrezione.

Le citazioni bibliografiche usano lo stile autore-anno alla (Schaerf 1999): la lista completa dei riferimenti si trova in fondo al volume, in ordine alfabetico per autore. I termini tecnici principali entrano nell'indice analitico in coda al libro, anch'esso ordinato alfabeticamente. Le formule matematiche, quando appaiono, sono sempre accompagnate da una frase che le introduce e da una frase che le commenta, come consigliato dalla buona didattica matematica: nessuna formula deve apparire “nuda” davanti al lettore.

Ringraziamenti

Il metodo descritto in queste pagine, e l'applicazione che lo realizza, nascono dal confronto e dal lavoro comune di quattro persone che, in tempi e modi diversi, hanno riflettuto sul problema dell'orario scolastico. A Fernando Gargiulo, Giovanni Borghi, Matteo Mariani e Stefano Bertozzi va il riconoscimento delle idee originali su cui il progetto si fonda: senza le loro intuizioni, le loro discussioni e la loro pazienza nello scendere nei dettagli concreti delle scuole italiane, π Tantum non sarebbe quello che è.

A chi mi ha mostrato Excel pieni di colori e di celle gialle nelle calde mattine di agosto, a chi mi ha chiesto perché il sabato pomeriggio non si può fare educazione fisica quando le palestre sarebbero libere, a chi mi ha aiutato a capire la differenza fra un curriculum e un indirizzo, a chi mi ha prestato un manuale degli anni '90 sulla teoria dei matrimoni di Hall che mi ha riaperto l'interesse: grazie. Anche solo per avermi raccontato la storia di un orario particolarmente difficile da chiudere, o per avermi mostrato come la matrice 5-stati può essere controintuitiva fin che non si capisce la differenza fra PREFERRED ed ENFORCED. Senza quelle conversazioni questo libro sarebbe stato molto più astratto, e probabilmente inutile.

G. B., maggio 2026

1 Il problema dell'orario scolastico

“Educational timetabling is one of the most studied problems in operations research, yet remains one of the hardest in practice.”

Andrea Schaerf, *A survey of automated timetabling* (Schaerf 1999)

PRIMA di entrare nel merito di come π Tantum costruisce un orario, conviene fermarsi a guardare con attenzione il problema che si sta affrontando. Lo si può descrivere a parole in poche righe, ma chi ci ha provato sa che quella semplicità è ingannevole: si tratta di assegnare a ogni lezione settimanale uno slot di tempo e un'aula, in modo che nessun docente, nessuna classe e nessuna aula si trovino impegnati in due luoghi nello stesso istante, e che il monte ore di ogni materia in ogni classe sia rispettato. Detto così sembra una pratica routinaria, di quelle che si fanno con un foglio di calcolo in mezza giornata. Nella realtà, una scuola di dimensioni medie produce un'esplosione combinatoria di possibilità che renderebbe impraticabile qualunque tentativo di enumerazione esaustiva, e le richieste personali dei docenti, le incompatibilità delle aule, le esigenze didattiche delle classi, i vincoli sindacali e le delibere collegiali si stratificano gli uni sugli altri fino a rendere il problema, in linea di principio, intrattabile. Vale dunque la pena dedicare un capitolo intero a comprenderne la natura prima di affrontarlo, perché una cattiva comprensione del problema porta inevitabilmente a una cattiva soluzione.

Lo scopo di queste pagine è dunque triplice. In primo luogo, si vuole dare un'idea quantitativa della dimensione del problema, perché i numeri aiutano a capire perché “provarci” non è una strategia praticabile. In secondo luogo, si vogliono distinguere con cura i diversi tipi di vincolo (rigidi, morbidi, preferenze, obbligazioni), perché è proprio questa distinzione che rende possibile costruire un sistema utilizzabile. In terzo luogo, si vuole tracciare una piccola storia dei metodi che la comunità di ricerca operativa ha sviluppato negli ultimi cinquant'anni, in modo che il lettore possa collocare π Tantum in una tradizione e capire quali idee gli sono state ereditate dai predecessori.

1.1 Quante combinazioni possibili

Si consideri una scuola italiana di dimensione media: trenta classi, sessanta docenti, sei giorni di lezione, sei ore al giorno. Ogni classe ha tipicamente una trentina di ore settimanali, e l'insieme delle lezioni da collocare è dunque dell'ordine di novecento. Per ognuna di queste lezioni è necessario decidere quale materia mettere in quale slot e quale docente farla insegnare. Anche limitando le scelte ai soli docenti compatibili (cioè quelli abilitati alla materia in questione e con sufficienti ore residue), per ogni lezione restano in media una decina di possibilità. Lo spazio delle soluzioni cresce dunque come dieci elevato a novecento, un numero che si scrive con un uno seguito da novecento zeri e che non ha alcun significato fisico immaginabile. Per dare un termine di paragone, si stima che il numero di atomi nell'universo osservabile sia attorno a dieci elevato alla ottantesima potenza: lo spazio delle assegnazioni di una scuola media è quindi 10^{820} volte più grande dell'universo intero. Esplorarlo per enumerazione cieca non è soltanto impraticabile sui computer di oggi: è fisicamente impossibile.

Questo è il motivo per cui non basta “provare le combinazioni”, anche con un computer veloce. Servono algoritmi che, anziché esaminare ogni possibilità, scartino sotto-spazi enormi senza neppure visitarli, sulla base di deduzioni logiche elementari. È quello che fa la *propagazione di vincoli*, una tecnica che vedremo nel dettaglio nel capitolo dedicato a CP-SAT (cap. 9). In alternativa, si possono accettare soluzioni *non garantite ottime*, ottenute attraverso processi di ricerca locale che migliorano una soluzione iniziale per piccoli passi: è l'approccio delle metaeuristiche, di cui parlerà il cap. 11. Si può anche spezzare il problema in sotto-problemi più piccoli e risolverli quasi indipendentemente, ricucendoli alla fine: è il principio della decomposizione, che

π Tantum applica in diverse forme (cap. 10 per la decomposizione spettrale, cap. 13 per quella lagrangiana). Tutte queste tecniche convivono e collaborano nella pipeline del sistema, che combina ciò che ognuna fa meglio.

1.2 Vincoli rigidi e vincoli morbidi

Una difficoltà almeno altrettanto grave della dimensione combinatoria deriva dalla *natura* dei vincoli che intervengono. Non sono tutti uguali: alcuni sono assoluti, non negoziabili, e una soluzione che li violi non è neppure una soluzione, mentre altri esprimono preferenze, desideri o abitudini di servizio, e si vorrebbero rispettare ma su di essi si può transigere se le circostanze lo richiedono. Ignorare questa distinzione e trattare tutto come ugualmente inderogabile produce, come si può immaginare, sistemi infattibili al primo conflitto fra preferenze, e l'utente si ritrova davanti a un solver che dichiara “nessuna soluzione esistente” senza la minima indicazione di cosa fare.

In π Tantum si è adottata, fin dall'inizio, una distinzione canonica della letteratura (Carter e Laporte 1998; Schaefer 1999) fra vincoli rigidi e morbidi, raffinata in cinque livelli distinti rappresentati nell'interfaccia da una piccola palette di colori. Il livello *HARD* (rosso) rappresenta i vincoli inderogabili: i due esempi più immediati sono “due classi non possono trovarsi nella stessa aula nello stesso slot” e “un docente non può tenere due lezioni contemporaneamente”. Il livello *SOFT* (giallo) esprime una preferenza negativa: si vorrebbe evitare quella configurazione (per esempio una sesta ora di matematica al sabato pomeriggio) ma se serve per chiudere l'orario la si può accettare, pagando un costo che entra nella funzione obiettivo. Il livello *PREFERRED* (blu) è simmetrico rispetto al precedente: indica una preferenza positiva, una configurazione che si vorrebbe vedere ricorrere (per esempio il laboratorio di fisica in due ore consecutive) e che riduce il costo se viene rispettata. Il livello *ENFORCED* (verde scuro) è ancora più rigido di *HARD* ma in senso opposto: si dichiara che quel particolare slot *deve* essere usato, non è ammessa l'alternativa. Infine il livello *ALLOWED* (verde chiaro) rappresenta la situazione neutra di default: nessuna preferenza in nessun senso.

Questa stratificazione, che a prima vista può apparire sovrabbondante, si rivela quasi sempre necessaria nella pratica. Una scuola italiana ha decine di vincoli che appartengono naturalmente al livello *SOFT* (le preferenze mattutine dei docenti, la concentrazione delle materie teoriche nelle prime ore, l'evitamento dei buchi nelle classi), accanto a quelli inderogabili. Fingere che siano tutti *HARD* vorrebbe dire produrre sistemi cronicamente infattibili; fingere che siano tutti *SOFT* vorrebbe dire produrre sistemi che violano regole contrattuali. La distinzione precisa è ciò che permette di ottenere soluzioni che funzionano davvero.

1.3 Una breve storia delle idee

Il problema del timetabling è oggetto di studio sistematico fin dagli anni sessanta del Novecento, quando le scuole più grandi cominciarono a cercare metodi sistematici per evitare i lunghi pomeriggi di settembre passati a comporre l'orario con scotch e cartoncini ritagliati. I primi tentativi furono basati su algoritmi di colorazione di grafi, perché una versione semplificata del problema (assegnare slot in modo che lezioni in conflitto ricevano slot diversi) si lascia ricondurre con eleganza al colorare i nodi di un grafo in modo che nodi adiacenti abbiano colori distinti. Il problema della colorazione dei grafi è stato uno dei primi classificati come *NP-completo* da Karp nel 1972, e per riduzione anche il timetabling fu identificato come problema intrinsecamente difficile. La sigla NP-completo, per chi non è abituato al gergo della complessità computazionale, vuole dire che non si conosce alcun algoritmo capace di risolvere *tutte* le istanze del problema in tempo crescente polinomialmente con la loro dimensione, e che questa difficoltà è condivisa con un'intera famiglia di problemi ritenuti intrattabili in generale (Garey e Johnson 1979).

Fortunatamente le istanze *reali* di una scuola italiana non sono i casi peggiori previsti dalla teoria: la struttura dei vincoli è molto regolare (gli indirizzi sono pochi, le classi di concorso sono ancora più poche, le aule specializzate sono distribuite in modo prevedibile), e questa regolarità si lascia sfruttare dagli algoritmi moderni in modo molto efficace. Negli anni ottanta e novanta, alle tecniche di backtracking si aggiunsero le *metaeuristiche*: il *simulated annealing* di Kirkpatrick, Gelatt e Vecchi del 1983 (Kirkpatrick et al. 1983), ispirato alla metallurgia, e il *tabu search* di Glover del 1986 (Glover 1986), che introdusse l'idea di una memoria delle mosse recenti per guidare la ricerca. Negli anni duemila i *constraint solver* acquisirono maturità industriale:

prodotti come GeCode, ECLiPSe e poi CP-SAT di Google permisero di affrontare istanze prima inarrivabili. Pisinger e Ropke nel 2006 introdussero la *Adaptive Large Neighborhood Search* (Ropke e Pisinger 2006), che combina distruzioni e ricostruzioni intelligenti su porzioni significative della soluzione. La conferenza biennale PATAT (*Practice and Theory of Automated Timetabling*), attiva dal 1995, divenne il riferimento mondiale per i ricercatori del settore (PATAT 1995–2024).

π Tantum si colloca in questa tradizione. Non inventa metodi nuovi: combina con cura quelli esistenti in una pipeline orchestrata che cerca di sfruttare i punti di forza di ciascuno. CP-SAT viene usato come motore principale per la fase di soddisfacimento dei vincoli rigidi, sia nella fase di matching docente–classe sia nella collocazione settimanale. La decomposizione spettrale, ispirata alle intuizioni di Fiedler del 1973 (Fiedler 1973), viene usata per spezzare le scuole più grandi in cluster naturali, in modo che CP-SAT possa lavorare su sotto-problemi trattabili. Le metaeuristiche raffinano la soluzione ottenuta, riducendo le violazioni dei vincoli morbidi. Il pre-controllo di Hall, basato su un teorema del 1935 che sembra antico ma che resta sorprendentemente attuale, verifica in pochi millisecondi se il problema è almeno formalmente fattibile, evitando di lanciare il solver su istanze infeasibili dall'inizio.

1.4 Le quattro famiglie di approcci

La letteratura sul timetabling distingue tradizionalmente quattro famiglie di approcci, ciascuna con i propri vantaggi e i propri limiti. La prima è la *programmazione a vincoli*, che modella il problema come insieme di variabili con domini finiti e di vincoli che le legano, lasciando poi che un solver come CP-SAT si occupi di trovare un'assegnazione che li soddisfi tutti. Il vantaggio di questo approccio è la sua estrema flessibilità: aggiungere un vincolo nuovo significa aggiungere una riga di codice e nessuna riprogettazione strutturale del modello. Lo svantaggio è che la scalabilità diventa critica oltre la soglia di una settantina di classi: il numero di variabili booleane equivalenti che il solver deve gestire cresce e i tempi di risoluzione esplodono.

La seconda famiglia è la *programmazione lineare intera* (MILP, *Mixed Integer Linear Programming*), che formula il problema come sistema di disuguaglianze lineari con variabili binarie e lo risolve con prodotti commerciali come Gurobi o CPLEX, oppure con alternative open-source come HiGHS. Il vantaggio teorico è che si possono ottenere certificati di ottimalità, almeno su istanze piccole. Lo svantaggio pratico è che la formulazione MILP del timetabling tende a richiedere milioni di variabili binarie e diventa pesante. π Tantum non utilizza MILP nella pipeline principale ma prevede uno scheletro di generazione di colonne (*column generation*) che si appoggia a un master LP risolto con HiGHS, applicabile alle istanze più grandi (cap. 13).

La terza famiglia è quella delle *metaeuristiche*. Si parte da una soluzione iniziale, ottenuta con un metodo qualunque (anche un greedy molto rudimentale), e la si migliora iterativamente attraverso operatori di ricerca locale: scambi di lezioni, perturbazioni casuali, accettazioni parziali di mosse peggioranti. Il vantaggio è la scalabilità: le metaeuristiche restituiscono *sempre* una soluzione, anche su istanze enormi, e in tempi controllabili. Lo svantaggio è che non si ha alcuna garanzia di ottimalità: la soluzione finale è di solito buona, ma raramente la migliore possibile.

La quarta famiglia è quella delle *decomposizioni*. Invece di affrontare il problema monolitico, lo si spezza in sotto-problemi più piccoli, lo si risolve indipendentemente, e lo si ricuce alla fine. La decomposizione può essere *spettrale*, basata sulla struttura algebrica del grafo bipartito che lega classi e docenti; *lagrangiana*, basata sul rilassamento dei vincoli che legano i sotto-problemi e sulla loro penalizzazione progressiva nell'obiettivo; *di Dantzig-Wolfe*, basata sulla generazione iterativa di nuove variabili nel master problem. π Tantum implementa tutte e tre, con preferenze diverse a seconda della dimensione dell'istanza.

1.5 La filosofia di π Tantum

La scelta progettuale fondamentale di π Tantum è di *non scegliere* fra le quattro famiglie sopra descritte, ma di combinarle in una pipeline a quattro fasi orchestrate. Ogni fase utilizza il metodo più appropriato al suo compito e produce un risultato che la fase successiva può usare come punto di partenza. Si tratta di un approccio non particolarmente originale dal punto di vista della ricerca (è ciò che molti sistemi commerciali e accademici fanno), ma di un approccio efficace, e che ha il pregio di rendere il sistema modulare: si può sostituire una fase con un'implementazione alternativa senza dover toccare le altre.

La prima fase, denominata *Phase A* nella terminologia del progetto, si occupa dell'*assegnamento*: per ogni classe e per ogni materia, decidere quale docente la insegnerà e quante ore. È un problema di matching nel grafo bipartito classi–docenti, vincolato dalle compatibilità fra classi di concorso e materie, dai monte ore disponibili per ciascun docente, e dalle indisponibilità per giorno. Si risolve con un algoritmo greedy seguito da un passo CP-SAT che ottimizza la distribuzione. Prima di lanciare la fase, π Tantum esegue il *pre-controllo di Hall*, che verifica in pochi millisecondi se il problema è almeno formalmente fattibile dal punto di vista dei matching: se non lo è, il sistema lo segnala all'utente con l'indicazione del sottoinsieme problematico, evitando un'attesa inutile su CP-SAT.

La seconda fase, *Phase B*, si occupa del *timetabling vero e proprio*: collocare ogni lezione in uno slot della settimana. Per le scuole grandi, prima di lanciare CP-SAT si applica la *decomposizione spettrale*, che spezza il problema in cluster relativamente indipendenti. CP-SAT risolve allora ogni cluster in poche decine di secondi, e un passo finale di ricucitura armonizza le interfacce fra cluster.

La terza fase, *Phase C*, si occupa del *raffinamento*: ALNS, simulated annealing, tabu search e iterated local search lavorano sulla soluzione di Phase B per ridurre ulteriormente il valore della funzione obiettivo SOFT. Si tratta di una fase opzionale ma quasi sempre attivata, che riduce il costo SOFT di un dieci-quindici per cento in pochi minuti aggiuntivi.

La quarta fase, *Phase D*, si occupa dell'*assegnazione delle aule*: dato che ogni lezione ha già uno slot e una classe, resta da decidere in quale aula si terrà, rispettando i tipi (lab, palestra, aula normale), le capienze, le preferenze classe-aula e le compatibilità materia-aula. Si tratta di un problema CP-SAT relativamente piccolo, che si risolve in pochi secondi.

L'intera pipeline, dal momento in cui l'utente preme il bottone “Pipeline completa” al momento in cui il sistema mostra l'orario finito, dura tipicamente un minuto su una scuola piccola, due minuti su una media, dieci minuti su una grande, e fino a un quarto d'ora sulle più grandi testate. Sono numeri che andranno verificati nel cap. B, dove riporto i risultati sperimentali ottenuti su una macchina consumer con i cinque profili di test.

1.6 Le peculiarità del caso italiano

Vale la pena chiudere questo capitolo con un'osservazione che tornerà spesso nelle pagine successive: i benchmark internazionali del timetabling, codificati in formati standard come ITC-2007, ITC-2011 e XHSTT (McCollum et al. 2010), sono importanti per il confronto fra ricercatori, ma non catturano alcune peculiarità del sistema scolastico italiano che invece π Tantum deve modellare con precisione. La prima peculiarità sono gli *indirizzi di studio* (Liceo Classico, Liceo Scientifico, Linguistico, Istituto Tecnico nelle sue varie articolazioni, Istituti Professionali), che hanno monte ore diversi anche per la stessa materia. Una classe “3A scientifico” fa matematica con un orario diverso da una “3A classico” o da una “3A ITIS Informatica”, nonostante che il nome della materia sia lo stesso. La seconda peculiarità sono le *classi di concorso*, codificate con sigle alfanumeriche (A027, A019, eccetera), che limitano quali docenti possono insegnare quali materie e in quali indirizzi. Un docente di A027 può insegnare matematica e fisica al biennio scientifico ma non lettere al classico, e queste compatibilità sono codificate in un grande database ministeriale che π Tantum sintetizza nelle sue tabelle.

La terza peculiarità è la presenza diffusa di *compresenze*: in molti indirizzi tecnici, le ore di laboratorio richiedono che due docenti diversi siano nella stessa classe nello stesso slot (l'insegnante teorico e l'insegnante tecnico-pratico). La compresenza è un vincolo HARD non banale, perché non si limita a chiedere “due docenti” ma chiede “proprio quei due docenti, contemporaneamente”. La quarta peculiarità sono i *gruppi articolati*: un sottoinsieme di studenti, eventualmente proveniente da più classi di base, che si riuniscono in slot dedicati per attività specifiche (la seconda lingua straniera del linguistico, la lezione di religione cattolica con l'alternativa per chi non si avvale, i corsi di recupero in itinere). La quinta peculiarità sono i *vincoli sindacali*: il giorno libero contrattuale CCNL, il monte ore massimo settimanale, la priorità basata sul punteggio di graduatoria. Sono vincoli HARD non negoziabili, ai quali si aggiungono i vincoli logici espressi via DSL dall'utente.

Tutte queste peculiarità sono codificate come entità o vincoli di prima classe nel modello dati di π Tantum (cap. ??), che è stato progettato a partire da esse e non come adattamento di un modello accademico generico.

È forse il contributo più distintivo del progetto: non un solver più veloce, ma un solver che parla la lingua del sistema scolastico italiano.

Detto questo, abbiamo dato uno sguardo d'insieme al problema. Nel prossimo capitolo descriveremo π Tantum dall'alto: quali sono i suoi tre layer (solver, web UI, prototipi legacy), come comunicano fra loro, e quale flusso di lavoro quotidiano un coordinatore d'orario segue per usarlo. Affronteremo poi, nei capitoli successivi, l'orario visto come oggetto strutturale (le entità e le relazioni che codifica), l'interfaccia utente, i casi d'uso ricorrenti, e infine ciascuno dei metodi algoritmici che fanno funzionare il sistema.

2 Una panoramica di π Tantum

“A program does what you tell it to do, not what you want it to do.”

adagio popolare della programmazione

PRIMA di entrare nei dettagli delle singole tecniche algoritmiche e nelle pieghe dell'interfaccia utente, conviene osservare π Tantum dall'alto: che cosa fa, come è organizzato, da quali pezzi è composto, e con quale flusso un coordinatore d'orario lo utilizza per costruire o per mantenere l'orario annuale di una scuola. Una panoramica di questo tipo è utile per due ragioni. In primo luogo, permette al lettore di costruirsi una mappa mentale del sistema, dentro la quale potrà poi collocare le informazioni più tecniche dei capitoli successivi. In secondo luogo, fornisce un riferimento al quale tornare quando si vorrà verificare se un certo concetto appartiene al solver, al backend, al frontend o alla pipeline complessiva.

2.1 Cosa fa il sistema, in tre frasi

π Tantum è un sistema software che, ricevuti in input l'anagrafica di una scuola (docenti, classi, materie, aule, indirizzi di studio, gruppi articolati, studenti), il monte ore richiesto per ciascun indirizzo e per ciascuna materia, le preferenze e le indisponibilità, produce in output un orario settimanale completo che soddisfa tutti i vincoli rigidi e minimizza la somma pesata delle violazioni dei vincoli morbidi. Una volta prodotto l'orario, lo stesso sistema lo gestisce in corso d'anno: consente modifiche manuali con controllo di fattibilità in tempo reale, supporta la gestione delle assenze e delle supplenze quotidiane, e offre viste alternative per docente, per classe, per aula, per slot. Tutte le operazioni avvengono attraverso un'interfaccia web a sedici tab, e tutti i dati sono persistiti in un singolo file SQLite che può essere esportato, importato, e versionato come qualunque altro file.

2.2 Tre layer in un unico repository

Il sistema è organizzato in tre strati distinti, ciascuno con il proprio dominio di responsabilità, e tutti contenuti in un unico repository Git per semplicità di sviluppo e di distribuzione. Questa scelta architetturale è meditata: se i tre layer fossero in repository separati, il loro co-versionamento richiederebbe attenzioni che, per un progetto di questa scala, non aggiungono valore. Mantenere tutto in un solo repository semplifica anche la pipeline di integrazione continua e l'esperienza dello sviluppatore, che può navigare l'intero codice da un'unica radice.

Il primo layer, denominato *solver layer*, risiede nella cartella `experiments/` e contiene tutto il codice algoritmico in Python puro. È uno strato deliberatamente ignaro del database e dell'interfaccia utente: legge un dizionario di input e produce un dizionario di output, secondo un'interfaccia ben definita. Le sue componenti principali sono il modulo `cpsat_v2_assignment.py` per la fase di assegnamento, il modulo `cpsat_v2_timetable.py` per la fase di timetabling vero e proprio, il modulo `decomposition_spectral_v2.py` per il clustering spettrale, il modulo `metaheuristics.py` che raggruppa le quattro varianti classiche (LNS, SA, TS, ILS), e i moduli `alns.py`, `vns.py`, `hall_check.py`, `lagrangian.py`, `column_generation.py`, `classroom_assignment.py` che implementano le tecniche aggiuntive descritte nei capitoli successivi.

Il secondo layer, denominato *web UI layer*, risiede nella cartella `webui/` e contiene il backend FastAPI e il frontend SvelteKit. Il backend espone tutte le operazioni del solver come API REST e gestisce la persistenza

in SQLite; il frontend fornisce l'interfaccia web a sedici tab che copre l'intero ciclo di vita di un orario, dall'inserimento dei dati anagrafici alla generazione finale, all'editing manuale, alla gestione operativa delle assenze e delle supplenze. La comunicazione fra frontend e backend avviene via HTTP/JSON; lo storage è un singolo file webui/data/timetable.db; le migrazioni del database sono idempotenti ed eseguite automaticamente a ogni avvio del backend.

Il terzo layer, denominato *legacy layer*, risiede nella cartella `schedule/` e contiene gli script monolitici che hanno fatto da prototipo per il solver attuale. Sono mantenuti per riferimento storico e per la riproducibilità dei benchmark contro le versioni precedenti, ma non fanno più parte del flusso operativo. Un nuovo utente del sistema può ignorarli completamente.

2.3 Diagramma a blocchi del sistema

Il diagramma in figura 2.1 riassume in forma grafica le relazioni fra i tre layer. Il frontend SvelteKit, in cima, parla al backend FastAPI attraverso chiamate REST sotto il path `/api/`, instradate dal proxy Vite in modalità di sviluppo o dal server di produzione in modalità di rilascio. Il backend, a sua volta, persiste i dati su SQLite via SQLAlchemy 2.0 e invoca il solver layer attraverso un convertitore esplicito (`engine_io.py`) che traduce le entità relazionali del database in dizionari Python adatti al solver, e viceversa. Il solver, infine, usa la libreria OR-Tools di Google per le risoluzioni CP-SAT.

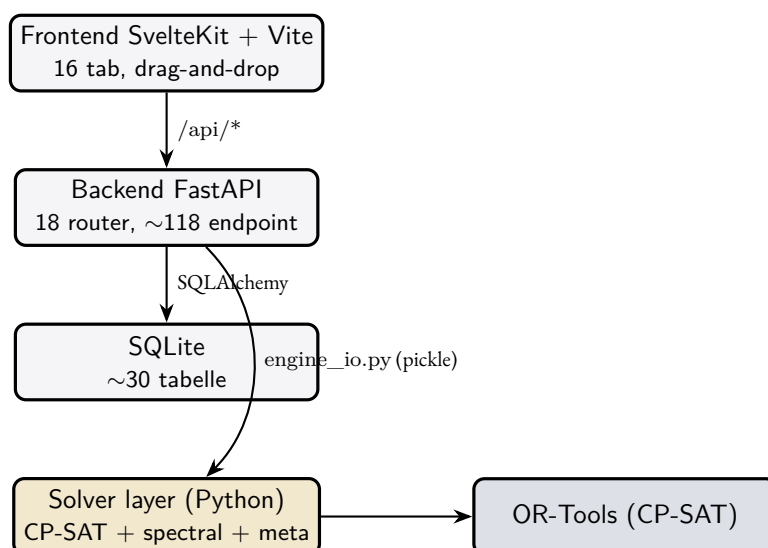


Figura 2.1: Architettura a blocchi di π Tantum: il frontend parla al backend tramite REST, il backend persiste su SQLite e dialoga con il solver attraverso un convertitore esplicito basato su pickle, il solver invoca OR-Tools per le risoluzioni CP-SAT.

Si noti che non vi è alcuna connessione diretta fra frontend e solver: ogni richiesta dell'utente attraversa sempre il backend, che la valida, la persiste, ed eventualmente la inoltra al solver. Questa scelta garantisce che lo stato del sistema sia sempre coerente e che ogni operazione lasci una traccia nel database, utile sia per l'audit sia per la possibilità di rieseguire le pipeline su dati storici.

2.4 Il flusso d'uso quotidiano

Un coordinatore d'orario alla prima esecuzione del sistema segue tipicamente il flusso che ora descriviamo. Si tratta di una sequenza ricorrente, che si ripete grosso modo identica all'inizio di ogni anno scolastico, e che diventa familiare dopo le prime due o tre esperienze. Le variazioni nascono soprattutto dalla quantità e dal tipo di vincoli particolari che la scuola desidera esprimere; il flusso di base resta lo stesso.

Il primo passo è l'*importazione dell'anagrafica*. L'utente può partire da un template Excel o CSV (un foglio per ogni entità: docenti, classi, materie, aule, indirizzi, studenti, gruppi articolati), che il sistema fornisce come download dalla pagina *Importa*. Può anche partire, soprattutto per fini di prova, da un dataset mock generato deterministicamente da π Tantum stesso, in una delle cinque taglie predefinite (small, medium, big, huge, superhuge). Una volta importati i dati, l'utente li verifica e li completa nei tab specifici (Docenti, Classi, Materie, eccetera), correggendo errori, aggiungendo indisponibilità tramite le matrici 5-stati, settando preferenze, eccetera.

Il secondo passo è l'*aggiunta dei vincoli logici* specifici. Mentre i vincoli di base (non sovrapposizione, copertura, classi di concorso) sono codificati nel sistema e non richiedono intervento dell'utente, i vincoli particolari della singola scuola ("il prof Bianchi non lavora sabato", "la palestra è occupata dall'altro istituto il martedì mattina", "in 1A alternativa religione cattolica una volta al mese al giovedì") vengono espressi tramite il DSL generico, descritto in dettaglio nel cap. 17. La documentazione è accessibile dall'interfaccia stessa con un bottone di aiuto, e ogni vincolo è validato in tempo reale prima di essere salvato.

Il terzo passo è il *pre-controllo di Hall*. Prima di lanciare la pipeline vera e propria, conviene verificare che il problema sia almeno formalmente fattibile: la copertura delle ore è sostenuta dalla disponibilità dei docenti? In caso contrario, il sistema mostra il sotto-insieme di classi o materie che presentano un deficit, e l'utente può intervenire prima di consumare tempo di calcolo. Il pre-controllo è accessibile da tre punti distinti dell'interfaccia (un bottone inline nella card di Phase A, una riga nel pannello *tecniche avanzate*, e una sezione del tab Diagnostica), tutti che invocano lo stesso modulo `hall_check.py` e che producono lo stesso output, descritto nel cap. 12.

Il quarto passo è il *lancio della pipeline*. Dal tab Workflow, l'utente preme il bottone "Pipeline completa" e il sistema esegue in sequenza Phase A (assegnazione), Phase B (decomposizione + CP-SAT), Phase C (metaeuristiche) e Phase D (aule). Il log di esecuzione viene mostrato in tempo reale attraverso un canale Server-Sent Events; l'utente può seguire l'avanzamento, leggere i messaggi del solver, ed eventualmente fermare la pipeline se decide di intervenire. Al termine, il sistema mostra il valore della funzione obiettivo, le eventuali violazioni dei vincoli morbidi, e una panoramica dei tempi di esecuzione.

Il quinto passo è l'*editing manuale*. Dal tab Orario, l'utente può trascinare le lezioni da uno slot all'altro per correggere a mano dettagli che il solver non aveva considerato o che non corrispondono ai desideri ultimi del coordinatore. Mentre l'utente trascina, il sistema verifica in tempo reale se la mossa è fattibile dal punto di vista dei vincoli rigidi, e mostra come cambia la funzione obiettivo morbida. Tutto questo avviene senza dover rilanciare il solver: è una valutazione differenziale (*delta-evaluation*) che sfrutta la struttura della funzione obiettivo per ricalcolare solo le componenti che cambiano, in tempo costante.

Il sesto e ultimo passo, ricorrente nel corso dell'anno, è la *gestione delle assenze e delle supplenze*. Dal tab omonimo, l'utente segna i docenti assenti per ogni giornata, vede in tempo reale quali classi rimangono scoperte, e trascina i docenti disponibili come supplenti. Il sistema calcola automaticamente le compatibilità (chi può sostituire chi, considerando classi di concorso e disponibilità) e mostra le opzioni con un'evidenziazione cromatica. Una cella rossa significa "ancora scoperta"; una cella verde significa "coperta".

Caso di studio

Liceo "Galileo", 32 classi, settembre. Il vicepresidente parte da un import Excel con sessantasette docenti, trentadue classi, undici indirizzi (quattro scientifico, tre classico, due linguistico, due scienze applicate). Aggiunge quattro vincoli logici specifici (un docente che non lavora sabato, il laboratorio di fisica disponibile solo nel triennio, due compresenze fisse, una classe quinta che il martedì mattina ha attività esterna) e lancia la pipeline. In nove minuti π Tantum produce un orario completo che soddisfa tutti i vincoli rigidi e minimizza una funzione obiettivo composta da sette termini SOFT pesati. Un'occhiata al log mostra che Phase A ha impiegato dodici secondi, Phase B quattro minuti (la scuola è stata spezzata in cinque cluster da sei–otto classi), Phase C tre minuti di LNS più un minuto di simulated annealing, Phase D quaranta secondi. L'obiettivo finale è a 11.2 punti, con zero violazioni rigide e ventitré violazioni morbide, di cui quattordici sono "ore isolate" di docente che il vicepresidente decide di accettare consapevolmente.

2.5 Quando il sistema non basta da solo

Sarebbe disonesto presentare π Tantum come una soluzione completa che risolve tutti i casi senza intervento umano. Vi sono almeno tre situazioni ricorrenti in cui il sistema da solo non basta, e nelle quali la collaborazione fra software e coordinatore diventa indispensabile.

La prima situazione è quella dei *dati di input incompleti o contraddittori*. Se un docente è abilitato alla classe di concorso A027 ma il suo monte ore residuo totale supera la somma delle ore di matematica disponibili, il pre-controllo di Hall fallisce, e nessun algoritmo del mondo può costruire un orario fattibile a partire da quel dato. È un problema dei dati, non del solver: π Tantum non può “inventare” classi che non esistono o ore che non ci sono. La soluzione richiede l'intervento del coordinatore, che dovrà o assumere un docente in più, o ridurre il monte ore, o convocare delle supplenze straordinarie.

La seconda situazione è quella dei *vincoli sociali impliciti*. “La professoressa Rossi vorrebbe avere tutte le sue ore al mattino” non è un dato che il sistema possa indovinare: deve essere esplicitato come matrice di preferenza con celle pomeridiane in SOFT. Se la richiesta non viene codificata, il solver non la conosce, e produrrà una soluzione che la ignora completamente. La regola pratica è: ogni richiesta personale o sociale deve essere tradotta in un vincolo prima di essere considerata.

La terza situazione è quella dell'*iterazione mancante con il committente*. La prima soluzione prodotta dal sistema produce sempre, in pratica, almeno qualche richiesta di modifica da parte del collegio dei docenti. Per questo π Tantum dispone di un editor manuale e di un sistema di vincoli logici incrementale: la pipeline non va rilanciata da zero ogni volta. Una richiesta del tipo “spostare l'inglese di 2B dal venerdì al giovedì” si gestisce con un drag-and-drop, e il sistema verifica in tempo reale che la mossa sia fattibile e ne calcola il delta sull'obiettivo. Ignorare questa modalità iterativa porta a un'esperienza frustrante per tutti.

2.6 Le tecnologie scelte e perché

L'ultima sezione di questa panoramica documenta le tecnologie sulle quali π Tantum è costruito e le ragioni della loro scelta. Tutte le tecnologie selezionate sono *open source* e prive di vincoli di licenza commerciale: nessuna scuola dovrebbe rinunciare al sistema per motivi economici, e nessun coordinatore dovrebbe trovarsi nella posizione di dover convincere il dirigente ad acquistare una licenza.

Il linguaggio principale è Python 3.11 o superiore. Si tratta del linguaggio dominante per il calcolo scientifico e per la modellazione a vincoli, e dispone di librerie mature per ogni esigenza che il progetto può avere. Le sue performance sono accettabili per questo dominio applicativo, perché la parte computazionalmente pesante (la risoluzione CP-SAT) è implementata in C++ all'interno della libreria OR-Tools, mentre il codice Python si limita a guidarla.

Il solver è OR-Tools di Google, in particolare il modulo CP-SAT. Si tratta di uno dei migliori solver al mondo per problemi combinatori di medie e grandi dimensioni, gratuito, con API Python pulite e documentate, e con prestazioni allineate ai prodotti commerciali più costosi. Ha vinto le edizioni 2018, 2019, 2020 e 2021 della MiniZinc Challenge, la competizione internazionale fra constraint solver. La sua maturità e la sua diffusione lo rendono una scelta naturale per chiunque voglia costruire un sistema di scheduling oggi.

Il backend è FastAPI, che fornisce un framework HTTP asincrono moderno con validazione automatica via Pydantic, generazione automatica della documentazione OpenAPI/Swagger, e ottime performance senza sacrificare la chiarezza del codice. La sua adozione semplifica drasticamente la scrittura e il mantenimento delle API REST.

Lo storage è SQLite, scelto per la sua zero-configuration: è un singolo file, non richiede un servizio dedicato, ed è più che sufficiente per i carichi di lavoro tipici di una scuola, anche grande. Le migrazioni sono scritte come funzioni Python idempotenti che vengono eseguite a ogni avvio del backend, evitando la necessità di un sistema di migration esterno come Alembic. Si tratta di una scelta consapevole che privilegia la semplicità operativa rispetto alla pulizia formale.

Il frontend è SvelteKit con Tailwind CSS, una combinazione che produce bundle finali piccoli, hot-reload veloce in sviluppo, e codice leggibile. La scelta di Svelte rispetto a React o Vue è legata alla sua filosofia “compiler-first”: il codice scritto viene tradotto in JavaScript vanilla ottimizzato, riducendo l'overhead a

runtime. Vite, infine, fornisce il dev-server con proxy nativo verso il backend e un ciclo di compilazione di pochi millisecondi.

Le alternative considerate sono state numerose: Django o Flask invece di FastAPI, React o Vue invece di Svelte, PostgreSQL invece di SQLite, Gurobi commerciale invece di OR-Tools, MiniZinc come linguaggio di modellazione invece di Python diretto. Tutte sono state scartate per ragioni di leggerezza, costo, facilità di deployment in scuole italiane con infrastruttura modesta. La regola implicita è stata: ridurre al minimo le dipendenze e i punti di fallimento, preferire sempre la soluzione più semplice se sufficiente al caso d'uso.

Cosa abbiamo visto in questo capitolo

Abbiamo guardato π Tantum dall'alto, individuando i suoi tre layer (solver, web UI, prototipi legacy), descrivendo come comunicano fra loro, e tracciando il flusso d'uso quotidiano di un coordinatore d'orario, dall'importazione dell'anagrafica alla gestione delle supplenze. Abbiamo anche documentato le situazioni in cui il sistema da solo non basta e l'intervento umano resta indispensabile, e abbiamo elencato le tecnologie sulle quali il progetto è costruito, con le ragioni della loro scelta. Da qui in poi, i capitoli successivi entrano nei dettagli: il prossimo descrive l'orario come oggetto strutturale, le entità coinvolte e le relazioni che le legano; quelli successivi descrivono l'interfaccia utente e i casi d'uso ricorrenti; i capitoli della parte centrale del libro descrivono ognuno dei metodi algoritmici che fanno funzionare il sistema; gli ultimi capitoli affrontano l'architettura software, il modello dati, le API REST, i benchmark misurati, e una sezione di lessons learned.

3 L'orario come oggetto strutturale

“First, solve the problem. Then, write the code.”

John Johnson

PRIMA di descrivere come π Tantum *costruisce* un orario, conviene fermarsi a riflettere su *che cosa sia* un orario, visto come oggetto strutturato. Un orario non è infatti una semplice tabella di celle riempite, ma un sistema di entità interconnesse da relazioni precise, ciascuna con i propri attributi e i propri vincoli. Comprendere questa struttura prima di affrontare gli algoritmi che la trattano consente di seguire con maggiore agio i capitoli successivi e di apprezzare le ragioni che hanno guidato la progettazione del modello dati.

Le entità coinvolte sono fondamentalmente sette: docenti, classi, materie, aule, indirizzi di studio, studenti e gruppi articolati. Ognuna di esse possiede una propria anagrafica, una serie di attributi che la caratterizzano, e una collezione di relazioni con le altre entità. Le pagine che seguono descrivono ciascuna di queste entità e le relazioni principali che le legano, soffermandosi sulle peculiarità del sistema scolastico italiano che il modello dati di π Tantum codifica esplicitamente.

3.1 I docenti

Il docente è l'entità forse più ricca di attributi del sistema, perché su di esso convergono numerose costrizioni contrattuali e organizzative. La sua anagrafica include il cognome, il nome, ed eventualmente un nickname per le visualizzazioni compatte. A ogni docente sono associate una o più *classi di concorso*, codificate con sigle alfanumeriche del tipo A027 o A019, che determinano quali materie egli sia abilitato a insegnare. Il monte ore massimo settimanale, tipicamente diciotto per il ruolo ordinario, costituisce un vincolo HARD invalicabile dal solver. Un giorno libero contrattuale, di norma sabato o lunedì a seconda della scuola, va parimenti rispettato in modo inderogabile.

Accanto a questi attributi obbligatori, il docente possiede una *matrice di disponibilità* di sei righe per sei colonne (sei giorni della settimana per sei ore al giorno), in cui ogni cella esprime uno dei cinque stati della palette di vincoli: ALLOWED per la disponibilità normale, SOFT per una preferenza negativa (“preferirei non lavorare in questo slot”), HARD per un'indisponibilità assoluta, PREFERRED per una preferenza positiva (“vorrei lavorare in questo slot”), ENFORCED per un'obbligazione (“devo lavorare in questo slot”). La matrice viene compilata dall'utente nell'interfaccia del tab Docenti, e ogni cella si modifica con un singolo click.

Il docente possiede anche una *matrice di preferenza per le aule*, sempre a cinque stati, che indica in quali aule egli preferisca insegnare; un *punteggio di graduatoria* usato dal solver per ordinare le scelte secondo il criterio dell'anzianità; e un'eventuale lista di *tre giorni preferiti per essere libero*, ordinati per priorità, che il sistema cerca di soddisfare in ordine. Ogni informazione di questo elenco si traduce in un contributo specifico alla funzione obiettivo o in un vincolo HARD per il solver.

3.2 Le classi

La classe rappresenta l'unità didattica di base. Ogni classe ha un nome (per esempio “3A scientifico”), un anno di corso da uno a cinque, un indirizzo di studio (che rimanda all'entità Indirizzo descritta più avanti), un numero di studenti attesi, e una *home class*, ovvero l'aula in cui di norma le sue lezioni si svolgono. Anche

la classe possiede una propria matrice di disponibilità a cinque stati e a sei per sei celle, che codifica eventuali indisponibilità (per esempio l'attività di gita una volta al mese) o preferenze.

Sulla classe è possibile impostare un attributo `max_hours_per_day`, che specifica quante ore al giorno la classe possa al massimo svolgere. Il valore di default è sei, ma alcune scuole adottano cinque o, in casi particolari, sette ore al giorno; il vincolo è HARD se specificato. Un peso SOFT separato controlla quanto si voglia evitare la sesta ora come ultima ora del giorno: alcuni istituti privilegiano la quinta ora come termine canonico, relegando la sesta a slot da usare solo se necessario. Anche per le classi sono ammessi tre giorni preferiti per essere libere, con la stessa semantica usata per i docenti.

3.3 Le materie

La materia è descritta dal suo nome (Matematica, Italiano, Storia, Religione, Scienze Motorie, e così via) e da una *matrice di compatibilità con le aule*. Questa matrice, anch'essa a cinque stati, esprime per ogni coppia materia-aula il livello di adeguatezza: alcune materie richiedono aule specializzate (la fisica deve stare nel laboratorio di fisica, la chimica nel laboratorio di chimica, le scienze motorie in palestra), e per esse l'incompatibilità con le aule normali è un vincolo HARD; altre materie sono più flessibili e ammettono qualsiasi aula con preferenze sfumate.

A ogni materia è associato un peso che ne quantifica il “carico cognitivo”, usato dal solver per evitare di mettere in sequenza due materie particolarmente impegnative. Una matrice ulteriore, denominata *subject-group-weight matrix*, codifica quali classi di concorso siano abilitate a insegnare la materia in questione, raffinando la corrispondenza grezza nome-materia. È un'informazione necessaria perché nel sistema scolastico italiano la relazione fra materia e classe di concorso non è biunivoca: “Matematica” può essere insegnata da docenti di A026 (matematica nel biennio scientifico) o di A027 (matematica e fisica nel triennio scientifico) a seconda dell'anno di corso e dell'indirizzo.

3.4 Le aule

L'aula è descritta dal proprio nome (per esempio “Aula 12” o “Lab Fisica 1”), dal proprio *tipo* (aula normale, laboratorio di fisica, laboratorio di chimica, laboratorio di informatica, palestra, aula magna, e così via), dalla propria capienza, e da un flag *multi-class* che indica se l'aula possa ospitare più classi contemporaneamente, come accade tipicamente per l'aula magna o per i laboratori molto grandi. Anche per le aule esiste una matrice di disponibilità a cinque stati, che codifica eventuali periodi di manutenzione o di occupazione da parte di altre attività.

Ogni aula ha inoltre una propria *matrice di preferenze per le classi*, che indica quanto sia preferibile ospitare ciascuna classe specifica. È lo strumento con cui si codifica, per esempio, “l'aula 17 è la home della 3A” (preferenza alta per la 3A) o “l'aula 12 è troppo lontana dal laboratorio di chimica per le classi del triennio scientifico” (preferenza bassa per quelle classi). Si aggiunge infine una lista di *tag* liberi che il coordinatore può associare all'aula: stringhe come “matematica”, “piano_terra”, “accessibile_disabili” che servono come metadato per filtri e per vincoli logici DSL espressi in modo compatto.

3.5 Gli indirizzi di studio

L'indirizzo, che gli ordinamenti italiani designano anche con i sinonimi “curriculum” o “ordinamento”, rappresenta il programma di studi di una particolare articolazione scolastica: Liceo Classico, Liceo Scientifico, Liceo Linguistico, Istituto Tecnico Industriale (con le sue varie specializzazioni), Istituto Professionale per i Servizi, e così via. Per ogni anno di corso (dal primo al quinto) e per ogni materia, l'indirizzo specifica il *numero di ore settimanali* che la classe di quell'anno deve dedicare alla materia in questione. Sono questi monte ore che, sommati sulle classi che adottano l'indirizzo, generano il volume totale di lezioni che la pipeline deve collocare.

L'indirizzo può avere associati propri vincoli logici specifici, espressi nel DSL generico, che si applicano a tutte le classi dell'indirizzo. Esempi tipici sono “in prima classico la matematica non si tiene mai dopo la quarta ora” o “nel triennio scientifico le ore di laboratorio devono essere consecutive”. Questi vincoli si combinano con quelli specifici delle singole classi e con quelli generali della scuola, formando il livello più esterno della gerarchia di vincoli che il solver rispetta.

3.6 Gli studenti

Lo studente è un'entità anagrafica che appartiene di norma a una classe di base, ma che può anche essere membro di uno o più *gruppi articolati*, come la seconda lingua straniera del linguistico, la lezione di religione cattolica con l'alternativa per chi non si avvale, o un corso di recupero pomeridiano. A ogni studente possono essere associati dei *tag* liberi, stringhe come BES (Bisogni Educativi Speciali), DSA (Disturbi Specifici dell'Apprendimento), debito__matematica__4 per indicare un debito formativo specifico, o studente__atleta per segnalare un percorso scolastico-sportivo. I tag sono metadato passivo che il sistema usa per filtri e per costruire vincoli logici espressi in modo conciso, senza modificare la struttura dei gruppi.

3.7 I gruppi articolati

Il gruppo articolato è l'entità che permette di rappresentare attività didattiche che attraversano i confini delle classi di base. Si tratta di insiemi di studenti, anche provenienti da classi diverse, che si incontrano in slot dedicati per un'attività comune. Esempi tipici sono il gruppo della seconda lingua straniera nel liceo linguistico, in cui gli studenti delle classi 4A e 4B si dividono fra francese, tedesco e spagnolo; oppure il gruppo IRC nei licei, in cui gli studenti che si avvalgono dell'insegnamento di religione cattolica si riuniscono in un'aula mentre quelli che hanno scelto l'alternativa vanno in un'altra; oppure ancora i corsi di recupero in itinere che il collegio docenti decide di attivare a metà anno per gli studenti con difficoltà in una particolare materia.

I gruppi articolati richiedono uno slot dedicato e un'aula specifica, e il loro funzionamento deve essere coordinato con l'orario delle classi di base dei loro membri: nessuno studente può essere richiesto in due luoghi contemporaneamente, e il sistema verifica questa coerenza come vincolo HARD. La semantica è quella che π Tantum chiama *Tipo C*, dal nome interno della prima implementazione: i membri possono provenire da qualunque combinazione di classi, e il gruppo non è mai un sottoinsieme rigido di una sola classe.

3.8 Le relazioni fra le entità

Le entità sopra descritte sono collegate da una rete di relazioni che costituisce il vero substrato del timetabling. La relazione fondamentale è la *cattedra*, che collega un docente a una classe e a una materia con l'indicazione del numero di ore settimanali. È l'unità elementare con cui si esprime “il professor Borghi insegna quattro ore di matematica alla 3A”. Una cattedra esiste soltanto se le classi di concorso del docente sono compatibili con la materia, e se il monte ore della classe per quella materia richiede effettivamente la copertura.

La *compresenza* è una relazione che lega due o più docenti alla stessa classe, alla stessa materia, e allo stesso slot. È un vincolo tipico delle materie tecnico-pratiche degli istituti tecnici, dove un insegnante teorico e un insegnante tecnico-pratico devono essere contemporaneamente in classe. La compresenza è un vincolo HARD non banale, perché non si limita a chiedere “due docenti” ma chiede “proprio quei due docenti, contemporaneamente, nella stessa aula”.

La *lezione* è l'istanza concreta che la pipeline produce: una specifica cattedra (o compresenza) collocata in uno slot specifico della settimana, con un'aula assegnata. Una soluzione completa di π Tantum non è altro che l'insieme delle lezioni che ricoprono interamente le ore richieste dalle cattedre, rispettando tutti i vincoli HARD e minimizzando il valore della funzione obiettivo SOFT.

L'*indisponibilità HARD* non è una relazione fra entità in senso stretto, ma una proprietà delle matrici sei per sei dei docenti, delle classi e delle aule. Tecnicamente è implementata come una collezione di celle marcate, e i

vincoli che ne derivano vengono iniettati direttamente nel modello CP-SAT al momento della risoluzione. Il *vincolo logico DSL*, infine, è un'espressione del DSL generico (descritto nel cap. 17) che lega entità in qualunque combinazione, ed è il meccanismo principale con cui l'utente esprime regole specifiche della sua scuola che il modello dati pre-confezionato non cattura.

3.9 I vincoli ricorrenti del caso italiano

Alcuni vincoli ritornano in tutte le scuole italiane, anche se con sfumature diverse, e meritano di essere descritti come categoria. Il vincolo di *copertura* richiede che ogni materia riceva il proprio monte ore in ogni classe, e costituisce la finalità stessa della pipeline: senza copertura completa non c'è neppure un orario. Il vincolo di *non sovrapposizione* richiede che docente, classe e aula siano impegnati in al più una lezione per slot, ed è naturalmente HARD. Il vincolo di *compatibilità per classe di concorso* richiede che soltanto i docenti abilitati possano insegnare una determinata materia.

I vincoli di *compatibilità aula-materia* sono per lo piuttosto rigidi quando l'attività richiede strumentazione specifica (la fisica nel laboratorio di fisica, le scienze motorie in palestra, il disegno tecnico nell'aula da disegno), ma diventano preferenze morbide nei casi più flessibili. Il *giorno libero contrattuale* è un vincolo HARD per contratto collettivo nazionale, che ogni docente ha diritto di vedere rispettato in uno specifico giorno della settimana.

I vincoli sull'*evitamento dei buchi* (le ore vuote nel mezzo della giornata di una classe o di un docente) sono HARD nelle scuole più rigide e SOFT in quelle più flessibili, e π Tantum li configura per default come HARD per le classi (perché una classe con buchi crea problemi organizzativi reali) e SOFT per i docenti (perché i buchi nei docenti sono fastidiosi ma non impediscono il funzionamento della scuola). I vincoli sulla *sesta ora*, sulla *distribuzione settimanale di una stessa materia*, e sulle *coppie di ore consecutive* per le materie che lo richiedono completano il quadro dei vincoli ricorrenti.

3.10 Cosa rende peculiare il sistema italiano

Il caso italiano si distingue dai benchmark accademici internazionali, codificati in formati standard come ITC-2007, ITC-2011 e XHSTT, per l'esistenza degli indirizzi di studio come entità di prim'ordine e la loro diversificazione di monte ore anche per materie nominalmente identiche. Si distingue anche per l'esistenza delle classi di concorso esplicite e gerarchiche, che vincolano in modo fine la corrispondenza fra docenti e materie. Si distingue ancora per la presenza diffusa di compresenze come vincoli HARD ricorrenti, per i gruppi articolati che attraversano le classi di base, e per i vincoli sindacali derivanti dal contratto collettivo nazionale.

Caso di studio

IIS “Galileo Ferraris” di Torino, ventotto classi distribuite su tre indirizzi (ITIS Informatica, ITIS Elettronica, IPS Servizi Commerciali). Le tre articolazioni hanno monte ore diversi anche per materie comuni come matematica e italiano. I docenti di matematica delle classi di concorso A026 e A027 coprono tutte e tre le articolazioni; i docenti specifici, come quello di Sistemi e Reti per l'indirizzo Informatica, sono limitati a una sola articolazione. Le compresenze sono numerose, in particolare fra matematica e laboratorio di informatica nell'indirizzo Informatica e fra fisica e laboratorio di fisica nell'indirizzo Elettronica. Le settimane di stage in azienda si concentrano nelle ultime settimane dell'anno scolastico, e per esse π Tantum gestisce la situazione attraverso un vincolo logico DSL che svuota i pomeriggi di martedì e giovedì delle classi quinte nel periodo aprile-maggio.

Tutte queste peculiarità sono codificate come entità di prima classe o come vincoli espliciti nel modello dati di π Tantum, descritto in dettaglio nel cap. ???. Il progetto è stato disegnato a partire da queste specificità, e non come adattamento postumo di un sistema accademico generico. È forse il contributo più distintivo della filosofia su cui il software è costruito: non un solver più veloce in assoluto, ma un solver che parla la lingua del sistema scolastico italiano e che si lascia istruire con il vocabolario di chi quel sistema lo conosce.

Detto tutto questo, abbiamo descritto l'orario come oggetto strutturato. Nei capitoli successivi vedremo come il sistema rappresenta queste entità internamente, come l'utente le manipola attraverso l'interfaccia web, come la pipeline le elabora, e come i vari metodi algoritmici si combinano per produrre l'orario finito.

4 Installazione

Guida cross-platform per installare ed eseguire piTantum su Windows, Linux e macOS. La versione *di riferimento* di questa guida è docs/installation.md nel repo (Markdown, sempre aggiornato); qui se ne riproduce il contenuto in forma compatta per il manuale PDF. In caso di divergenza, fa fede il file Markdown.

Architettura del lancio. Il progetto è composto da due processi:

- **Backend:** uvicorn che serve l'app FastAPI in webui/backend sulla porta 8000.
- **Frontend:** server di sviluppo vite che serve l'app SvelteKit in webui/frontend sulla porta 5173.

Uno script unico (start.bat su Windows, start.sh su Linux/macOS) li avvia entrambi. Al primo lancio crea la *venv* Python, installa i requirements, fa npm install; ai lanci successivi parte in pochi secondi.

4.1 Windows

Prerequisiti.

- Python ≥ 3.11 – installer ufficiale: <https://www.python.org/downloads/windows/>
- Node.js ≥ 20 LTS – installer ufficiale: <https://nodejs.org/en/download> (scegliere “LTS”).
- Git – installer ufficiale: <https://git-scm.com/download/win>.

Note importanti.

- Sull’installer Python, **spuntare** “Add python.exe to PATH”. Senza, lo script non trova il comando python.
- Riavviare il terminale dopo le installazioni: il PATH non si aggiorna nelle finestre già aperte.

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
webui\start.bat
```

Lo script apre due finestre cmd (“piTantum – backend” e “piTantum – frontend”). Aprire il browser su <http://localhost:5173>.

Stop. Chiudere le due finestre cmd, oppure Ctrl+C in ciascuna.

Troubleshooting Windows.

- “python non riconosciuto”: l’installer Python non è stato eseguito con “Add to PATH”. Reinstallare oppure aggiungere C:\Users\tu\AppData\Local\Programs\Python\Python311 al PATH.
- Porta 8000 occupata: netstat -ano | findstr :8000 + taskkill /PID <pid> /F.
- Antivirus blocca start.bat: aggiungere alle eccezioni di Windows Defender.

4.2 Linux (Ubuntu, Debian, Fedora, Arch)

Prerequisiti. Python ≥ 3.11 , Node ≥ 20 LTS, Git, build-essential.

Ubuntu / Debian.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nodejs npm \
    git build-essential
```

I repo apt di Ubuntu 22.04 / Debian 12 hanno Node 12-18, troppo vecchio per SvelteKit 2 (richiede 20+). In quel caso usare **nvm**:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install --lts && nvm use --lts
```

Fedora.

```
sudo dnf install -y python3 python3-virtualenv nodejs npm git \
    gcc make
```

Arch / Manjaro.

```
sudo pacman -S python python-pip nodejs npm git base-devel
```

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
chmod +x webui/start.sh webui/stop.sh
./webui/start.sh # background; log in webui/logs/
./webui/start.sh --foreground # log live in console (Ctrl+C ferma)
```

Stop.

```
./webui/stop.sh
```

Troubleshooting Linux.

- Porta in uso: `sudo lsof -i :8000 + kill <pid>` (eventualmente `kill -9 <pid>`).
- python3 mancante su distro minimal: usare l'eseguibile chiamato `python (3.x)`.
- WSL2: `start.sh` funziona normalmente. Aprire il browser dal lato Windows su `http://localhost:5173`; WSL2 inoltra le porte automaticamente.

4.3 macOS (Intel + Apple Silicon)

Prerequisiti.

- Xcode Command Line Tools: `xcode-select --install`.
- Homebrew: vedere <https://brew.sh/>.
- Python ≥ 3.11 , Node ≥ 20 LTS, Git: tutti via Homebrew.

Installazione.

```
# Homebrew (se non presente)
/bin/bash -c "$(curl -fsSL
  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Toolchain
brew install python@3.12 node git

# Verifica
python3 --version # >= 3.11
node --version # >= v20
```

Note Apple Silicon.

- ortools dalla v9.7 spedisce wheel ARM64 nativi: Rosetta non serve.
- Se Python è installato da python.org invece che Homebrew, scegliere l'installer "macOS 64-bit universal2".

Clone + lancio + stop. Identico a Linux.

Troubleshooting macOS.

- Gatekeeper blocca start.sh: `xattr -d com.apple.quarantine webui/start.sh webui/stop.sh`.
- ortools senza wheel ARM (versioni vecchie): `pip install --upgrade ortools` dentro la venv del backend.
- brew non in PATH: eval `"$(/opt/homebrew/bin/brew shellenv)"` in `~/zprofile`.

4.4 Verifica installazione

Dopo il primo lancio andato a buon fine:

```
python --version # >= 3.11 (Linux/macOS: spesso python3)
node --version # >= v20
npm --version # >= 10

# Backend health
curl http://127.0.0.1:8000/api/health
# atteso: {"status":"ok","name":"pitantum","version":"0.1.0"}

# Async layer (Section 2.5 P2)
curl http://127.0.0.1:8000/api/health/async
# atteso: {"status":"ok","async":true,"dialect":"sqlite"}

# Frontend
curl -I http://127.0.0.1:5173
# atteso: HTTP/1.1 200 OK
```


4.5 Aggiornamento

Quando si fa `git pull` per portare a casa nuovi commit:

```
cd /path/to/timetable
git pull origin main

# se sono cambiate dipendenze Python:
webui/backend/.venv/bin/pip install -r webui/backend/requirements.txt
# (Windows: webui\backend\.venv\Scripts\pip install ...)

# se sono cambiate dipendenze frontend:
cd webui/frontend && npm install && cd ../..

# se ci sono migration DB:
cd webui/backend && ../.venv/bin/alembic upgrade head && cd ../..
# (Windows: webui\backend\.venv\Scripts\alembic upgrade head)
```

In pratica `start.sh/start.bat` rilanciato dopo un `git pull` gestisce automaticamente `pip install` e `npm install` quando servono. Le migration DB sono coperte da `__apply__lightweight__migrations()` allo startup come `safety-net` (vedi capitolo *Modello dati*); per la via canonica usare `alembic upgrade head`.

Dopo un aggiornamento backend, fare un **hard reload** del browser (Ctrl+Shift+R) per pulire chunk JavaScript cached.

4.6 Disinstallazione

```
cd /path/to/timetable
./webui/stop.sh # Linux/macOS
# (Windows: chiudere le finestre cmd di start.bat)

rm -rf webui/backend/.venv # ~150 MB
rm -rf webui/frontend/node_modules # ~400 MB
rm -rf webui/data/timetable.db # DATI UTENTE (salvarli prima!)
```

Per rimuovere completamente, cancellare la cartella `timetable/`. Python, Node, Git restano installati nel sistema; rimuoverli dal gestore di pacchetti solo se non servono ad altri progetti.

5 Vincoli

5.1 I 5 stati / 4 kind

Il sistema usa una tassonomia uniforme di 5 *stati* sia per le matrici di disponibilità (cella per cella) sia per le preferenze di aula. I vincoli logici disgiuntivi (DNF) hanno 4 *kind* (ALLOWED non si applica qui).

Stato	Colore	Semantica
ALLOWED	verde chiaro	default (solo grid materia↔aula e docente↔aula); nessuna contribuzione all'obiettivo
SOFT	giallo	penalità positiva quando lo slot è usato (+penalty aggiunto all'obiettivo, che è minimizzato)
HARD	rosso	il solver deve evitare questo slot, oppure il vincolo logico DEVE essere soddisfatto
PREFERRED	blu	bonus negativo quando lo slot è usato / il vincolo logico è soddisfatto (-bonus riduce l'obiettivo)
ENFORCED	verde scuro	il solver DEVE far accadere questo slot / soddisfare il vincolo con presence required

5.2 Matrici di disponibilità 5-stati

Il ciclo cliccando una cella è:

free → soft → hard → preferred → enforced → free

Le celle gialle e blu mostrano un input numerico per editare la penalità (positiva sul giallo, negativa sul blu, sign-clamp automatico al salvataggio).

Sincronizzazione free_day. Quando il docente ha free_day settato, tutte e 6 le ore di quel giorno appaiono HARD nella matrice automaticamente. Se l'utente edita la matrice manualmente e tutte le ore di un giorno diventano HARD, il backend deduce il free_day; viceversa cambiare il free_day sostituisce le auto-cells.

Mapping CP-SAT. In optimization.py::_availability_constraints ogni tabella si traduce in: <entity>_hard (set di tuple, forbidden assignment), <entity>_soft (dict tuple→penalty signed), <entity>_enforced (set di tuple, presence required nel solver).

5.3 Vincoli logici (DNF)

5.3.1 Sintassi

```

expr := or_expr
or_expr := and_expr ('OR' and_expr)*
and_expr := not_expr ('AND' not_expr)*
not_expr := ('NOT' | 'mai') not_expr | atom
atom := slot | predicate | '(' expr ')'
slot := <giorno><ora>
predicate:= <kind> ':' <name> ('@' <giorno> <ora>)?
kind := aula | materia | classe | gruppo | docente | prof
giorno := lun|mar|mer|gio|ven|sab (anche lunedì|...|sabato,
        monday|...|saturday)
ora := 1..6 (ordinale, 1=08:00) | 8..13 (assoluta)

```

Sono accettati & (AND), | (OR), ! (NOT). mai è alias di NOT.

5.3.2 Predicate atoms

```

aula:LabFisica -- l'entita' usa l'aula a qualunque slot
aula:LabFisica@mar -- il martedì a qualunque ora
aula:LabFisica@mar4 -- martedì 4a ora (= 11:00)
materia:Religione@lun8 -- lezione di religione lunedì alle 8
classe:1A_Scientifico -- riferimento alla classe
gruppo:IRC -- riferimento al gruppo

```

I literal predicate vengono parsati e persistiti correttamente. Il solver li tratta otticamente per ora; l'integrazione strong con la fase classroom assignment è pianificata.

5.3.3 Esempi pratici

```

% Per docenti
lun8 AND lun9
(lun8 AND lun9) OR (mar8 AND mar9)
gio11 OR gio12 OR gio13
NOT (mer3 AND mer4)
mai aula:LabFisica@mar
mai materia:Religione@lun8

% Per classi
mer4 AND mer5 AND mer6
mai aula:Palestra@gio
aula:LabInformatica@mar OR aula:LabInformatica@gio

% Per aule
gio4 AND gio5 AND gio6
mai materia:Religione
gruppo:IRC OR gruppo:Alternativa

```

5.3.4 Mapping CP-SAT

Per ogni regola con DNF [clause1, clause2, ...]:

- **HARD**: il solver impone almeno una clausola fully active.
- **SOFT**: se nessuna clausola è fully active, paga +soft_penalty.
- **PREFERRED**: se almeno una clausola è fully active, ottiene +soft_penalty (negativo, quindi reduce dell'obiettivo).
- **ENFORCED**: come HARD ma con presence required – almeno uno degli slot della DNF deve coincidere con un'ora di lezione attiva per quell'entità.

5.4 Vincoli HARD a toggle (per classe)

In school_classes ci sono 7 boolean strutturali: hard_entry_at_8, hard_exit_after_12, hard_no_holes, hard_dual_math, hard_dual_italian, hard_motorie_pairs, hard_max_6_per_day. Il soft_minimize_sixth_weight regola il peso della 6a ora.

5.5 Conflict detection

Il tab *Vincoli* ha un bottone "Cerca conflitti" (GET /api/monitor/conflicts) che segnala:

- **matrix_hard_enforced** – stessa cella marcata sia HARD sia ENFORCED.
- **enforced_on_free_day** – cella ENFORCED del docente nel suo free_day.
- **logical_unsatisfiable** – vincolo logico HARD/ENFORCED la cui DNF non è soddisfacibile dato l'insieme degli HARD slot dell'owner.

5.6 Tag aule – etichette libere

Per esprimere "questa aula ha il proiettore", "questa aula è adatta alla matematica perché ha la lavagna grande", "questa aula è una sala riunioni", il sistema offre i **tag delle aule**: etichette libere che il coordinatore attacca dalla scheda Aule. Ogni aula può avere quanti tag vuoi; ogni tag può essere riutilizzato su più aule.

I tag sono utilissimi nei vincoli: invece di scrivere "solo queste tre aule precise" (lista che andrebbe aggiornata ogni volta che le aule cambiano), scrivi "aule taggate matematica" e il vincolo segue automaticamente le etichettature correnti.

Esempio: vogliamo che la matematica si tenga solo in aule taggate matematica. Una volta etichettate le aule giuste dalla scheda Aule, il vincolo nel DSL generico è:

```
forall l in lessons where l.subject == Matematica:  
    "matematica" in l.classroom.tags
```

Il sistema ha anche una serie di tag automatici prodotti dalla generazione mock (per esempio le aule home class ricevono il tag dell'indirizzo della classe assegnata). Per dettagli vedi il modulo webui/backend/mock_classrooms.py.

5.7 Preferenze di giorno libero del docente

I docenti hanno diritto contrattuale a un certo numero di giorni liberi a settimana (tipicamente uno per il CCNL italiano). π Tantum gestisce questo aspetto in modo flessibile attraverso tre slot di preferenza ordinati:

- **1^a preferenza** – il giorno libero ideale del docente (es. “sabato”).
- **2^a preferenza** – il giorno di ripiego se il primo non è possibile (es. “mercoledì”).
- **3^a preferenza** – l’ultima alternativa (es. “lunedì”).

Per ciascuna preferenza il docente (o il coordinatore) sceglie se è **HARD** (assoluta) o **SOFT** con un peso. Più un campo **required_free_days_count** indica *quanti* giorni liberi servono in totale (di default 1 per CCNL).

Esempio: un docente con preferenze [sabato HARD, mercoledì SOFT peso 50] e count=1 vorrà sicuramente sabato libero, e in secondo ordine il mercoledì se il sabato non è fattibile.

5.8 Preferenze di giorno libero della classe

Anche le classi possono avere giorni preferiti di non-attività. La struttura è analoga a quella dei docenti: tre slot ordinati con flag HARD/SOFT, più un **required_free_days_count**.

In aggiunta le classi hanno il campo **max_hours_per_day** (default 5) che impone un tetto HARD al numero di ore in un giorno. Riducendolo (a 4 per esempio) si ottengono giornate più leggere — ma se il monte ore della classe è alto bisognerà estendere il calendario settimanale al sabato.

6 Workflow di ottimizzazione

6.1 Schema delle fasi

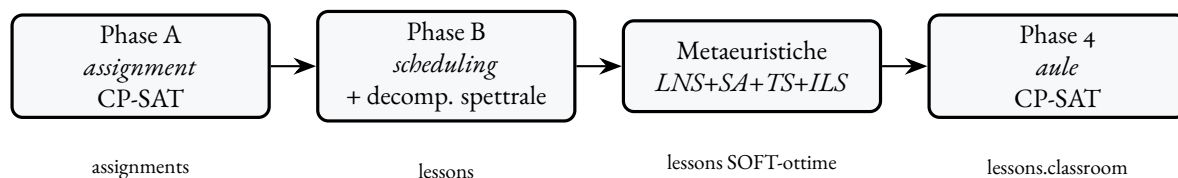


Figura 6.1: Pipeline a 4 fasi.

6.2 Phase A: assegnazione (cattedre)

Modulo. `experiments/cpsat_v2_assignment.py`.

Lancio. `POST /api/optimize/assignment` con `{time_limit_s, workers, log}`.

Input. L'insieme di `(class, subject, hours_per_week)` letto da `class_subjects` e/o `curriculum_subject_hours` + le abilitazioni docente in `teacher_subjects` + la classe-di-concorso preference in `subject_group_weights`.

Output. La tabella `assignments` (`teacher x class x subject` con `hours`). Problema di covering: ogni `(class, subject)` riceve un docente abilitato, rispettando `max-hours` per docente, classi-di-concorso e `teacher_compatible_classes`.

6.3 Phase B: scheduling

Modulo. `experiments/cpsat_v2_timetable.py` + `experiments/decomposition_spectral_v2.py`.

Decomposizione spettrale. Per istanze grandi, il problema CP-SAT integrale è intrattabile. La pipeline calcola un grafo di co-occupazione fra docenti, ne fa un'embedding spettrale via Laplaciano normalizzato e usa K-means su quell'embedding per partizionare in k cluster *loosely-coupled*. Ogni cluster diventa una sub-istanza CP-SAT indipendente; un secondo step "bridges" risolve solo i bordi; un terzo step "ricucitura" fa full-CPSAT con tutto il problema partendo dalla soluzione stitched.

6.4 Metaeuristiche

Modulo. experiments/metaheuristics.py. Cascata di:

- **LNS** – destroy-and-repair su finestre (teacher, day) o (class, day); freeze del 60-70% delle lezioni e CP-SAT-resolve del resto.
- **SA** (Simulated Annealing) – single-swap accettati con Metropolis a T decrescente con fattore α .
- **TS** (Tabu Search) – best-improvement local search con tabu list (default size 80).
- **ILS** (Iterated Local Search) – alterna n cicli di TS con *kick* perturbativi (random 4-opt fra giorni).

Tutti contribuiscono al SOFT score: holes per teacher, isolated 1- and 5-hour days, weekly distribution, dual-hour pairs, no 6th hour, banded preferences su materie. Il SOFT include la contribuzione delle matrici di disponibilità SOFT/PREFERRED e dei logical SOFT/PREFERRED.

6.5 Phase 4: assegnazione aule

Modulo. experiments/classroom_assignment.py. Input: la tabella lessons (già temporalmente fissata) + le classroom rules (kind, capacity, multi_class, preferenze materia↔aula, classe↔aula, docente↔aula). Output: lessons.classroom_name popolato.

6.6 Drag-and-drop con preview live

Trascinando una lezione su un altro slot la UI fa due chiamate:

1. POST /api/schedule/move-preview – il backend simula la mossa per tutte le 36 (day, hour) e per ognuna restituisce:
 - ok con $\text{delta_soft} \leq 0$ – mossa accettabile (verde se $\text{delta} < 0$, verde chiaro se $\text{delta} = 0$).
 - soft_worse – mossa accettabile ma peggiora il SOFT (giallo).
 - hard_violation – vincolo HARD violato (rosso).
 - noop – slot di origine.
2. L'utente dropa, la UI invia PUT /api/schedule/move-lesson che valida e persiste.

6.6.1 Room-follows-lesson

Il move-preview NON considera l'aula nel calcolo HARD. Se la nuova posizione è compatibile con docente / classe ma l'aula attuale è già occupata o HARD-unavailable in destination, il backend:

1. Sposta la lezione.
2. Lascia l'aula libera sul nuovo slot solo se non c'è conflitto.
3. Se c'è conflitto, rimuove l'aula dalla lezione spostata e risponde con `room_cleared=true`, `cleared_room=<nome>`.
4. La UI apre un Modal che spiega cosa è successo e chiede di scegliere una nuova aula dal dropdown (verde libera / rosso occupata).

6.7 Slot picker matriciale (Monitor)

Nel tab *Monitor*, espandendo una cattedra si vedono le singole lezioni; cliccando il bottone *Giorno/Ora* si apre un modal con una matrice 6x6 dove ogni cella è colorata in base alla disponibilità:

- verde: free (docente e classe liberi)
- rosso: HARD (docente o classe già impegnati)
- ambra: aula occupata da un'altra lezione
- azzurro: slot attuale

Click su una cella libera fa un dry-run via PUT `/api/monitor/event/{aid}/lesson/{lid}`. Se ci sono conflitti, apre un secondo Modal con 3 opzioni: **Annulla**, **Disassegna le lezioni in conflitto**, **Disassegna e ottimizza dopo**.

6.8 Punteggio di graduatoria e criterio Anzianita

Ciascun docente nell'anagrafica ha un campo **graduatoria_score** (numerico). Il valore esprime il punteggio di graduatoria utile a discriminare i docenti quando si tratta di assegnare cattedre o supplenze: più alto = maggiore precedenza.

Il campo è usato in due punti:

1. **Modulo supplenze**: quando il sistema cerca un candidato per coprire un'assenza, ordina la rosa per `graduatoria_score` decrescente.
2. **Phase A, criterio "Anzianita"**: il DSL della funzione obiettivo della Phase A include un preset **Anzianita** che, oltre ai criteri standard di bilancio (capacità non utilizzata, n. di indirizzi, ecc.), privilegia le assegnazioni che fanno coincidere docenti con alto `graduatoria_score` con classi del triennio finale (a parità di altre condizioni).

Il preset è selezionabile dalla card **Phase A (assegnazione)** del tab Workflow, dropdown "Criterio". Il DSL completo della funzione obiettivo è visibile nel modal "Mostra DSL attivo" della stessa card.

7 Guida UI

7.1 Funzionalità trasversali

7.1.1 Query DSL e sort multi-livello

Ogni pagina lista ha:

- barra Query (DSL: =, !=, <, <=, >, >=, contains, startswith, endswith, in [...], logica AND / OR, parentesi);
- sort multi-livello (max 4 livelli);
- pannello Help con la lista dei campi.

7.1.2 Multi-select

Sulle pagine Docenti / Classi / Aule la lista supporta:

- click semplice: single-select (replace);
- Ctrl+click: toggle;
- Shift+click: range;
- bottone "Vincolo collettivo" che apre il *BulkApplyModal*.

7.1.3 Excel/CSV import

Ogni pagina lista ha "Importa Excel/CSV" + "Template". Lo stesso endpoint POST `/api/import/{entity}` gestisce le 7 entità (teachers, subjects, classes, classrooms, curricula, students, groups). Modalità: upsert (default), append, replace. Header alias italiani / inglesi accettati.

7.1.4 Esportazione liste (xlsx / csv)

Sopra ogni tabella, accanto a "Reset query" / "Reset sort", trovi 3 bottoni di esportazione:

- **xlsx**: la *vista corrente* (solo le righe filtrate dalla query DSL, nell'ordine determinato dal sort multi-livello) come file .xlsx con header colorato e auto-fit colonne.
- **csv**: stesso contenuto, formato CSV UTF-8 con BOM (compatibile con Excel italiano).
- **tutto**: ignora query e sort, esporta tutte le righe dell'entità.

Filename: `<entita>_<YYYYMMDD>_<HHMMSS>.<ext>`. I parametri `?format=xlsx|csv` sono accettati da tutti gli endpoint di lista, quindi gli stessi export sono disponibili anche via curl.

7.1.5 Viste salvate

Sopra ogni tabella, accanto ai bottoni di Reset e Export, trovi un dropdown "Viste salvate..." e i bottoni "Salva vista" / "x":

- **Salva vista:** chiede un nome e memorizza la query DSL + il sort multi-livello come SavedView(entity, name, dsl_query, sort_levels). La vista è globale.
- **Dropdown "Viste salvate...":** applica una vista (sostituisce query + sort + ricarica la lista).
- **x:** elimina la vista correntemente selezionata.

Endpoint: /api/saved-views?entity=<entity> (GET / POST / PUT {id} / DELETE {id}). Tabella: saved_views.

7.1.6 Tag (aule e studenti)

Sistema many-to-many parallelo per aule e studenti.

Tag aule: tabella classroom_tags + join classroom_tag_assignments. UI: multi-select autocomplete nella scheda aula, bottone "Gestisci tag" per CRUD globale, colonna "Tag" con pill colorate (un colore stabile per nome). Il generatore mock tagga automaticamente le aule per kind (lab_fisica → [lab, fisica, scienze], palestra → [palestra, motorie], ecc.) e propaga il curriculum dalla home class (3B_scientifico → tag scientifico).

DSL: "matematica" in l.classroom.tags (vincoli) o has_tag(matematica) (query lista). I tag sconosciuti vengono creati al volo al salvataggio.

Tag studenti: tabella student_tags + join student_tag_assignments. Casi d'uso: BES, DSA, debito_matematica_4, pcto_ditta_<x>, studente_atleta. UI analoga a quella delle aule. Pannello "Precompila da tag" nel modal *Gruppi* che aggiunge in massa gli studenti che matchano any_of o all_of su una lista di tag.

Importante: i tag NON sostituiscono i gruppi. I gruppi restano l'unità operativa di scheduling, i tag sono metadato passivo per filtrare/raggruppare.

7.1.7 Import bulk dedicato (/import)

Pagina dedicata raggiungibile dalla nav. Layout a due colonne: configurazione (entità, modalità, "Scarica template") + file da importare (drop area + selettore file). Il template xlsx ha 3 fogli:

- Istruzioni: tabella Campo | Hint con descrizione di ogni colonna + note generali (sinonimi italiani, righe vuote ignorate, etc.);
- Esempi: una riga di esempio pre-popolata;
- Dati: foglio vuoto da compilare. L'importer legge per default questo foglio.

Campi importabili già integrati con le feature recenti: tags su students e classrooms, graduatoria_score / required_free_days_count / preferred_free_days_json su teachers, e max_hours_per_day su classes.

7.1.8 Import / Export DB (Dashboard)

Card prominente nella prima riga della Dashboard. Funzioni:

- **Esporta DB completo:** scarica un .zip contenente database.db (raw SQLite), tables/<t>.csv per ogni tabella e metadata.json (schema_version + sha256).
- **Solo schema:** omette i dati di scheduling – utile come template per inizializzare un'altra scuola.
- **Import:** selettore file .zip + conferma esplicita. Una copia timetable.db.pre_import_backup viene salvata prima dello swap.
- **Snapshot:** timestampati in webui/data/snapshots/, lista con bottoni Ripristina / Elimina.

Endpoint: /api/dashboard/export-db, /api/dashboard/import-db, /api/dashboard/snapshot/{create|list|restore/<f>}, DELETE /api/dashboard/snapshot/{f}.

7.2 Tab per tab

Dashboard

Importa profilo (5 size) con 3 checkbox per i pool (curricula, aule, studenti); genera scuola di test; stato corrente con 7 card-numero; *RunLogPanel* streaming.

Docenti, Classi, Indirizzi, Studenti, Gruppi, Materie, Aule, Compresenze

CRUD entità con modal di edit. Vedere ui_guide.md per i campi specifici di ognuna.

Cattedre

Layout a card: una card per classe, lista materie+docente+ore. Bottone "cambia": apre modal con dropdown filtrato per materia, ogni opzione mostra Cognome Nome - assigned/max [SFORA: +Xh] / [pieno]. Bottone "Warnings cattedre": pannello con docenti SFORA / sotto / vuoti / ok.

Orario

4 viste (per classe, per docente, per aula, per slot), ognuna con toggle matrice/lista. Drag-and-drop con preview live, dropdown aule colorate (verde libera / rosso occupata).

Assenze e supplenze

Vista settimanale 6×6. Click su intestazione giorno: modal gestione assenze. Click su cella rossa: modal con classi scoperte (sinistra) + docenti disponibili (destra), drag-drop per assegnare supplenti.

Monitor

Lista *eventi* (uno per Assignment). Espansione lezione-per-lezione con bottone Giorno/Ora che apre slot picker matriciale (vedere §5.6). In cima alla pagina un *segmented control* con quattro tab per filtrare velocemente: **Tutti**, **Incompleti** (eventi con `n_assigned_hours < n_expected_hours`), **Senza orario** (eventi senza alcuna lezione piazzata) e **Lockati** (eventi con almeno una lezione marcata `lesson.locked == true`). Per ogni riga, le azioni disponibili sono *Dissocia*, *Blocca/Sblocca* e *Piazza* (eseguito singolarmente o in massa via multi-selezione + toolbar bulk).

Vincoli

Lista flat di tutti i vincoli editabili con pill colorate per livello. Bottone "Cerca conflitti": pannello con conflitti rilevati.

Workflow

Pulsanti per lanciare le 4 fasi separatamente o "Pipeline completa". SSE log streaming.

8 Launcher

8.1 Windows: start.bat

Apri due finestre cmd separate. Pre-flight:

- npm reachable (forza nodejs nel PATH);
- venv esiste in backend/.venv;
- backend/main.py e frontend/package.json presenti;
- se node_modules manca, lancia npm install;
- avvisa se 8000 / 5173 sono già occupate.

I path sono relativi a %~dp0, quindi lo script funziona ovunque venga messo.

8.2 Linux / macOS: start.sh + stop.sh

Default background: log in webui/logs/, PID in webui/logs/pids. Flag -f / -foreground per Ctrl+C cleanup. Auto-crea venv e fa npm install se mancano. Hint per apt, dnf, pacman, brew se python3 / npm non sono nel PATH.

stop.sh legge logs/pids, manda SIGTERM con fallback SIGKILL dopo 3s, kill anche i child via pkill -P e ha un fallback per ammazzare qualunque processo ancora in ascolto su 8000 / 5173 via lsof.

9 CP-SAT: il motore che cerca, deduce, ritratta

9.1 Il problema concreto

Immagina un dirigente scolastico la prima settimana di settembre. Sul tavolo ha l'elenco delle classi (20–80, a seconda della scuola), l'elenco dei docenti (40–150), i monte ore di ogni materia in ogni indirizzo, le aule disponibili e una pila di richieste personali (“il prof Bianchi non lavora sabato”, “il lab di fisica è occupato il martedì mattina da un altro istituto”). Deve produrre un orario settimanale in cui ogni lezione sta in uno slot compatibile, nessun docente è in due posti contemporaneamente, nessuna classe ha buchi inutili e ogni materia rispetta il monte ore.

A occhio sembra che basti “provare”. In pratica il numero di assegnamenti possibili è astronomico. Una scuola media di 30 classi con 5 ore al giorno per 6 giorni ha $30 \times 5 \times 6 = 900$ slot da riempire scegliendo per ognuno una lezione fra centinaia di possibili: 10^{2700} combinazioni circa. Nessun computer al mondo le può esaminare una per una. Serve una macchina che, mentre prova, capisca *quali combinazioni si possono escludere subito* senza esaminarle.

CP-SAT è questa macchina. Il nome significa *Constraint Programming over SAT*: prende l'idea classica della programmazione a vincoli e la implementa sopra un solver SAT moderno (cioè un risolutore di formule booleane), con tutte le ottimizzazioni che la comunità SAT ha sviluppato negli ultimi vent'anni. È sviluppato da Google come parte del pacchetto OR-Tools Google 2010, ed è oggi uno dei migliori solver gratuiti per problemi combinatori di medie/grandi dimensioni.

Intuizione

In una frase: CP-SAT è un motore che, dato un elenco di variabili, di vincoli da rispettare e di un obiettivo da minimizzare, cerca un'assegnazione di valori alle variabili che rispetti tutto e renda l'obiettivo il più piccolo possibile, senza enumerare le possibilità una a una.

9.2 Una storia in cinque tappe

Per capire come funziona CP-SAT è utile ripercorrere brevemente cinque idee successive.

9.2.1 (1) Il problema SAT puro

Negli anni '60 i logici si chiedevano: data una formula booleana (un'espressione di variabili true/false collegate da AND, OR, NOT), esiste un assegnamento di verità che la rende vera? Questo è il problema SAT, formalmente NP-completo Garey e Johnson 1979. Una formula tipica appare così:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee \neg x_3)$$

Le parentesi tonde si chiamano *clausole*; ogni clausola è un OR di letterali (una variabile o la sua negazione). Una formula in questa forma è detta *normalizzata in CNF* (*conjunctive normal form*). Il solver deve trovare un'assegnazione che renda *vera ogni* clausola.

9.2.2 (2) DPLL: backtracking + propagazione unaria

Negli anni '60 Davis, Putnam, Logemann e Loveland proposero l'algoritmo DPLL: scegli una variabile, prova $x=\text{true}$, propaga le conseguenze, se trovi una contraddizione prova $x=\text{false}$. Si chiama *backtracking*: se sbagli torni indietro. La propagazione unaria si chiama *unit propagation*: se in una clausola tutti i letterali tranne uno sono falsi, quello rimasto deve essere vero.

9.2.3 (3) CDCL: imparare dai propri errori

Negli anni '90 si è capito che il backtracking puro spreca energia: spesso il solver torna indietro a uno stato che *contiene la stessa contraddizione* di quello appena abbandonato, e ricomincia da capo. La svolta è stata il *conflict-driven clause learning*: quando trovi una contraddizione, analizza le scelte che ti ci hanno portato e sintetizzane una nuova clausola che *vieta* quella combinazione fatale. Aggiungi la clausola alla formula. La prossima volta che ti avvicini a quella combinazione, la unit propagation ti blocca subito. Questa idea, implementata in solver come Chaff Moskewicz et al. 2001 e tutti i suoi successori, ha portato a un salto di prestazioni di ordini di grandezza.

9.2.4 (4) Constraint Programming: variabili intere e vincoli ad alto livello

Nel frattempo, la comunità di constraint programming sviluppava un linguaggio diverso: invece di sole variabili booleane e clausole, lavorava con variabili intere su domini finiti (es. $x \in \{1, \dots, 30\}$) e vincoli “ad alto livello” come $\text{AllDifferent}(x_1, \dots, x_n)$ o Cumulative per problemi di scheduling. Per ogni vincolo si scrivono *algoritmi di propagazione specializzati* che, dato il dominio corrente di ogni variabile, eliminano i valori che non possono più far parte di una soluzione.

9.2.5 (5) CP-SAT: la fusione

CP-SAT prende il meglio dei due mondi:

- dal lato CP, eredita le variabili intere e i vincoli globali (il problema rimane scrivibile in modo naturale);
- dal lato SAT, eredita CDCL, unit propagation e tutte le ottimizzazioni ingegneristiche (decision heuristic, restart, in-processing).

L'implementazione internamente traduce le variabili intere e i vincoli ad alto livello in una rete di booleane equivalente (“lazy clause generation”): ogni volta che il solver prende una decisione su una variabile intera, le booleane associate vengono propagate dal motore SAT, e quando il SAT solver scopre una contraddizione, la nuova clausola appresa si traduce di nuovo in vincoli sulla rete intera. È una danza continua fra i due livelli.

Nota storica

La famiglia DPLL ha origine nel 1962 (Davis, Putnam, Logemann, Loveland). CDCL nasce nel 1996–1997 con GRASP (Marques-Silva, Sakallah). Chaff (2001) introduce le strutture dati moderne (two-watched literals). CP-SAT come prodotto Google nasce intorno al 2010 con OR-Tools, e ha vinto le edizioni 2018, 2019, 2020 e 2021 della MiniZinc Challenge, la principale competizione di constraint solver.

9.3 Costruzione passo-passo: cosa serve sapere

9.3.1 Variabile in CP

Definizione

Una **variabile** in CP è un'incognita con un *dominio* finito di valori possibili. Esempi:

- $\text{slot}_{\text{matematica-1A}} \in \{1, 2, \dots, 36\}$ (lo slot in cui sta la prima ora di matematica della 1A; numero da 1 a 36 se ci sono 6 giorni $\times$ 6 ore).
- $\text{aula}_{\text{fisica-3B-martedì-9}} \in \{\text{LabFisica1}, \text{LabFisica2}\}$.
- $\text{is_present}_{\text{Rossi-lunedì-1}} \in \{0, 1\}$ (variabile booleana).

In Python con OR-Tools si scrivono così:

```
from ortools.sat.python import cp_model
m = cp_model.CpModel()

slot = m.NewIntVar(1, 36, "slot_mate_1A")
aula = m.NewIntVarFromDomain(
    cp_model.Domain.FromValues([101, 102]), "aula_fisica_3B_mar_9")
present = m.NewBoolVar("Rossi_lun_1")
```

9.3.2 Vincolo

Definizione

Un **vincolo** è una relazione fra una o più variabili che limita le combinazioni di valori ammissibili. Esempi:

- *Aritmetico*: $x + y \leq 10$.
- *Disuguaglianza*: $x \neq y$.
- *Globale*: $\text{AllDifferent}(x_1, \dots, x_n)$ significa “i valori assunti da queste variabili devono essere tutti distinti”. Equivale a $\binom{n}{2}$ vincoli $x_i \neq x_j$, ma il propagatore globale è molto più efficiente (sfrutta il teorema di Hall, vedi cap. 12).

9.3.3 Propagazione

Definizione

La **propagazione** è l'applicazione di regole locali che, vista una scelta o restrizione su una variabile, restringono il dominio di *altre* variabili senza che si debba fare alcun “tentativo”. Si tratta di *deduzione automatica*.

Esempio

Tre variabili $x, y, z \in \{1, 2, 3\}$ e tre vincoli: $x \neq y, y \neq z, z \neq x$. Supponi di aver fissato $x = 1$. La propagazione fa:

1. Da $x \neq y$: rimuove 1 dal dominio di y . Ora $y \in \{2, 3\}$.
2. Da $z \neq x$: rimuove 1 dal dominio di z . Ora $z \in \{2, 3\}$.

A questo punto se aggiungi $y = 2$, propaga ancora:

3. Da $y \neq z$: rimuove 2 dal dominio di z . Ora $z = 3$ (forzato).

La propagazione ha dedotto z senza enumerare. Senza propagazione il solver avrebbe provato $z = 1, 2, 3$ uno per uno e fallito su due dei tre tentativi.

9.3.4 Ricerca

Definizione

La **ricerca** è la fase in cui il solver, esaurita la propagazione, sceglie a caso una variabile e un valore, lo prova, e ricomincia a propagare. Se la propagazione raggiunge una contraddizione (un dominio vuoto), la ricerca *fa backtracking*: torna indietro, elimina quel valore dal dominio della variabile scelta, e prova un altro.

La sequenza “decisione $\rightarrow$ propagazione $\rightarrow$ eventuale backtracking” è il cuore del solver. La differenza fra un solver lento e uno veloce sta in:

- **quale variabile scelgo?** Euristiche tipiche: “la più vincolata” (smallest domain first), “la più attiva nei conflitti recenti” (VSIDS).
- **quale valore provo?** “Il più promettente” (last solution first), o random.
- **quanto imparo da ogni conflitto?** CDCL avanzato con clause learning + restart + clause forgetting.

9.3.5 Combinazione: il ciclo CP-SAT

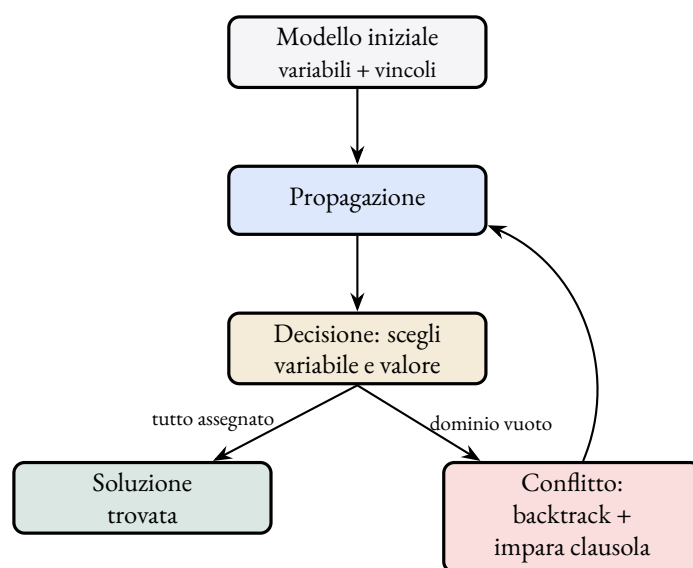


Figura 9.1: Il ciclo CP-SAT. Dopo aver propagato il modello iniziale, il solver alterna decisioni e propagazioni; ogni conflitto produce una clausola appresa che evita di ripetere lo stesso errore.

La fig. 9.1 mostra il ciclo. Importante: *ogni conflitto non è tempo perso*, perché produce una clausola appresa che diventa parte permanente del modello e accelera tutte le ricerche future.

9.4 Esempio mini-completo: 3 docenti, 2 classi

Per *vedere* CP-SAT all’opera, prendiamo un orario miniatura: 3 docenti (Adami, Bruni, Conti), 2 classi (1A, 1B), 4 slot (lunedì 1, lunedì 2, martedì 1, martedì 2). Materie e monte ore:

- 1A: matematica 2h, italiano 1h, storia 1h.
- 1B: matematica 1h, italiano 2h, storia 1h.

Assegnazioni docenti–materia (già fatte da Phase A):

- Adami insegna matematica in 1A (2h) e in 1B (1h).
- Bruni insegna italiano in 1A (1h) e in 1B (2h).
- Conti insegna storia in 1A (1h) e in 1B (1h).

Vincolo aggiuntivo: Bruni non lavora lunedì 1 (indisponibilità HARD).

Modellazione. Per ogni (classe, materia, ora) ho una variabile intera che dice in quale slot va. Per esempio $S_{1A, \text{mate}, 1} \in \{1, 2, 3, 4\}$ (l'ora di matematica della 1A in lunedì-1, lunedì-2, martedì-1 o martedì-2). In totale ho 8 variabili: due ore di mate-1A, una di ita-1A, una di sto-1A, una di mate-1B, due di ita-1B, una di sto-1B.

Vincoli HARD:

1. Una classe non può avere due lezioni nello stesso slot: AllDifferent sui 4 slot di ogni classe.
2. Un docente non può tenere due lezioni nello stesso slot: per ogni coppia di lezioni dello stesso docente, $S_a \neq S_b$.
3. Bruni non lavora lunedì 1: i due slot di ita-1B e l'uno di ita-1A non possono valere 1.

Propagazione iniziale. Dal vincolo (3): tre variabili perdono subito 1 dai loro domini. Ora ita-1A $\in \{2, 3, 4\}$, e analogamente per le due ita-1B.

Prima decisione. Il solver sceglie la variabile con dominio più piccolo. Tutte le 8 hanno dominio simile, ma le 3 di Bruni ne hanno solo 3. Sceglie ita-1A = 2 (lunedì-2, primo valore disponibile).

Propagazione dopo la decisione. Il vincolo AllDifferent sulla 1A rimuove 2 dal dominio delle altre 3 variabili della 1A. Il vincolo “Bruni non in due slot” rimuove 2 dalle due variabili di ita-1B. Ora ita-1B ha dominio $\{3, 4\}$ e devono essere entrambi distinti: in pratica $\{3, 4\}$ è l'unica assegnazione possibile.

In altre parole, la propagazione ha già forzato i due slot di Bruni in 1B. Resta da piazzare 5 variabili: mate-1A (2), sto-1A (1), mate-1B (1), sto-1B (1).

Seconda decisione. Il solver tenta mate-1A,1 = 1. Propaga: rimuove 1 da mate-1A,2 (dominio: $\{3, 4\}$), da sto-1A (dominio $\{3, 4\}$), e da mate-1B (Adami non può essere in due posti, dominio $\{2, 3, 4\}$, ma il 2 è già stato preso da nessuna ora di Adami quindi rimane $\{2, 3, 4\}$).

Successive decisioni. Il solver continua. Tipicamente in 5–10 step di decisione+propagazione tutte le 8 variabili sono fissate. Se in qualche step trovasse mate-1A,2 = 3 ma sto-1A = 3, il vincolo AllDifferent darebbe contraddizione: backtracking.

In questo esempio il solver trova soluzione in pochi millisecondi, senza neanche dover imparare clausole. Il punto è che il *numero di tentativi* è drasticamente ridotto dalla propagazione: invece di esaminare $4^8 = 65,536$ combinazioni, il solver ne esamina meno di 20.

Esempio

Una soluzione possibile:

	Lun-1	Lun-2	Mar-1	Mar-2
1A	mate (Adami)	ita (Bruni)	sto (Conti)	mate (Adami)
1B	sto (Conti)	mate (Adami)	ita (Bruni)	ita (Bruni)

Ogni classe ha 4 ore distinte; Adami non sta mai in due posti contemporanei (lun-1, lun-2, mar-2 sono i suoi 3 slot); Bruni non insegna lunedì 1; Conti insegna lun-1 e mar-1. Tutto rispetta i vincoli.

9.5 Generalizzazione alla scuola reale

In una scuola di 30 classi e 60 docenti la stessa logica funziona, con due differenze importanti:

1. **La propagazione fa gran parte del lavoro.** I vincoli AllDifferent, le indisponibilità, le preferenze rigide e i lab condivisi creano una rete molto fitta di deduzioni: una scelta su una classe del triennio spesso costringe decine di altri slot.
2. **La ricerca diventa interessante quando i vincoli HARD sono “quasi soddisfatti”.** Quando il solver ha piazzato il 70% delle lezioni e si trova davanti alle ultime, qualunque scelta porta a contraddizioni e il backtracking si scatena. È qui che le clausole apprese fanno la differenza: dopo qualche centinaio di conflitti il solver “capisce” che certe combinazioni non funzionano e smette di provarle.

9.5.1 L’obiettivo: ottimizzazione, non solo soddisfacibilità

CP-SAT non si limita a trovare *una* soluzione: può *ottimizzare*. Si dichiara una variabile aggiuntiva obj definita come somma pesata di pezzi “brutti” (es. ore isolate dei docenti, sesta ora delle classi, lezioni in slot non preferiti) e si chiede al solver di minimizzarla. Internamente CP-SAT alterna ricerca della soluzione e ricerca di una migliore: ogni volta che ne trova una, aggiunge il vincolo $\text{obj} < \text{best}$ e ricomincia. Termina quando dimostra che non esiste soluzione migliore o quando finisce il tempo.

In π Tantum la funzione obiettivo è descritta nel cap. ??; si veda anche la documentazione DSL per la sintassi degli obiettivi componibili.

9.6 Quando usarlo (e quando no)

CP-SAT brilla quando:

- Il problema è *combinatorio puro* (variabili intere, vincoli logici): scheduling, configurazione, packing, routing piccoli/medi.
- I vincoli sono *molto interconnessi*: la propagazione ha materiale su cui lavorare.
- Il numero di variabili è nell’ordine 10^3 – 10^5 : oltre, conviene decomporre o usare metaeuristiche.

Limiti:

- Sopra $\sim 10^6$ variabili boolean equivalenti, anche CP-SAT diventa lento. È uno dei motivi per cui π Tantum decompone le scuole grandi (cap. 10).
- Sui problemi *continui* (variabili reali) CP-SAT non lavora; servono solver LP/MIP/SOCP.
- Se l’obiettivo è una somma di molti termini con pesi fini, la ricerca dell’ottimo esatto può impiegare a lungo: spesso conviene fermarla con un `time_limit` e raffinare con metaeuristiche.

Attenzione

CP-SAT è una “scatola intelligente” ma non magica. Modellare male un problema (es. con vincoli ridondanti, o con variabili intere che potevano essere booleane) lo rallenta di ordini di grandezza. La regola pratica è: *usa i vincoli globali quando puoi*, perché ogni vincolo globale ha un propagatore specializzato che lavora molto meglio di n^2 vincoli binari.

9.7 Connessioni con il resto del sistema

Connessioni

- **Decomposizione spettrale** (cap. 10): per scuole oltre ~ 80 classi, π Tantum spezza il problema in cluster e invoca CP-SAT su ognuno.
- **LNS / SA / TS / ILS** (cap. 11): ogni metaeuristica usa CP-SAT per “riparare” sotto-vicinati ad ogni iterazione (ALNS in particolare).
- **Hall pre-check** (cap. 12): prima di lanciare CP-SAT, π Tantum verifica che l’assegnamento docenti $\rightarrow$ classi sia *matching-feasible*. Se Hall fallisce, CP-SAT non potrebbe trovare nessuna soluzione e darebbe INFEASIBLE dopo aver bruciato minuti di calcolo.
- **Lagrangian Relaxation** (cap. 13): rilassa alcuni vincoli HARD trasformandoli in penalità sull’obiettivo, riducendo il problema CP-SAT a sotto-problemi più piccoli.

Per approfondire

- Lettura introduttiva (divulgativa): Russell & Norvig, *Artificial Intelligence: A Modern Approach*, capp. 6–7 sui CSP.
- Manuale di riferimento: Rossi, Van Beek, Walsh, *Handbook of Constraint Programming* Rossi et al. 2006, capp. 3 (CSP), 4 (consistency), 6 (search).
- Storia di CDCL e SAT moderno: Marques-Silva, Lynce, Malik, *Conflict-Driven Clause Learning SAT Solvers*, in *Handbook of Satisfiability*, IOS Press 2009.
- Documentazione OR-Tools: https://developers.google.com/optimization/cp/cp_solver – molti esempi pratici in Python.
- Contesto teorico (NP-completezza): Garey & Johnson Garey e Johnson 1979 resta il classico.

10 Decomposizione spettrale: spezzare per regnare

“The eigenvectors of the Laplacian know more about the graph than you might think.”

Fan R. K. Chung, *Spectral Graph Theory* (Chung 1997)

IMMAGINA di dover organizzare l'orario di un istituto comprensivo enorme: 90 classi, 200 docenti, 12 indirizzi diversi. Sembra un'impresa formidabile da risolvere “tutta insieme”. Ma se osservi il grafo che ha per nodi le classi e i docenti, e archi che dicono “questo docente insegna in questa classe”, noti qualcosa: il grafo non è omogeneo. Ci sono *cluster* naturali. Le classi del liceo classico hanno docenti che lavorano quasi esclusivamente con il classico; le classi dell'ITIS hanno i loro. I docenti di matematica si distribuiscono fra indirizzi affini ma raramente toccano l'IPSIA accanto.

Se tu potessi *vedere* questi cluster e risolvere l'orario di ognuno separatamente, il problema cambierebbe faccia: invece di una scuola da 90 classi, avresti 5–7 sotto-scuole da 12–18 classi. Ognuna trattabile in pochi minuti. Questo è il trucco della decomposizione spettrale: *usare l'algebra del grafo per individuare i cluster naturali*.

10.1 Da dove viene l'idea

Nota storica

Nel 1973 il matematico ceco Miroslav Fiedler (Fiedler 1973) pubblicò un risultato che all'epoca sembrava puramente teorico: il *secondo* autovettore del Laplaciano di un grafo (oggi chiamato *vettore di Fiedler*) ne rivela la struttura comunitaria. Quasi vent'anni dopo Shi e Malik (Shi e Malik 2000) riadattarono l'idea per la *segmentazione di immagini*: vedi un'immagine come grafo (pixel = nodi, archi = somiglianza fra pixel adiacenti), calcoli il vettore di Fiedler e ottieni i contorni degli oggetti. Da lì in poi il *spectral clustering* (Luxburg 2007) è diventato uno standard nella community di machine learning.

L'idea nascosta nel risultato di Fiedler è che la *struttura combinatoria* di un grafo (chi è connesso con chi) si riflette nello *spettro* (gli autovalori) di una matrice associata. E lo spettro è calcolabile in tempo polinomiale standard.

10.2 Costruzione passo-passo

10.2.1 (1) Il grafo bipartito classe–docente

Definizione

Per una scuola con n classi e m docenti, costruiamo un *grafo bipartito* $G = (V, E)$ con:

- $V = V_C \cup V_D$, dove V_C è l'insieme delle classi e V_D l'insieme dei docenti.
- E : c'è un arco $\{c, d\}$ con peso w_{cd} se il docente d insegna w_{cd} ore alla classe c .

Esempio in miniatura: una scuola con 3 classi (c_1, c_2, c_3) e 4 docenti (d_1, d_2, d_3, d_4), in cui d_1, d_2 insegnano solo nelle prime due classi e d_3, d_4 solo nella terza, ha due cluster evidenti: $\{c_1, c_2, d_1, d_2\}$ e $\{c_3, d_3, d_4\}$. Il grafo bipartito ha pochissimi (o zero) archi fra i due cluster.

10.2.2 (2) La matrice di adiacenza e quella dei gradi

Definizione

La *matrice di adiacenza* A è una matrice quadrata $|V| \times |V|$ con $A_{ij} = w_{ij}$ se esiste l'arco $\{i, j\}$, o altrimenti.

La *matrice dei gradi* D è diagonale con $D_{ii} = \sum_j A_{ij}$, ovvero la somma dei pesi degli archi incidenti su i . Per un docente, D_{ii} è il monte ore totale; per una classe, è la somma delle ore di lezione ricevute.

10.2.3 (3) Il Laplaciano (normalizzato)

Definizione

Il *Laplaciano normalizzato* di G è:

$$L = I - D^{-1/2} A D^{-1/2}$$

dove I è la matrice identità.

Come si legge: prendi la matrice di adiacenza A (che dice chi è connesso a chi), la *normalizzi* dividendo ogni elemento per la radice del prodotto dei gradi dei due estremi (per togliere l'effetto della diversa "importanza" dei nodi), e sottrai il risultato dall'identità.

Proprietà chiave:

- L è simmetrica e semi-definita positiva.
- Tutti i suoi autovalori $\lambda_0, \lambda_1, \dots$ sono nell'intervallo $[0, 2]$.
- L'autovalore minimo λ_0 vale sempre 0; il corrispondente autovettore è proporzionale a $D^{1/2} \mathbf{1}$ (banale).
- Il *secondo autovalore* λ_1 è detto *algebraic connectivity* (Fiedler 1973): è "piccolo" quando il grafo ha cluster ben separati, "grande" quando il grafo è ben connesso.

10.2.4 (4) Il vettore di Fiedler

Definizione

Il *vettore di Fiedler* v_1 è l'autovettore associato al secondo autovalore λ_1 di L .

L'idea cruciale: ogni nodo del grafo corrisponde a una componente di v_1 , cioè a un numero reale. Se ordini i nodi per il valore della loro componente in v_1 , ottieni un *ordinamento naturale* che separa le comunità.

10.2.5 (5) k-means sui top- k autovettori

Per ottenere k cluster (non solo 2), si calcolano i primi k autovettori non-banali $v_1, v_2, \dots, v_k$ di L . Si ottiene così una matrice $|V| \times k$: ogni nodo diventa un punto in $\mathbb{R}^k$. A questo punto si applica k -means standard su questa rappresentazione *spettrale* e si ottengono k cluster.

Esempio: se $v_1 = (-0.4, -0.5, +0.3, +0.6)^\top$, l'ordinamento per componenti riordina i nodi come $\{2, 1, 3, 4\}$. Tagliando fra il 2° e il 3° ottieni i due cluster.

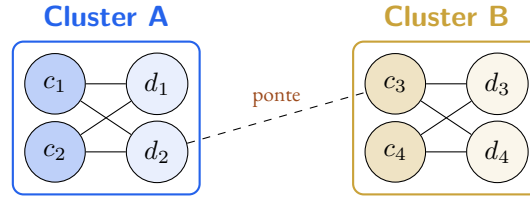


Figura 10.1: Esempio di grafo bipartito con due cluster naturali e un “ponte” debole. Il vettore di Fiedler assegna componenti di segno opposto ai due cluster.

10.3 Esempio mini-completo a mano

Prendiamo la scuola di fig. 10.1: 4 classi, 4 docenti, archi come disegnati (incluso un “ponte” debole d_2-c_3).

Passo 1: matrici. Numeriamo i nodi 1, 2, ..., 8 in ordine $c_1, c_2, c_3, c_4, d_1, d_2, d_3, d_4$. La matrice di adiacenza A è (con peso 1 ovunque):

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

I gradi sono: $D = \text{diag}(2, 2, 3, 2, 2, 3, 2, 2)$.

Passo 2: Laplaciano normalizzato. Si ottiene applicando la formula $L = I - D^{-1/2}AD^{-1/2}$ (il calcolo a mano è tedioso, lo demandiamo a NumPy).

Passo 3: spettro. Calcolando con `numpy.linalg.eigh(L)` si ottiene:

$$\lambda_0 \approx 0, \quad \lambda_1 \approx 0.18, \quad \lambda_2 \approx 0.72, \quad \dots$$

Il piccolo valore $\lambda_1 \approx 0.18$ conferma che il grafo è debolmente connesso: i due cluster sono ben separati, il ponte d_2-c_3 è l'unico legame.

Passo 4: vettore di Fiedler. È l'autovettore associato a λ_1 . Calcolandolo si trova qualcosa come:

$$v_1 \approx (+0.40, +0.40, -0.35, -0.40, +0.35, +0.30, -0.30, -0.30)^\top$$

I primi quattro nodi (cluster A: c_1, c_2, d_1, d_2) hanno componenti positive; gli altri quattro (cluster B: c_3, c_4, d_3, d_4) hanno componenti negative. La separazione è netta. Tagliando in zero, otteniamo esattamente i due cluster.

Esempio

Per ottenere $k > 2$ cluster, si usano i primi k autovettori e si applica k -means. Su questa stessa scuola con $k = 2$, k -means su v_1 produce gli stessi due cluster ottenuti dal segno.

10.4 Generalizzazione: cosa cambia su scuola reale

In una scuola con 90 classi e 200 docenti il calcolo è identico, solo che le matrici sono 290×290 e gli autovettori si calcolano in qualche secondo con LAPACK/scipy. La differenza pratica è:

- I cluster non sono *così* netti come nell'esempio giocattolo. Tipicamente ci sono cluster ben separati e poi “nodi di confine” che potrebbero stare in più cluster.

- Il numero k di cluster non è dato a priori: si sceglie con una *eigengap heuristic*: si cerca il k tale che λ_k sia molto più grande di λ_{k-1} (un “salto” nello spettro). π Tantum esegue un’analisi automatica del gap e suggerisce un valore di k , ma l’utente può forzarlo.
- Una volta definiti i cluster, la *ricucitura* (π Tantum la chiama *Phase B stitch*) richiede un passo CP-SAT globale che armonizza i “ponti” fra cluster (lezioni che coinvolgono nodi di cluster diversi).

10.5 Quando funziona, quando no

Intuizione

La decomposizione spettrale funziona benissimo quando *i cluster naturali esistono*: scuole con indirizzi distinti, classi-di-concorso disgiunte, presenza di sedi distaccate. Il guadagno tipico su istanze grandi è $5\times-10\times$ rispetto a CP-SAT monolitico.

Attenzione

Quando il grafo è *denso* (i docenti insegnano un po’ ovunque, niente cluster naturali), il vettore di Fiedler non separa nulla e i cluster prodotti sono artificiali. In quel caso la decomposizione genera “ponti” molti e pesanti, e la fase di ricucitura diventa più costosa della risoluzione monolitica. π Tantum lo rileva guardando λ_1 del Laplaciano: se è troppo grande (sopra ~ 0.5), suggerisce di saltare la decomposizione.

10.5.1 Parametri principali

- `n_clusters`: numero di cluster da produrre. Default: scelto via *eigengap heuristic*; tipicamente 4–8 per scuole medio-grandi.
- `laplacian_type`: `normalized` (default) o `unnormalized`. Il primo è raccomandato per grafi non bilanciati nei pesi.
- `kmeans_init`: `k-means++` (default, robusto) o `random`.
- `stitch_passes`: numero di iterazioni della fase di ricucitura. Default: 1.

10.6 Connessioni

Connessioni

- **CP-SAT** (cap. 9): ogni cluster è risolto da CP-SAT come problema autonomo.
- **Hall pre-check** (cap. 12): si esegue *su ciascun cluster* oltre che globalmente; se un cluster non passa Hall, si rilassano i ponti per ridistribuire la domanda.
- **Lagrangian Relaxation** (cap. 13): un’alternativa alla ricucitura combinatoria è la dualizzazione dei vincoli inter-cluster con moltiplicatori di Lagrange.
- **Analisi del grafo** (cap. 15): modularità, betweenness, densità sono *measure* che π Tantum espone in dashboard per dare al coordinatore un’idea della “segmentabilità” della sua scuola *prima* di lanciare la pipeline.

Per approfondire

- Fiedler 1973: il paper originale di Fiedler. Tre pagine, ancora leggibili.
- Chung 1997: il libro di riferimento sulla teoria spettrale dei grafi. Tecnico ma chiaro.
- Luxburg 2007: tutorial divulgativo sullo spectral clustering, con tutti i dettagli pratici (scelta di k , normalizzazioni, k -means vs alternative).

- Shi e Malik 2000: l'applicazione al *normalized cut* per immagini, che ha reso popolare l'idea fuori dalla matematica pura.

11 Metaeuristiche: scolpire una soluzione

“Heuristics are mental shortcuts; meta-heuristics are shortcuts about how to take shortcuts.”

Christian & Griffiths, *Algorithms to Live By* (Christian e Griffiths 2016)

CP-SAT è bravissimo a trovare soluzioni *fattibili* (che rispettano i vincoli HARD) e sa anche ottimizzare la funzione obiettivo, ma su istanze grandi tende a fermarsi su un *plateau*: trova una soluzione decente in pochi minuti e poi continua a frugare senza migliorarla. Le metaeuristiche sono una famiglia di tecniche complementari: prendono una soluzione *già esistente* e la *migliorano iterativamente*, esplorando il *vicinato* (lo spazio delle soluzioni “simili”) in modo intelligente.

In π Tantum la Phase C è dedicata interamente alle metaeuristiche. Sono sei: LNS, ALNS, SA, TS, ILS, VNS. Hanno parecchio in comune (sono tutte forme di *local search*) ma differiscono nel modo in cui *esplorano* e nel modo in cui *accettano* le mosse.

11.1 La radice comune: local search

Definizione

La *ricerca locale* è uno schema iterativo:

1. Parti da una soluzione iniziale x_0 .
2. Definisci un *vicinato* $N(x)$: l'insieme delle soluzioni ottenibili da x con una singola mossa (es. swap di due lezioni).
3. Cerca dentro $N(x)$ una soluzione migliore x' .
4. Se la trovi: $x \leftarrow x'$, ripeti. Altrimenti ti fermi (sei in un *ottimo locale*).

Il problema della local search pura è che *si incastra nei minimi locali*. Le metaeuristiche sono varianti più sofisticate che provano ad evitarlo.

11.2 Simulated Annealing (SA): l'analogia con la metallurgia

Nota storica

Nel 1983, Kirkpatrick, Gelatt e Vecchi pubblicarono su Science l'articolo *Optimization by Simulated Annealing* (Kirkpatrick et al. 1983) che importò nell'ottimizzazione un'idea presa dalla metallurgia: per ottenere una struttura cristallina ordinata in un metallo, non basta raffreddarlo bruscamente (rimane in uno stato disordinato). Bisogna scaldarlo e raffreddarlo *lentamente*, lasciandogli il tempo di trovare configurazioni a bassa energia. La temperatura agisce come un parametro di “rumore”: alta temperatura permette movimenti che peggiorano la situazione, bassa temperatura ammette solo miglioramenti.

Intuizione

SA accetta sempre le mosse che migliorano l'obiettivo. Le mosse che lo peggiorano di Δ sono accettate con probabilità $e^{-\Delta/T}$, dove T è la temperatura corrente. All'inizio T è alta: anche peggioramenti grossi passano. Man mano che T scende, solo i piccoli peggioramenti passano. Alla fine si comporta come una local search greedy.

11.2.1 La regola di Metropolis

Il criterio di accettazione è dovuto a Metropolis (Metropolis et al. 1953), che lo introdusse per simulare sistemi termodinamici:

$$P(\text{accetta } x') = \begin{cases} 1 & \text{se } f(x') \leq f(x) \\ e^{-(f(x')-f(x))/T} & \text{altrimenti} \end{cases}$$

Come si legge: f è la funzione obiettivo da minimizzare. Se la nuova soluzione x' è migliore o uguale, la accetto sempre. Altrimenti la accetto con probabilità esponenzialmente decrescente nella differenza $\Delta = f(x') - f(x)$ e nella temperatura T .

11.2.2 Schema di raffreddamento

In π Tantum lo schema di default è *geometrico*: $T_{k+1} = \alpha T_k$ con $\alpha = 0.95$. Si parte da T_0 tale che le mosse iniziali abbiano una probabilità di accettazione del 70–80% e si scende fino a $T \sim 0.01$.

11.3 Tabu Search (TS): il taccuino delle mosse vietate**Nota storica**

Fred Glover propose Tabu Search nel 1986 con il celebre articolo *Future paths for integer programming and links to artificial intelligence* (Glover 1986). La parola *tabu* viene dal polinesiano *tabu*, che significa “proibito”. L'idea: in local search, dopo aver visitato una soluzione, vietare di tornarci *per un po'*, per forzare la ricerca a esplorare altrove.

Definizione

Tabu Search mantiene una *lista tabu* delle ultime k mosse fatte (es. “ho spostato mate-1A da lunedì-1 a martedì-3”). Le mosse nella lista tabu non possono essere usate finché non escono dalla finestra. La lista è di lunghezza fissa: ogni nuova mossa spinge fuori la più vecchia. C'è un *aspiration criterion*: se una mossa tabu produrrebbe la migliore soluzione mai vista, la facciamo lo stesso.

In π Tantum la lista tabu è tipicamente di 20–40 mosse; il valore è tarato sui benchmark.

11.4 Iterated Local Search (ILS): perturbare e ricominciare**Intuizione**

L'idea di ILS: una local search semplice si ferma su un ottimo locale; rilanciandola da una soluzione *simile ma perturbata* arrivi spesso a un ottimo locale diverso, eventualmente migliore. Iterando si esplorano molti bacini di attrazione.

Definizione

ILS alterna due operazioni:

1. **Local search:** arriva a un ottimo locale x^* .

2. **Perturbazione:** applica un disturbo casuale a x^* producendo x' , di solito leggero (es. 5–10 swap casuali di lezioni).

La local search ricomincia da x' . Si confronta il nuovo ottimo locale x^{**} con x^* e si tiene il migliore.

ILS è descritto formalmente da Loureno, Martin e Stuetzle (Lourenco et al. 2003). È spesso il metaeuristico “base” che si combina con altri (es. ILS+TS, ILS+SA).

11.5 Variable Neighborhood Search (VNS): cambiare obiettivo

Nota storica

VNS è dovuto a Mladenović e Hansen (1997) (Mladenovic e Hansen 1997). L’idea è cambiare il *tipo* di vicinato quando uno si esaurisce: invece di “perturbare e ricominciare” come ILS, VNS *cresce* il raggio della ricerca.

In π Tantum sono definiti quattro vicinati di dimensione crescente:

1. **1-swap:** scambia due lezioni dello stesso docente.
2. **2-swap:** scambia due coppie indipendenti di lezioni.
3. **3-chain:** rotazione di 3 lezioni ($a \rightarrow b \rightarrow c \rightarrow a$).
4. **k-opt** con $k \in [4, 6]$: rotazione lunga.

VNS parte dal vicinato 1; quando trova un miglioramento, *ricomincia* dal vicinato 1. Quando un’intera passata $1 \rightarrow 4$ non trova nulla, si ferma. Termina sempre in un ottimo locale rispetto a tutti e quattro i vicinati.

11.6 Large Neighborhood Search (LNS): distruggi e ricostruisci

Nota storica

LNS fu introdotto da Paul Shaw nel 1998 (Shaw 1998) per il problema del vehicle routing. L’idea: invece di esplorare un vicinato piccolo (1 o 2 swap), *distruggi* una porzione significativa della soluzione (es. 20% delle lezioni) e *ricostruisci*la ottimamente con un solver esatto. Il vicinato è “grande” (l’insieme delle soluzioni ottenibili ricostruendo la parte distrutta) ma esplorato *in modo intelligente* dal solver.

Definizione

LNS alterna due operatori:

1. **Destroy:** rimuove un sotto-insieme delle variabili dalla soluzione corrente.
2. **Repair:** re-ottimizza le variabili rimosse (tipicamente con CP-SAT, mantenendo fisse tutte le altre).

In π Tantum gli operatori destroy includono: `random_window` (rimuove un blocco di slot), `day_cluster` (rimuove tutte le lezioni di un giorno), `worst_fit_day` (rimuove il giorno con peggior obiettivo), `teacher_day` (rimuove tutte le lezioni di un docente in un giorno), `classroom_day`, `single_class_week`. Operatori repair: `cp_sat_window` (re-risolve con CP-SAT in time-budget di pochi secondi), `greedy_by_soft`, `bfs_fill_back`.

11.7 Adaptive LNS (ALNS): scolpire con scalpelli adattivi

Nota storica

ALNS fu proposto da Ropke e Pisinger nel 2006 (Ropke e Pisinger 2006) estendendo LNS con un meccanismo di *apprendimento*: invece di scegliere a caso fra gli operatori destroy/repair, ALNS impara quali funzionano meglio sull'istanza corrente e li sceglie più spesso.

Intuizione

Analogia: lo scultore davanti a un blocco di marmo non usa sempre lo stesso scalpello. Ne ha tanti, e impara quale serve per quale parte della statua. ALNS fa lo stesso: ha tanti operatori, ne misura il successo recente (*exponentially-decayed score*), e li seleziona via *roulette wheel* pesata sui punteggi.

L'aggiornamento dei punteggi: dopo ogni iterazione, l'operatore usato riceve un premio $\sigma_1, \sigma_2, \sigma_3$ a seconda del risultato (mossa migliorativa globale, mossa migliorativa locale, mossa accettata via SA-like). Il suo score w_i viene aggiornato come $w_i \leftarrow \rho w_i + (1 - \rho)\sigma$ con $\rho = 0.95$ (decay factor).

11.8 Quale scegliere?

Intuizione

Regola pratica:

- Se hai **poco tempo** e vuoi un risultato “decente”: SA con schema di raffreddamento veloce.
- Se vuoi **esplorazione robusta**: ILS o VNS.
- Se hai **molto tempo** e vuoi un risultato vicino all'ottimo: ALNS, perché impara a fare le mosse giuste.
- Se la soluzione iniziale è **molto strutturata** e bisogna fare mosse grandi: LNS con CP-SAT come repair.

In Phase C π Tantum di default esegue *ALNS*, e opzionalmente le altre per chi vuole confrontare.

11.9 Esempio mini-completo: SA su 8 lezioni

Esempio

Considera l'esempio del cap. 9: 3 docenti, 2 classi, 8 lezioni. Una soluzione fattibile è già stata trovata con CP-SAT, con costo $f(x_0) = 5$ (es. somma pesata di ore-isolate).

Iter 1: SA prova uno swap fra le ore 1 e 2 di mate-1A. Calcola $f(x_1) = 4$. Migliora! Accetta.

Iter 2: SA prova uno swap fra ita-1A (mar-1) e sto-1A (lun-2). Calcola $f(x_2) = 6$. Peggiora di $\Delta = 2$. Con $T = 1.0$: $P = e^{-2/1.0} = 0.135$. Estrae un random $r = 0.4 > P$: rifiuta. Resta su x_1 .

Iter 3: SA prova lo swap mate-1A,1 $\leftrightarrow$ mate-1B. Calcola $f = 3$. Migliora. Accetta.

Raffreddamento: $T \leftarrow 0.95T = 0.95$.

E così via per migliaia di iterazioni. Dopo 5000 iter tipiche, $T < 0.01$ e il sistema converge a un ottimo locale globale rispetto al vicinato 1-swap.

11.10 Limiti e parametri da tarare

Attenzione

- Le metaeuristiche *non* garantiscono l'ottimo. Su problemi piccoli, possono trovare soluzioni peggiori di CP-SAT esatto in poche iterazioni. Conviene quindi *combinare*: CP-SAT esatto fino al plateau, poi metaeuristiche.
- I parametri (temperatura iniziale di SA, lunghezza della lista tabu di TS, dimensione della perturbazione di ILS) richiedono tuning. π Tantum ha valori di default ragionevoli per ciascun profilo (small/medium/big/huge), ma su istanze atipiche conviene fare uno sweep.
- Le mosse devono essere *economiche da valutare*: π Tantum implementa una *delta-evaluation* della funzione obiettivo per evitare di ricalcolare tutto da zero ad ogni iterazione (costo $O(1)$ per swap invece di $O(n)$).

11.11 Connessioni

Connessioni

- **CP-SAT** (cap. 9): usato come solver di *repair* in LNS/ALNS.
- **Decomposizione spettrale** (cap. 10): le metaeuristiche girano *sopra* la soluzione di Phase B; spesso raffinano localmente i singoli cluster.
- **Editor manuale Orario**: gli swap dell'utente sono *1-mosse* dello stesso vicinato che usa SA; l'editor mostra in tempo reale il delta della funzione obiettivo, esattamente come fa SA internamente.

Per approfondire

- Kirkpatrick et al. 1983: SA originale, su Science. Tre pagine.
- Glover 1986: TS originale, di Glover.
- Shaw 1998: LNS originale, su CP'98.
- Ropke e Pisinger 2006: ALNS originale, su Transportation Science.
- Glover e Kochenberger 2003: *Handbook of Metaheuristics*, riferimento collettivo.
- Van Hentenryck e Michel 2005: *Constraint-Based Local Search*, sintesi fra CP e local search.

12 Hall pre-check: il livello di benzina prima di partire

“A necessary and sufficient condition that all the sets in the family S have a system of distinct representatives is that for every k , the union of any k sets of S contains at least k elements.”

Philip Hall, 1935 (Hall 1935)

PRIMA di accendere la macchina per un viaggio lungo, qualsiasi automobilista controlla se c'è abbastanza benzina. Sembra ovvio. Eppure quando si lancia un solver di timetabling è una pratica che spesso si salta: si parte, si aspetta dieci minuti, e si scopre che il problema era *infeasibile dall'inizio*. Tutti i tentativi del solver erano destinati a fallire perché i dati di input non potevano produrre nessun orario valido.

Il pre-check di Hall è quel controllo del livello di benzina: in pochi millisecondi, prima di lanciare CP-SAT, verifica se le condizioni necessarie alla fattibilità sono soddisfatte. Se Hall fallisce, π Tantum lo dice all'utente (e indica *quale* sotto-insieme di entità è il problema), evitando lunghe attese inutili.

12.1 Il teorema dei matrimoni

Nota storica

Nel 1935 Philip Hall pubblicò sul *Journal of the London Math. Society* (Hall 1935) un teorema deliziosamente semplice. Considera un gruppo di n ragazze e m ragazzi; ogni ragazza ha una lista di ragazzi che sarebbe disposta a sposare. Quando è possibile sposare *tutte* le ragazze in modo che ognuna abbia un suo ragazzo distinto?

La risposta di Hall: *quando per ogni sotto-insieme di k ragazze, la lista combinata di candidati contiene almeno k ragazzi.*

È condizione necessaria e sufficiente. Da allora il risultato si chiama *teorema dei matrimoni di Hall*, o *Hall's marriage theorem*.

12.2 Dal teorema all'orario

L'analogia con il timetabling è diretta. Considera la versione semplificata in cui devi assegnare ad ogni *lezione* (combinazione classe-materia-ora) un *docente* compatibile (con la classe di concorso giusta e la disponibilità sufficiente).

Definizione

Costruiamo un grafo bipartito:

- A sinistra: le *ore-cattedra* richieste (es. “2 ore di matematica per la 3A”).
- A destra: i *docenti* disponibili.
- Un arco $\{l, d\}$ se il docente d è compatibile con la lezione l (giusta classe di concorso, ore residue sufficienti, etc.).

Una soluzione di Phase A esiste se e solo se questo grafo ha un *matching che copre tutte le ore-cattedra*.

Intuizione

Per il teorema di Hall, il matching esiste **se e solo se** per ogni sotto-insieme S di ore-cattedra, la lista combinata dei docenti compatibili con almeno una di esse contiene un *numero di ore disponibili almeno pari* a $|S|$.

In linguaggio operativo: *ogni gruppo di lezioni che si contendono un pool di docenti compatibili deve avere abbastanza ore complessive nel pool.*

12.3 Tre controlli pratici

Verificare la condizione di Hall *esattamente* richiede di esaminare *tutti* i sottoinsiemi di lezioni: 2^n con n ore-cattedra, infattibile per n grande. π Tantum implementa allora tre controlli rapidi che, combinati, intercettano la stragrande maggioranza degli infeasibility che si vedono in pratica.

12.3.1 Controllo 1: per (classe, materia)

Definizione

Per ogni coppia (c, m) con h_{cm} ore richieste, verifica che la somma delle ore disponibili dei docenti abilitati a m nella classe di concorso pertinente per c sia $\geq h_{cm}$.

Tipico caso di fallimento: “servono 5 ore di matematica nel liceo classico ma i due docenti di Ao27 hanno già saturato le loro ore in altri istituti”. π Tantum lo intercetta immediatamente e mostra il deficit.

12.3.2 Controllo 2: subject supply vs demand

Definizione

Per ogni materia m , la *domanda* è la somma di h_{cm} su tutte le classi c . L'*offerta* è la somma delle ore residue dei docenti abilitati a m . Verifica che offerta $\geq$ domanda.

Se questo fallisce, l'intera materia è in deficit strutturale: nessuna distribuzione potrà coprire tutte le classi. Soluzione: aggiungere docenti, ridurre il monte ore, o convocare supplenze.

12.3.3 Controllo 3: Hall sampling random-subset

Definizione

Si campionano casualmente K sotto-insiemi di lezioni (tipicamente $K = 200$, di taglie fra 5 e 50) e per ognuno si verifica direttamente la condizione di Hall.

Quando un'istanza ha un *sottoinsieme cattivo* di lezioni che soddisfano i controlli 1 e 2 ma non Hall, il campionamento ha alta probabilità di “inciampare” su di esso e segnalarlo. È un controllo *euristico* (può non beccare casi rari) ma nella pratica π Tantum lo trova sufficiente.

12.4 Esempio mini-completo

Esempio

Scuola: 3 classi (1A, 1B, 1C), tutte con 2 ore di matematica. Due docenti di Ao27 (Adami e Bruni), Adami con 4 ore residue, Bruni con 3 ore residue. Totale offerta: 7. Totale domanda: 6. Controllo 2: **ok**.

Controllo 1: per ogni (classe, mat), 2 ore richieste. Adami+Bruni hanno entrambi ≥ 2 ore residue. **ok**. Aggiungiamo un vincolo: Bruni non può insegnare in 1C. Domanda di 1C-mat: 2 ore. Offerta di docenti compatibili con 1C-mat: solo Adami con 4 ore residue. **ok**. Ma se aggiungiamo un secondo vincolo: Adami satura 3 ore con altre materie e ne restano solo 1 per le classi 1A/B/C matematica. Allora la domanda complessiva su Adami+Bruni per matematica diventa 6 ore, l'offerta $1+3 = 4$. Controllo 2 fallisce: si vede subito che servono 2 ore in più (rilassare un altro docente, o ridurre il monte).

12.5 L'integrazione nell'UI

In π Tantum il pre-check di Hall è esposto in tre punti dell'interfaccia:

1. Bottone inline nella card di Phase A del Workflow.
2. Riga in AdvancedTechniquesCard, con bottone “Esegui” che lancia tutti e tre i controlli.
3. Sezione del tab Statistiche/Diagnostica, dove il risultato è mostrato come run asincrono visualizzabile in dettaglio.

L'output è sempre una lista di “deficit” con il sotto-insieme problematico evidenziato. Esempio:

HALL CHECK FAILED

Deficit 1 (controllo 2):

materia=Matematica, domanda=18, offerta=15

-- 3 ore mancanti

Deficit 2 (controllo 1):

classe=4B, materia=Storia: 2 ore richieste,

ma il docente compatibile (Conti) ne ha solo 1

residua.

Hall è il *primo* controllo che il sistema esegue. Se fallisce, l'utente vede subito quale gruppo di classi/materie è il problema, prima ancora che la pipeline parta.

12.6 Limiti e quando non basta

Attenzione

- Il pre-check è *necessario* ma non *sufficiente*: può passare anche quando il problema è infeasibile per ragioni *temporali* (es. lo stesso docente serve in due slot incompatibili). Un Hall “ok” garantisce solo che *le ore totali bastano*, non che si possano disporre nello spazio settimanale.
- I controlli 1 e 2 sono esatti; il controllo 3 è euristico (random sampling). Aumentare K rende il controllo più robusto a discapito del tempo.
- Hall non considera vincoli logici DSL complessi (es. “il prof X e Y mai stesso giorno”). Quelli vengono verificati solo durante l'esecuzione di CP-SAT.

12.7 Approfondimento: l'algoritmo di Hopcroft-Karp

Approfondimento

Verificare *esattamente* l'esistenza di un matching che copre un lato di un grafo bipartito si può fare in tempo $O(E\sqrt{V})$ con l'algoritmo di Hopcroft-Karp del 1973 (Hopcroft e Karp 1973). È quello che usa internamente OR-Tools per il propagatore globale AllDifferent: ogni volta che CP-SAT propaga un vincolo del tipo “ n variabili devono prendere valori distinti”, costruisce il grafo bipartito *variabili vs valori* e chiama Hopcroft-Karp per verificare la fattibilità residua. Se non c'è matching $\rightarrow$ contraddizione $\rightarrow$ backtracking immediato.

In π Tantum non si usa Hopcroft-Karp esplicitamente nel pre-check (i tre controlli rapidi sono più che sufficienti nei casi pratici), ma l'algoritmo è *dentro* CP-SAT ad ogni propagazione.

12.8 Connessioni

Connessioni

- **CP-SAT** (cap. 9): la propagazione del vincolo AllDifferent in CP-SAT usa internamente un calcolo di matching basato sullo stesso teorema di Hall.
- **Decomposizione spettrale** (cap. 10): Hall si esegue *su ciascun cluster* oltre che globalmente.
- **Workflow**: il pre-check è raccomandato come *primo* step della pipeline; risparmia minuti di CP-SAT su istanze infeasible.

Per approfondire

- Hall 1935: l'articolo originale di Hall. Cinque pagine, una gemma.
- Hopcroft e Karp 1973: l'algoritmo polinomiale per matching bipartito, fondante della complessità moderna.
- Lovász e Plummer 1986: *Matching Theory*, libro classico per chi vuole approfondire.
- Papadimitriou e Steiglitz 1998, capp. 10–11: sezione accessibile sui matching e i flussi.

13 Lagrangian Relaxation e Column Generation

“If you cannot solve a problem, relax it.”

adagio della ricerca operativa, attribuito a vari autori

CI SONO problemi che, pur essendo risolvibili in linea di principio da CP-SAT, sono *così grandi* che il solver impiega ore per produrre una soluzione mediocre. Una scuola con 200 classi, 400 docenti, 12 indirizzi non sta nei tempi di Phase B di un ciclo standard. Per questi casi π Tantum offre due tecniche di *rilassamento e decomposizione* di derivazione classica: il rilassamento lagrangiano e la generazione di colonne. Entrambe sono di default *disattivate*, da abilitare esplicitamente da chi gestisce la pipeline.

13.1 Rilassamento lagrangiano: dualizzare i ponti

Nota storica

Il rilassamento lagrangiano nasce nella ricerca operativa degli anni '60-'70. Held e Karp (1971) (Held e Karp 1971) lo applicarono al *Traveling Salesman Problem* ottenendo risultati eccellenti su istanze grandi. Geoffrion (1974) (Geoffrion 1974) formalizzò l'approccio per la programmazione intera generale; Fisher (1981) (Fisher 1981) ne fece il manuale operativo per chi voleva applicarlo nella pratica industriale.

Intuizione

L'idea con un'analogia. Hai un problema enorme che non riesci a risolvere. Decidi di *dimenticare* alcuni vincoli scomodi (che “legano” parti diverse del problema) e di *punirli* con una multa proporzionale alla loro violazione. Risolvi il problema rilassato. Misuri quanto i vincoli scomodi sono violati. Aggiusti la multa: dove sono violati di più, alzi la multa; dove sono soddisfatti con margine, la abbassi. Iteri.

Si dimostra che, sotto ipotesi tecniche, le multe convergono a un valore tale che la soluzione del problema rilassato soddisfa anche i vincoli “dimenticati” — ottenendo quindi una soluzione del problema originale.

13.1.1 Formalizzazione

Supponi un problema di minimo:

$$\min_{x \in X} f(x) \quad \text{soggetto a } g(x) \leq 0$$

dove X è un insieme di soluzioni “facili” (es. unione di sotto-problemi indipendenti) e $g(x) \leq 0$ rappresenta i vincoli “difficili” (i ponti fra sotto-problemi).

Definizione

La *Lagrangiana* è:

$$L(x, \lambda) = f(x) + \lambda^\top g(x)$$

con $\lambda \geq 0$ vettore dei *moltiplicatori di Lagrange*.

Il *problema rilassato* è:

$$q(\lambda) = \min_{x \in X} L(x, \lambda)$$

La funzione $q(\lambda)$ è detta *funzione duale*.

Come si legge: per ogni valore del vettore λ , risolvi un problema senza i vincoli scomodi $g(x) \leq 0$ ma con una penalità $\lambda^\top g(x)$ aggiunta all'obiettivo. Se $g(x) > 0$ (vincolo violato), la penalità sale; se $g(x) < 0$ (vincolo soddisfatto con margine), la penalità scende (il termine è negativo). Questo “incentiva” x a soddisfare i vincoli senza forzarli.

13.1.2 Subgradient ascent: aggiornare i moltiplicatori

Si dimostra che $q(\lambda)$ è concava in λ e che $\max_{\lambda \geq 0} q(\lambda) \leq \min_{x: g(x) \leq 0} f(x)$ (weak duality). L'algoritmo classico aggiorna λ con un passo di *subgradient ascent*:

$$\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g(x_k))$$

dove $\alpha_k > 0$ è il *passo*, tipicamente $\alpha_k = \alpha_0 / (1 + k)$ in π Tantum.

Come si legge: dopo aver risolto il rilassato e trovato x_k , calcolo quanto i vincoli sono violati ($g(x_k)$). Aggiorno λ aggiungendoci un multiplo di $g(x_k)$: i moltiplicatori associati a vincoli più violati salgono di più. Tronco a zero per mantenere $\lambda \geq 0$.

13.1.3 Cosa rilassare nel timetabling?

In π Tantum la decomposizione lagrangiana si applica *dopo* la decomposizione spettrale: si rilassano i *ponti inter-cluster*, cioè i vincoli che legano lezioni di cluster diversi (tipicamente docenti che insegnano a cavallo fra due cluster, oppure vincoli logici DSL che attraversano cluster).

Ogni cluster diventa un sotto-problema CP-SAT indipendente con un costo aggiuntivo $\lambda_b \cdot (\text{violazione del ponte } b)$. Iterando il subgradient, il sistema converge a un assegnamento dei ponti coerente con tutti i cluster.

13.1.4 Esempio in miniatura

Esempio

Due cluster: A (3 classi) e B (3 classi). Un docente, Conti, deve insegnare 2 ore in cluster A (storia in 1A) e 2 ore in cluster B (storia in 4B). Vincolo ponte: “Conti non in due slot contemporanei”.

Senza rilassamento: Phase B risolve A e B insieme, vincolo HARD globale, propagazione attraverso i cluster (lento).

Con rilassamento: λ inizia a 0. Risolvo A da solo (mette Conti in lun-1, lun-2). Risolvo B da solo (mette Conti in lun-1, lun-2). Conflitto: Conti in lun-1 e lun-2 in entrambi i cluster.

Aggiorno $\lambda \leftarrow \alpha \cdot 2 = 0.2$ (violazione = 2 slot duplicati).

Iter 2: A ora penalizza Conti in lun-1, lun-2 con costo 0.2 ciascuno; trova Conti in mar-1, mar-2 (costo SOFT più alto ma niente penalità). B mantiene Conti in lun-1, lun-2. Conflitto risolto, ma A ha pagato un piccolo costo SOFT.

Dopo 5–10 iterazioni il sistema trova un equilibrio in cui Conti è in slot diversi nei due cluster e nessun vincolo globale è violato.

13.1.5 Perché non sempre?

Attenzione

Lagrangian Relaxation è potente ma costoso da implementare bene:

- Richiede il *tuning* del passo α_0 . Troppo grande $\rightarrow$ oscillazioni; troppo piccolo $\rightarrow$ convergenza lentissima.
- Può *non convergere* se il problema duale ha un *duality gap* grosso (frequente nei problemi combinatori interi).
- La soluzione finale del rilassato *non* è necessariamente fattibile per il problema originale: serve un passo di “ricostruzione” (in π Tantum: un giro di SA per chiudere i ponti residui).

Per queste ragioni π Tantum lo offre come *opzione avanzata*, non come default.

13.2 Column Generation: il catalogo immobiliare

Nota storica

La generazione di colonne (*Column Generation*) nasce nel 1960 con Dantzig e Wolfe (Dantzig e Wolfe 1960). L'idea: in molti grandi problemi di programmazione lineare, *la maggior parte delle variabili non serve*; sono attive solo poche. Invece di costruire il problema con tutte le variabili, lo si costruisce con poche e si *generano nuove colonne* (= nuove variabili) solo quando servono.

Intuizione

Analogia. Sei un'agenzia immobiliare con un milione di appartamenti in catalogo. Un cliente cerca un trilocale in zona centro. Non gli passi tutto il catalogo: gli proponi 3–4 appartamenti che pensi gli vadano bene. Se non gli piacciono, ne tiri fuori altri 3–4 con caratteristiche diverse. Itera finché il cliente è soddisfatto. Hai usato il catalogo intero ma esaminandone solo una frazione. Column generation fa lo stesso: ha un *master problem* (piccolo, con poche variabili) e uno o più *sub-problem* che *generano* nuove variabili da aggiungere al master, basandosi sui prezzi duali correnti del master.

13.2.1 Schema operativo

1. Costruisci il master problem con un sotto-insieme iniziale di variabili (colonne).
2. Risolvi il master come LP. Ottieni i prezzi duali π delle restrizioni.
3. Per ogni sotto-problema, cerca una colonna con *costo ridotto negativo* ($c_j - \pi^T a_j < 0$). Se ne trovi, aggiungila al master.
4. Se nessun sotto-problema produce una colonna vantaggiosa, fermati: il master corrente è ottimo per l'LP rilassato.
5. (Per problemi interi) lancia un branch-and-bound sopra (*branch-and-price*).

13.2.2 Cosa generano le colonne nel timetabling?

In π Tantum la formulazione è: il master è un *set covering* dove ogni colonna rappresenta un *pattern settimanale completo* di un docente (la sequenza di tutti i suoi slot in tutte le sue classi). Il master sceglie un pattern per ogni docente in modo da coprire tutte le ore-cattedra delle classi. I sub-problem generano nuovi pattern docenti, uno per docente, sulla base dei prezzi duali.

Il vantaggio: il numero di pattern possibili per docente è combinatorialmente esplosivo (centinaia di milioni), ma il master ne usa solo qualche decina. Il sotto-problema è relativamente piccolo (un docente alla volta) e CP-SAT lo risolve in pochi millisecondi.

13.2.3 Limiti pratici

Attenzione

π Tantom offre column generation come *skeleton single-iteration* per scuole oltre 200 classi: una sola generazione di colonne, poi master LP, poi branch-and-bound. Non è l'implementazione completa di un branch-and-price moderno (richiederebbe gestione delle strategie di branching, gestione dei tagli, etc.). È considerato un *pre-totale* che produce un upper bound per l'obiettivo prima di lanciare la pipeline standard. Default: `enable_cg=False`.

13.3 Connessioni

Connessioni

- **Decomposizione spettrale** (cap. 10): naturale alleato del rilassamento lagrangiano (i cluster diventano i sotto-problemi, i ponti inter-cluster diventano i vincoli da dualizzare).
- **CP-SAT** (cap. 9): risolve i sotto-problemi della decomposizione lagrangiana e i sub-problem della column generation.
- **Metaeuristiche** (cap. 11): un giro di SA dopo Lagrangian per chiudere i ponti residui.

Per approfondire

- Geoffrion 1974, Fisher 1981: i due riferimenti classici sul rilassamento lagrangiano per IP.
- Held e Karp 1971: l'applicazione storica al TSP.
- Dantzig e Wolfe 1960: il paper originale sulla decomposizione di Dantzig-Wolfe e generazione di colonne.
- Desrosiers e Lübbecke 2005: *A Primer in Column Generation*, manuale moderno e accessibile.
- Wolsey 1998: capp. 10–11 sui rilassamenti e le decomposizioni in IP.

14 Monte Carlo: simulare per capire

“In any conflict between two ideas, lapse into random sampling and you’ll learn surprising things.”

adagio del calcolo statistico

DOPO aver prodotto un orario, una domanda naturale del coordinatore è: *quanto è robusto rispetto a piccoli cambiamenti?* Cosa succede se un docente cambia indisponibilità all’ultimo minuto? Cosa succede se una classe si sposta di sezione? Cosa succede se viene aggiunta una compresenza dell’ultim’ora?

Una risposta secca è impossibile. Una risposta statistica, invece, si può ottenere con il metodo *Monte Carlo*: si *perturba* l’input in modo casuale, si *rilancia* il solver molte volte, e si *guarda la distribuzione* dei risultati.

14.1 La storia del nome

Nota storica

Il metodo Monte Carlo nasce a Los Alamos negli anni ’40 nel contesto del progetto Manhattan. Stanislaw Ulam, durante una convalescenza, ideò di simulare l’evoluzione dei neutroni in una bomba atomica con simulazioni stocastiche. Lo chiamò “Monte Carlo” come scherzo, perché suo zio amava il casinò di Monte Carlo a Monaco. Metropolis e Ulam pubblicarono il primo articolo formale nel 1949 (Metropolis e Ulam 1949).

L’idea si è poi diffusa in fisica, finanza, biologia, statistica. Oggi *Monte Carlo simulation* è un metodo trasversale: ogni volta che un sistema deterministico è troppo complesso per essere analizzato esattamente, si campiona stocasticamente e si lavora sulle distribuzioni.

14.2 Sensitivity analysis: cosa misurare

Definizione

La *sensitivity analysis* consiste nel *perturbare* un parametro di un modello e nel misurare quanto cambia l’output. Versione Monte Carlo: invece di perturbare un parametro alla volta, perturba molti parametri contemporaneamente con distribuzioni assegnate, e analizza la distribuzione dell’output.

In π Tantum si offrono diverse sensitività, attivabili dalla pagina Diagnostica:

- **Indisponibilità docenti:** scegli K scenari in cui un docente casuale acquisisce un’indisponibilità in uno slot casuale. Per ognuno, rilancia la pipeline e registra: ha trovato soluzione? quanto è cambiato l’obiettivo? quante lezioni sono state spostate?
- **Aggiunta compresenze:** simula l’arrivo dell’ultimo momento di una compresenza in una classe random per una materia random.
- **Spostamento di una classe:** simula il cambio di curriculum/indirizzo di una classe.
- **Generico:** l’utente passa una funzione di perturbazione personalizzata via DSL.

14.3 Schema operativo

1. Fissa un seed iniziale e un numero K di campioni (default: 50).
2. Per ogni $i = 1, \dots, K$:
 - a) Applica una perturbazione casuale δ_i all'input.
 - b) Rilancia la pipeline (Phase A + B + C + D).
 - c) Registra: feasibility, obiettivo finale, tempo, numero di lezioni cambiate rispetto al baseline.
3. Calcola statistiche descrittive: media, deviazione standard, mediana, percentili 5/95, frazione di scenari infeasible.
4. Visualizza la distribuzione (istogramma) del valore obiettivo.

14.3.1 Esempio numerico

Esempio

Scuola medium: 28 classi, 50 docenti, baseline con obiettivo $f_0 = 11.4$. Lanciamo Monte Carlo con $K = 50$ scenari di tipo “indisponibilità docente”.

Risultati: 47 scenari producono soluzione fattibile (3 infeasible); fra i 47, $\bar{f} = 12.1$, $\sigma = 0.8$, mediana = 12.0, $P_5 = 11.0$, $P_{95} = 13.6$. Il numero medio di lezioni spostate è 6.3.

Lettura: il sistema è *robusto in media* (l'obiettivo peggiora di solo $\sim 6\%$), ma il 6% degli scenari diventa infeasible. Conviene investigare quali docenti sono “critici” (la cui indisponibilità aggiuntiva rende il problema infeasible).

14.4 Quando è utile

Intuizione

- **Pianificazione preventiva:** a settembre il coordinatore vuole sapere quali docenti sono “critici” prima che si manifestino le assenze. Monte Carlo li identifica.
- **Stima dell'impatto di una nuova richiesta:** un docente chiede di non lavorare il martedì. Monte Carlo simula l'aggiunta del vincolo (anche su altri docenti, per analogia) e dice quanto perde l'obiettivo in media.
- **Validazione di una soluzione:** una soluzione con basso f_0 ma alta varianza nel Monte Carlo è *fragile*; meglio cercarne una con f_0 leggermente più alto ma varianza minore.

14.5 Quando non basta

Attenzione

- Monte Carlo è *costoso*: K esecuzioni della pipeline. Per scuole grandi $K = 50$ può richiedere ore. π Tantum lo lancia come run asincrono e mostra il progresso live.
- L'analisi è *indicativa*, non garantita: se K è basso (es. 20), le statistiche hanno alta varianza. Aumentare K migliora ma costa.
- Monte Carlo *non* dice perché una perturbazione rompe il sistema; dice solo che lo rompe. Per capire perché serve guardare i log dei singoli scenari (CP-SAT mostra il sottoinsieme di vincoli incompatibili).

14.6 Connessioni

Connessioni

- **Hall pre-check** (cap. 12): ogni scenario Monte Carlo passa prima per Hall; gli scenari che falliscono Hall sono contati come “infeasible immediato” senza dover lanciare CP-SAT.
- **Statistica applicata** (cap. 16): le distribuzioni prodotte da Monte Carlo si analizzano con KS-test, istogrammi, percentili come qualsiasi altra distribuzione.

Per approfondire

- Metropolis e Ulam 1949: il primo paper formale. Cinque pagine.
- Robert e Casella 2004: *Monte Carlo Statistical Methods*, manuale di riferimento moderno.
- Wasserman 2004: capp. 24–25 sull’inferenza Monte Carlo, accessibile.

15 Analisi del grafo bipartito: vedere la struttura

“A network is more than the sum of its nodes.”

Mark E. J. Newman, *Networks: An Introduction* (Newman 2010)

OGNI scuola può essere vista come un *grafo bipartito* che ha per nodi le classi e i docenti, con un arco fra un docente e una classe se il docente insegna in quella classe (eventualmente pesato dal numero di ore). Questo grafo non è un'astrazione gratuita: ne emergono *misure quantitative* che dicono molto sulla struttura organizzativa della scuola e sulla difficoltà del problema di scheduling.

In π Tantum la pagina Diagnostica espone tre misure classiche della network analysis: *modularità*, *betweenness centrality* e *densità*. Ciascuna ha un significato preciso e un suggerimento pratico associato.

15.1 Modularità: quanto è “a cluster” la scuola

Nota storica

La modularità è una misura introdotta da Newman e Girvan nel 2004 (Newman e Girvan 2004) per quantificare la *community structure* di un grafo: è alta quando il grafo si divide naturalmente in gruppi (comunità) ben separati, è bassa o negativa quando il grafo è omogeneo. È diventata una delle misure più usate nella scienza delle reti.

Definizione

Per un grafo G con m archi e una partizione dei nodi in comunità $C_1, \dots, C_k$, la modularità è:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

dove A_{ij} è la matrice di adiacenza, k_i è il grado del nodo i , $\delta(c_i, c_j) = 1$ se i e j sono nella stessa comunità, o altrimenti.

Come si legge: per ogni coppia di nodi nella stessa comunità, si confronta il *numero effettivo* di archi fra loro (A_{ij}) con il *numero atteso* se gli archi fossero distribuiti casualmente preservando i gradi ($k_i k_j / 2m$). Se i nodi della stessa comunità sono *più connessi* di quanto ci si aspetterebbe a caso, Q è positiva. Valori tipici: $Q < 0.3$ (struttura debole), $0.3 < Q < 0.5$ (struttura moderata), $Q > 0.5$ (struttura marcata).

Esempio

La scuola di fig. 10.1 (cap. 10) ha modularità $Q \approx 0.42$ (struttura moderata: due cluster ben separati ma con un ponte). Una scuola con un solo indirizzo e docenti che insegnano un po' ovunque ha tipicamente $Q < 0.2$.

Suggerimento pratico: se $Q > 0.4$, la decomposizione spettrale (cap. 10) funzionerà bene; la pipeline può essere accelerata risolvendo cluster indipendenti. Se $Q < 0.2$, la decomposizione produrrà cluster artificiali e conviene risolvere monoliticamente.

15.2 Betweenness centrality: i “ponti” del sistema

Nota storica

La *betweenness* fu introdotta da Linton Freeman nel 1977 (Freeman 1977) nella sociologia dei piccoli gruppi: misura quanto un individuo è “intermediario” nelle comunicazioni del gruppo. Si è poi diffusa in tantissimi campi (citation networks, Internet, biology).

Definizione

Per un nodo v , la *betweenness centrality* è:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

dove σ_{st} è il numero di shortest path da s a t , $\sigma_{st}(v)$ il numero di quelli che passano per v .

Come si legge: $C_B(v)$ è alta se v è “in mezzo” a molti percorsi del grafo. Un docente con betweenness alta è quello che, se si toglie, “sconnette” parecchio il sistema. Tipicamente sono docenti che insegnano in più indirizzi diversi, fungendo da ponte fra cluster.

Suggerimento pratico: i top- k docenti per betweenness sono quelli su cui il timetabling è più fragile. Una loro indisponibilità aggiuntiva ha alta probabilità di rendere il problema infeasible. Vale la pena tenerli sotto controllo durante l’anno.

15.3 Densità: quanto è “pieno” il grafo

Definizione

La *densità* di un grafo bipartito con n_C classi e n_D docenti, che ha $|E|$ archi, è:

$$\rho = \frac{|E|}{n_C \cdot n_D}$$

Vale fra 0 (nessun arco) e 1 (tutti i docenti insegnano in tutte le classi). Per scuole italiane reali, ρ è tipicamente 0.05–0.15.

Suggerimento pratico: ρ alta significa molti docenti che insegnano in molte classi (es. ITIS dove i docenti coprono più indirizzi). π Tantum tipicamente risolve in più tempo le scuole con ρ alta perché i vincoli fra docenti diversi sono più intrecciati. Valori $\rho > 0.3$ sono considerati “densi” e suggeriscono di *non* usare la decomposizione spettrale (vedi sopra).

15.4 Visualizzazione e interpretazione

In Diagnostica π Tantum mostra:

- Una *matrice riassuntiva* con Q , ρ , numero di componenti connesse, top-5 docenti per betweenness.
- Un *istogramma* dei gradi (quante classi per docente; quanti docenti per classe).
- Un *heatmap* della matrice di adiacenza con righe ordinate per cluster spettrale: a colpo d’occhio si vedono i cluster e i ponti.

Caso di studio

Liceo “Manzoni”, 24 classi. $Q = 0.51$, $\rho = 0.09$, top docente per betweenness: prof Esposito (insegna Ao26 in liceo classico e in liceo scientifico, fa da ponte fra i due). Modularità alta $\rightarrow$ decomposizione spettrale produce 3 cluster (classico, scientifico, linguistico) ben separati. Esposito è l'unico ponte significativo; basta tenerlo come “shared” fra due cluster con vincolo lagrangiano. Pipeline completa in 4 min.

15.5 Connessioni

Connessioni

- **Decomposizione spettrale** (cap. 10): la modularità Q predice la qualità della decomposizione.
- **Hall pre-check** (cap. 12): ρ alta in pratica significa molti vincoli di matching da verificare; Hall richiede più tempo.
- **Monte Carlo** (cap. 14): i docenti con alta betweenness sono i candidati naturali per le perturbazioni del Monte Carlo (testare cosa succede se diventano indisponibili).

Per approfondire

- Newman e Girvan 2004: il paper originale sulla modularità (Newman & Girvan).
- Freeman 1977: il paper originale sulla betweenness.
- Newman 2010: *Networks: An Introduction*, manuale moderno e accessibile per la network analysis.

16 Statistica applicata: leggere i risultati

“Statistics is the grammar of science.”

Karl Pearson, attribuito

DOPO aver eseguito una pipeline (o un batch Monte Carlo), il coordinatore vuole sapere: la soluzione è bilanciata? alcune classi sono sistematicamente svantaggiate? c'è una correlazione fra qualcosa che ho dato in input e la qualità del risultato?

Per rispondere π Tantum integra una piccola toolbox di statistica applicata, accessibile dalla pagina Diagnostica. Le tre famiglie sono: *distribuzioni e istogrammi*, *correlazioni e regressioni*, *test di ipotesi* (chi-quadro, Kolmogorov-Smirnov).

16.1 Distribuzioni: vedere la forma dei dati

Definizione

Un *istogramma* di una variabile X è una funzione che, suddiviso il range di X in b *bin* di larghezza uguale, conta quanti valori cadono in ognuno. È una stima non parametrica della densità di X .

In π Tantum si possono richiedere istogrammi di:

- Ore di lezione per docente (mostra se i carichi sono distribuiti uniformemente o se ci sono outlier).
- “Buchi” per docente nella settimana (ore vuote fra lezioni).
- Slot per materia per ora del giorno (mostra l'effetto della preferenza “mattutino”).
- Tempo di esecuzione per cluster nella decomposizione spettrale.

16.1.1 Bin sizing

π Tantum usa la regola di Freedman-Diaconis come default:

$$h = 2 \cdot \text{IQR}(X) \cdot n^{-1/3}$$

dove IQR è l'*interquartile range* ($Q_3 - Q_1$) e n il numero di osservazioni. Il numero di bin è $\lceil (\max - \min) / h \rceil$. L'utente può sempre forzare un numero specifico.

La scelta del numero di bin è un piccolo problema in sé: troppi pochi $\rightarrow$ perdi struttura; troppi $\rightarrow$ rumore.

16.1.2 Test di Kolmogorov-Smirnov

Definizione

Il *test KS* confronta una distribuzione empirica con una distribuzione di riferimento (o due distribuzioni empiriche fra loro). La statistica del test è:

$$D = \sup_x |F_n(x) - F(x)|$$

ovvero la massima distanza verticale fra la CDF empirica F_n e la CDF di riferimento F .

In π Tantum il test KS è usato ad esempio per confrontare la distribuzione delle ore-per-docente fra “baseline” e “post-perturbazione Monte Carlo”. Un valore D piccolo (con p -value alto) indica che la distribuzione è robusta; un D grande indica che la perturbazione ha significativamente alterato il bilancio.

16.2 Correlazioni e regressioni

16.2.1 Correlazione di Pearson

Definizione

La *correlazione di Pearson* fra due variabili X, Y è:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

con $\text{Cov}(X, Y) = E[(X - \bar{X})(Y - \bar{Y})]$. Vale fra -1 e $+1$: $r > 0$ relazione positiva, $r < 0$ negativa, $r \approx 0$ nessuna relazione lineare.

Come si legge: r vicino a ± 1 indica relazione *lineare* forte; r vicino a 0 indica *nessuna relazione lineare*, ma non esclude relazioni non lineari. Un valore di $r = 0.5$ significa: *circa il 25% della varianza di Y è spiegato linearmente da X* ($r^2 = 0.25$).

Esempio

Calcoliamo la correlazione fra “numero di indisponibilità HARD del docente” e “numero di buchi nel suo orario finale”. Se $r = +0.6$, significa che più un docente ha indisponibilità, più tende ad avere buchi. Insight per il coordinatore: i docenti con molte indisponibilità si prendono comunque l'orario peggiore in termini di buchi; forse vale la pena rivedere le loro indisponibilità o accettare di fare strappi sulle preferenze altrui.

16.2.2 Regressione lineare (OLS)

Definizione

Una *regressione lineare ordinaria* (OLS) modella Y come funzione lineare di una o più X :

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

e stima i β minimizzando la somma dei quadrati dei residui: $\hat{\beta} = (X^\top X)^{-1} X^\top Y$.

Come si legge: ogni coefficiente β_i dice di quanto cambia Y se X_i aumenta di 1, tenendo le altre X_j fisse. Il coefficiente β_0 è l'*intercetta*. π Tantum riporta anche R^2 (frazione di varianza spiegata) e i p -value per ciascun β_i .

In π Tantum la pagina “Correlazioni e regressioni” permette di costruire interattivamente il modello (scegli le X , scegli la Y , scegli OLS o Logit) e mostra i coefficienti con intervalli di confidenza.

16.2.3 Regressione logistica (Logit)

Definizione

La *regressione logistica* modella la probabilità di un evento binario:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots)}}$$

Si usa quando la Y è binaria. Esempio in π Tantum: “probabilità che una classe abbia ≥ 3 buchi settimanali in funzione del suo numero di indirizzi serviti e del numero di docenti coinvolti”.

16.3 Chi-quadro: indipendenza

Definizione

Il test *chi-quadro* χ^2 verifica se due variabili categoriali sono indipendenti. Si costruisce la tabella di contingenza, si calcola

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

dove O_{ij} è la frequenza osservata e E_{ij} quella attesa sotto ipotesi di indipendenza. Si confronta con la distribuzione χ^2 con $(r - 1)(c - 1)$ gradi di libertà.

Tipico uso: “il giorno della settimana e la materia sono indipendenti?” Se p-value basso, la materia *influisce* su quale giorno si tiene (es. matematica concentrata al mattino). È un’indicazione che il sistema sta rispettando la preferenza “mattutino”.

16.4 Visualizzazione interattiva

Tutti i grafici in π Tantum sono prodotti con ECharts 6, lazy-loaded sul frontend. Hanno sempre i bottoni:

- **Export PNG** (con pixel-ratio 2 e sfondo bianco).
- **Clear graph**: per fare spazio in finestra.
- **Restore zoom**: per resettare lo zoom dopo esplorazione.

I parametri (numero di bin, scelta delle X , etc.) sono modificabili dall’UI senza dover rilanciare il run; le visualizzazioni sono ricalcolate al volo dal browser.

16.5 Connessioni

Connessioni

- **Monte Carlo** (cap. 14): produce campioni che si analizzano con distribuzioni e KS-test.
- **Analisi del grafo** (cap. 15): la modularità e la betweenness sono input naturali per le regressioni (es. “il tempo di esecuzione del solver dipende dalla modularità del grafo?”).
- **Workflow**: dopo ogni esecuzione il sistema popola automaticamente le tabelle delle statistiche (lezioni, buchi, ore-isolate, etc.); il coordinatore le può esplorare immediatamente.

Per approfondire

- Wasserman 2004: *All of Statistics*, manuale conciso di statistica matematica accessibile agli informatici.
- James et al. 2013: *An Introduction to Statistical Learning*, gratuito online, ottimo per regressioni e modelli predittivi.

17 Il DSL generico: un linguaggio per i vincoli

“A domain-specific language is a computer language specialized to a particular application domain.”

Martin Fowler, *Domain-Specific Languages* (Fowler 2010)

PRIMA o poi tutti i sistemi di scheduling devono affrontare lo stesso problema editoriale: gli utenti vogliono esprimere vincoli che non sono mai stati previsti. “Mai due ore di matematica consecutive nei giovedì pari delle classi prime”; “almeno un’ora di laboratorio per ogni classe di scientifico”; “il prof Bianchi e la prof Verdi mai stesso slot stessa giornata”. È tecnicamente impossibile prevedere tutto e codificarlo hard-coded nel solver.

La soluzione classica è un *Domain-Specific Language*: un piccolo linguaggio dichiarativo che gli utenti possono usare per scrivere i propri vincoli. In π Tantum il DSL si chiama *DSL generico* e lo descriviamo in questo capitolo dal punto di vista implementativo.

17.1 Filosofia: “un parser, molti compilatori”

Intuizione

Il DSL generico ha una grammatica unica e un parser unico, ma ha più *back-end*: lo stesso vincolo si può *valutare* sui dati correnti (per check sincroni o per filtri lista), si può *compilare* a vincoli CP-SAT (per Phase B), oppure si può *compilare* a delta SOFT per il drag-and-drop in tempo reale dell’editor Orario. È lo schema classico “parser $\rightarrow$ AST $\rightarrow$ molti compilatori” del Dragon Book (Aho et al. 2006).

17.2 Grammatica (sintesi)

La grammatica del DSL è costruita con discesa ricorsiva. La forma BNF semplificata:

```
program := constraint+
constraint := quantifier source ('where' filter)? ':' body
quantifier := 'forall' | 'exists' | 'count'
source := IDENT 'in' (IDENT | path)
path := IDENT ('.' IDENT)+
filter := expr
body := expr
expr := orexpr
orexpr := andexpr ('or' andexpr)*
andexpr := notexpr ('and' notexpr)*
notexpr := 'not'? cmp
cmp := value relop value | builtin
relop := '==' | '!=' | '<' | '<=' | '>' | '>='
        | 'in' | 'not in' | 'contains'
value := literal | path | builtin
builtin := 'consecutive' '(' value ',' value ')'
        | 'same_day' '(' value ',' value ')'
```



```
| 'lesson' '(' value ',' value ')'  
| 'has_tag' '(' STRING ')'
```

Le sorgenti supportate sono: teachers, classes, subjects, classrooms, groups, students, lessons, slots; più qualsiasi *path* che restituisca una collezione (es. s.groups, l.classroom.tags).

17.3 Parser e AST

Definizione

Il *parser* è un *recursive descent* hand-rolled (Python puro, no PLY/Lark). Riconosce token con una funzione `tokenize()` basata su regex, e costruisce un AST (*Abstract Syntax Tree*) dove ogni nodo è una piccola dataclass: Quant, Count, BinOp, Cmp, Ref, Lit, Builtin.

Esempio: l'espressione

```
forall t in teachers where t.subject == "Fisica":  
    count l in lessons where l.teacher == t and  
                                l.classroom.type == "lab_fisica":  
        l == 1
```

produce questo AST (semplificato):

```
Quant(kind='forall', var='t', source='teachers',  
      filter=Cmp('==', Ref(['t', 'subject']), Lit('Fisica')),  
      body=Count(var='l', source='lessons',  
                 filter=And([  
                     Cmp('==', Ref(['l', 'teacher']), Ref(['t'])),  
                     Cmp('==', Ref(['l', 'classroom', 'type']),  
                         Lit('lab_fisica'))]),  
      cmp=Cmp('==', Var('count'), Lit(1))))
```

17.4 Tre back-end

17.4.1 Evaluator: “vale o no?”

Definizione

L'*evaluator* prende un AST e i dati correnti del sistema (teachers, lessons, ecc.) e restituisce True/False per i vincoli forall/exists, e un intero per i count.

È usato in modalità sincrona per i filtri lista (“mostrami tutti i docenti per cui questo vincolo è violato”), per i check pre-pipeline (verifica di consistenza), e come oracolo nei test.

17.4.2 Compilatore CP-SAT

Definizione

Il *compilatore CP-SAT* traduce l'AST in vincoli OR-Tools. Le quantificazioni diventano cicli sui domini; le comparazioni diventano vincoli aritmetici; i count diventano somme con vincoli di uguaglianza/disuguaglianza.

Esempio: `count l in lessons where l.teacher == t: l == 1` diventa

```
total = sum(BoolVar(f"l_{l.id}_active") for l in lessons
            if l.teacher_id == t.id)
model.Add(total == 1)
```

17.4.3 Compilatore delta SOFT

Definizione

Il *compilatore delta* produce, dato un vincolo SOFT e una proposta di mossa (drag-and-drop di una lezione), il *delta* dell'obiettivo (positivo se la mossa peggiora, negativo se migliora) in $O(1)$.

È quello che permette all'editor di mostrare in tempo reale “+0.3 SOFT” o “−0.7 SOFT” mentre l'utente trascina una lezione.

17.5 Estensione: aggiungere un built-in

Per aggiungere un nuovo built-in (es. `distinct_days`):

1. Estendere la lista dei token nel `tokenize()`.
2. Aggiungere il caso nel parser (`_parse_builtin`).
3. Implementare la valutazione nell'evaluator.
4. Implementare la compilazione CP-SAT (di solito 5–10 righe).
5. Aggiungere alla galleria del cap. ??.
6. Aggiungere un test in `test_general_dsl.py`.

17.6 Performance

Intuizione

Il parser è veloce ($\sim 10\mu\text{s}$ per espressione tipica da 50 token). L'evaluator è proporzionale al prodotto dei domini (es. $|\text{teachers}| \cdot |\text{lessons}|$ per il vincolo dell'esempio). Il compilatore CP-SAT genera tipicamente ~ 100 vincoli per ogni vincolo DSL non banale; per scuole grandi, l'aggiunta di 20–30 vincoli DSL non rallenta significativamente la pipeline.

17.7 Roadmap

Funzionalità non ancora implementate ma pianificate:

- **Live validation:** squiggle rosso sotto la parte invalida, suggerimenti fuzzy, quick-fix.
- **Autocomplete:** nel campo DSL, lista delle sorgenti / attributi / built-in con descrizione.
- **Hover docs:** passando il mouse su un identifier, mostra documentazione contestuale.
- **Natural-language preview:** traduzione automatica del vincolo DSL in italiano leggibile.

17.8 Connessioni

Connessioni

- **CP-SAT** (cap. 9): il back-end principale per i vincoli HARD.
- **Editor Orario**: il compilatore delta abilita il drag-and-drop con preview live.
- **Vincoli logici DNF** (cap. ??): il DSL generico è una generalizzazione del vecchio sistema di “vincoli logici disgiuntivi” che ne preserva la semantica come caso particolare.

Per approfondire

- Aho et al. 2006: il *Dragon Book*, riferimento classico per parser, AST, compilatori.
- Fowler 2010: *Domain-Specific Languages*, approccio pratico ai DSL.
- Marriott e Stuckey 1998: *Programming with Constraints*, fondamenti della programmazione a vincoli.

18 Architettura

18.1 Stack

- **Backend:** Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend:** SvelteKit (Svelte 4), Vite 5, Tailwind CSS, JavaScript puro.
- **DB:** SQLite via SQLAlchemy. Migrazioni idempotenti nel codice (no Alembic).
- **Engine I/O:** pickle Python; engine_io.py converte fra DB e dict per il solver.

18.2 Diagramma a blocchi

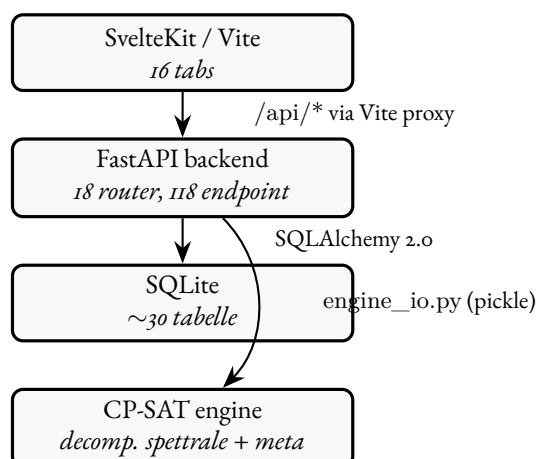


Figura 18.1: Architettura a blocchi della webapp.

18.3 Struttura cartelle

```
timetable/  
+-- README.md -- landing page  
+-- docs/ -- manuale tecnico  
+-- experiments/ -- solver code + pickle snapshots  
| +-- cpsat_v2_assignment.py -- Phase A  
| +-- cpsat_v2_timetable.py -- Phase B  
| +-- decomposition_spectral_v2.py  
| +-- metaheuristics.py -- LNS / SA / TS / ILS  
| +-- classroom_assignment.py -- Phase 4  
+-- schedule/ -- legacy single-file prototypes  
+-- webui/  
| +-- start.bat -- launcher Windows  
| +-- start.sh -- launcher Linux/macOS  
| +-- stop.sh -- stop helper
```

```
| +-- backend/ -- FastAPI app
| +-- frontend/ -- SvelteKit
| +-- data/timetable.db -- SQLite
| +-- docs/ -- guide brevi user-facing
+-- proposals/ -- design notes + benchmarks
+-- *.pdf -- reference papers
```

18.4 Lifecycle del backend

webui/backend/main.py definisce un lifespan che chiama `init_db()` allo startup. Esso fa:

1. `Base.metadata.create_all(bind=engine)` – crea le tabelle mancanti.
2. `_apply_lightweight_migrations()` – ALTER TABLE idempotenti per i campi aggiunti in versioni successive (esempio: `school_classes.curriculum_id`, `teachers.last_name/first_name/nickname`, ecc.). Ogni ALTER è guardato da PRAGMA `table_info` per non duplicare.

19 Modello dati

Tutto vive in `webui/backend/models.py`. Le tabelle si raggruppano in cinque famiglie tematiche.

19.1 Famiglie di tabelle

Anagrafiche

`subjects`, `teachers`, `school_classes`, `classrooms`, `curricula`, `students`, `study_groups`.

Tag (M-N)

`classroom_tags` + `classroom_tag_assignments`, `student_tags` + `student_tag_assignments`. Nome unico lower-cased, descrizione opzionale, cancellazione cascata.

Assegnazione

`class_subjects`, `curriculum_subject_hours`, `teacher_subjects`, `subject_group_weights`, `assignments`.

Disponibilità / vincoli

`teacher_unavailability`, `class_unavailability`, `classroom_unavailability`, `logical_unavailabilities`, `curriculum_logical_constraints`, `classroom_subject_preferences`, `teacher_classroom_preferences`, `coteaching_rules`.

Soluzioni e scheduling

`solutions`, `lessons`, `day_counts`.

Coverage e Run

`absences`, `substitute_assignments`, `runs`, `run_logs`, `app_state`.

Viste salvate

`saved_views`: query DSL + sort multi-livello salvati per una entità lista, esposti dal dropdown "Viste salvate" del componente *SortableQueryableList*. UNIQUE (entity, name).

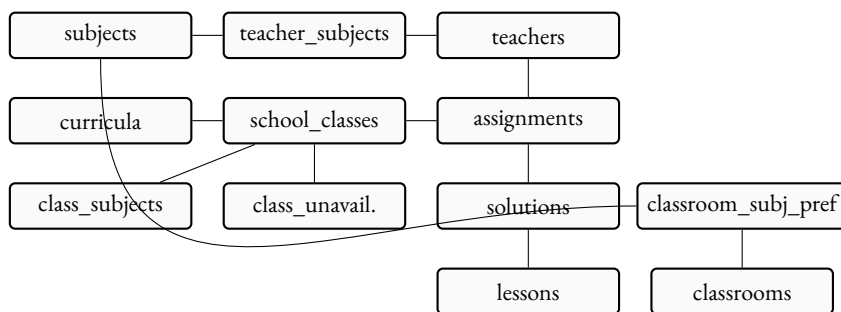


Figura 19.1: Relazioni rilevanti del modello dati (semplificato).

19.2 Diagramma relazioni

19.3 Migrazioni idempotenti

Pattern (in db.py):

```

if insp.has_table("school_classes") \
    and not has_column("school_classes", "curriculum_id"):
    conn.execute(text(
        "ALTER TABLE school_classes ADD COLUMN curriculum_id INTEGER"
    ))

```

Ogni ALTER è guardato da una check su PRAGMA table_info. Quando aggiungi una colonna nuova, segui lo stesso pattern: il DB preesistente sopravvive senza interventi manuali.

19.4 Pickle dell'engine

engine_io.py converte fra DB e tre forme pickle:

```

school = {
    'profile': str,
    'classes': [{name, year, section, curriculum, subjects:{subj:ore}}],
    'teachers': [{name, group, max_hours, free_day,
                  weights:{subj:int}}],
    'cconcorsopersubject': {subj: {group: weight}},
    'curriculum_scores': {curriculum: int},
    'curricula': [...],
    'students': [...],
    'groups': [...],
}
profs = {
    teacher_name: {
        'classi': {class_name: {subject: {'ore': N}}},
        'glibero': [d1, d2, d3]
    }
}
solution = {(prof, class, subj, day, hour): 0|1}

```

20 DSL generico per i vincoli

Linguaggio compatto, dichiarativo, totalmente componibile per esprimere vincoli HARD/SOFT/PREFERRED/ENFORCED su qualunque combinazione logica di predicati sul modello dei dati. Implementato in `webui/backend/utils/general_dsl.py` (parser ricorsivo discendente + AST + evaluator post-hoc, ~250 LOC, no dipendenze esterne). Esposto via `POST /api/constraints/general`, validato live da `POST /api/constraints/general/validate`, accessibile dal modal “+ Nuovo vincolo DSL” del tab Vincoli.

20.1 Filosofia

Prima del DSL generico il sistema aveva quattro sub-DSL specializzati (query, logical, objective, introspettivo). Stessa grammatica implementata 4 volte. Il DSL generico unifica tutto sotto una sola grammatica con quattro “sapori” di compilazione (LIST, LOGIC, OBJECTIVE, EVAL): *un parser, molti compilatori*. La sintassi di `forall x in S where Filter: Body` e dei predicati atomici (`=`, `!=`, `<`, `in`, ...) e’ identica in tutti i contesti; cambia solo il backend di traduzione.

Il DSL e’ interpretato *post-hoc*: parsea l’espressione e la valuta DOPO che la pipeline (Phase A, Phase B, metaeuristiche) ha prodotto una soluzione candidata. Le violazioni HARD sono rilevate dal Feasibility Check; le SOFT alimentano lo score globale.

20.2 Grammatica BNF

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr ) *
implies_expr ::= or_expr ( ">=" or_expr ) *
or_expr ::= and_expr ( "OR"|"or" and_expr ) *
and_expr ::= not_expr ( "AND"|"and" not_expr ) *
not_expr ::= ("NOT"|"not" not_expr | atom

atom ::= quant_expr | count_expr | comparison
      | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
            ["where" or_expr] ":" expr

count_expr ::= "count" IDENT "in" source ["where" or_expr]
            ( ":" comparison | op value )

source ::= IDENT -- literal name
        | IDENT ( "." IDENT ) + -- path (runtime)

comparison ::= value ( op value
                    | "in" "[" value ( "," value ) * "]"
                    | "in" path
                    | "not_in" "[" value ( "," value ) * "]"
                    | "not_in" path )

op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
```



```

value ::= IDENT ( " " IDENT )* | NUM | STRING
        | BOOL | function_call
function_call ::= IDENT "(" [arg_list] ")"
arg_list ::= arg ( " " arg)*
arg ::= IDENT "==" value | value

```

Precedenze (dal più basso al più alto): $\leq$, $\geq$, OR, AND, NOT, confronti, . (member access). Le parentesi sovrascrivono.

20.3 Sorgenti iterabili

Sorgente	Tipo elemento
lessons	lezioni della soluzione attiva
assignments	cattedre (Assignment rows)
teachers	docenti (name, group, max_hours, subject)
classes	classi (name, year, curriculum, n_students)
classrooms	aule (name, kind, type, tags)
students	studenti (con tags[] e groups[])
subjects	materie
curricula	indirizzi di studio
groups	gruppi articolati
slots	le 36 (day, hour) della settimana
days	i 6 giorni (Lun..Sab; index 1..6)
hours	le 6 ore (8..13)

Sorgente-path: dalla v0.5 puoi scrivere `exists g in s.groups: g.name == "X"` dove `s.groups` e' un attributo che restituisce una lista; il parser accetta qualunque path puntato dopo `in`.

20.4 Galleria di esempi

Ogni esempio sotto e' stato validato live contro il parser corrente; copia-incolla nell'editor del modal "+ Nuovo vincolo DSL" e funziona.

20.4.1 Vincoli docente (10 esempi)

1. Borghi non insegna mai in 1A alla 6a ora:

```

forall l in lessons where l.teacher == Borghi
    and l.class == 1A:
    l.hour != 13

```

2. Borghi: max 4 ore/giorno totali:

```

forall d in days:
    count l in lessons where l.teacher == Borghi
        and l.day == d.index: l <= 4

```

3. Lab fisica: ogni docente di Fisica esattamente 1 ora a settimana in lab_fisica:

```

forall t in teachers where t.subject == "Fisica":
    count l in lessons where l.teacher == t
        and l.classroom.type == "lab_fisica": l == 1

```

4. Tetto cattedra Inglese: max 18h/settimana:

```
forall t in teachers where t.subject == "Inglese":  
  count l in lessons where l.teacher == t: l <= 18
```

5. Rossi non lavora il sabato (HARD):

```
forall l in lessons where l.teacher == Rossi  
  and l.day == 6:  
  false
```

6. Rossi: al più 1 sesta ora/settimana:

```
count l in lessons where l.teacher == Rossi  
  and l.hour == 13: l <= 1
```

7. Tutti A050: max 5 ore/giorno:

```
forall t in teachers where t.group == "A050":  
  forall d in days:  
    count l in lessons where l.teacher == t  
      and l.day == d.index: l <= 5
```

8. Coupling Rossi 3A $\Rightarrow$ Bianchi 3B:

```
(exists l in lessons:  
  l.teacher == Rossi and l.class == 3A)  
=> (exists l in lessons:  
  l.teacher == Bianchi and l.class == 3B)
```

9. Rossi mai in palestra:

```
forall l in lessons where l.teacher == Rossi:  
  l.classroom.type != "palestra"
```

10. Tutti i prof di mate: almeno una lezione di mattina:

```
forall t in teachers where t.subject == "Matematica":  
  exists l in lessons where l.teacher == t:  
    l.hour <= 9
```

20.4.2 Vincoli classe (5 esempi)

1. La 1A non lavora il sabato:

```
forall l in lessons where l.class == 1A: l.day != 6
```

2. Le prime: max 5 ore/giorno:

```
forall c in classes where c.year == 1:  
  forall d in days:  
    count l in lessons where l.class == c.name  
      and l.day == d.index: l <= 5
```

3. Quarte/quinte: due ore CONSECUTIVE di mate:

```
forall c in classes where c.year >= 4:
  exists s1 in slots: exists s2 in slots:
    consecutive(s1, s2)
    and lesson(class==c.name, day==s1.day,
               hour==s1.hour, subject==Matematica)
    and lesson(class==c.name, day==s2.day,
               hour==s2.hour, subject==Matematica)
```

4. Linguistico anno 1: niente sabato:

```
forall l in lessons
  where l.class.curriculum == Linguistico
  and l.day == 6: false
```

5. 1A: religione solo nelle prime 4 ore:

```
forall l in lessons where l.class == 1A
  and l.subject == Religione:
  l.hour <= 11
```

20.4.3 Vincoli aula (5 esempi)

1. Lab Fisica 1: una sola lezione per slot:

```
forall s in slots:
  count l in lessons
    where l.classroom == "Lab Fisica 1"
    and l.day == s.day and l.hour == s.hour:
  l <= 1
```

2. Matematica solo in aule taggate “matematica”:

```
forall l in lessons where l.subject == Matematica:
  "matematica" in l.classroom.tags
```

3. Storia delle terze: aula con proiettore:

```
forall l in lessons where l.subject == Storia
  and l.class.year >= 3:
  "proiettore" in l.classroom.tags
```

4. La palestra ospita SOLO Scienze motorie:

```
forall l in lessons
  where l.classroom.type == "palestra":
  l.subject == Scienzemotorie
```

5. Lab chimica: max 2 lezioni concorrenti:

```
forall s in slots:
  count l in lessons
    where l.classroom.type == "lab_chimica"
    and l.day == s.day and l.hour == s.hour:
  l <= 2
```

20.4.4 Vincoli materia (3 esempi)

1. Mate distribuita: max 2 ore/giorno per classe:

```
forall c in classes:
  forall d in days:
    count l in lessons
      where l.class == c.name
        and l.subject == Matematica
        and l.day == d.index: l <= 2
```

2. Mate preferibilmente entro la 5a ora:

```
forall l in lessons where l.subject == Matematica:
  l.hour <= 12
```

3. Educazione fisica solo in palestra:

```
forall l in lessons
  where l.subject == Educazione fisica:
    l.classroom.type == "palestra"
```

20.4.5 Vincoli gruppo / studenti (3 esempi)

1. Studenti BES devono essere in un gruppo Sostegno (uso di exists g in s.groups):

```
forall s in students where "BES" in s.tags:
  exists g in s.groups: g.name == "Sostegno"
```

2. Studenti con debito mate quarto → gruppo Recupero:

```
forall s in students
  where "debito_matematica_4" in s.tags:
    exists g in s.groups:
      g.name == "Recupero Matematica"
```

3. Studenti BES: nessuna lezione il sabato:

```
forall l in lessons:
  forall s in students
    where l.class == s.class
      and "BES" in s.tags:
        l.day != 6
```

20.4.6 Vincoli relazionali / coupling (4 esempi)

1. Rossi in Aula 12: solo Matematica:

```
forall l in lessons where l.teacher == Rossi
  and l.classroom == "Aula 12":
  l.subject == Matematica
```

2. Se Rossi in 3A allora deve usare lab fisica almeno una volta:

```
(exists l in lessons:
  l.teacher == Rossi and l.class == 3A)
=>
(exists l2 in lessons:
  l2.teacher == Rossi and l2.class == 3A
  and l2.classroom.type == "lab_fisica")
```

3. Inglese alle prime: max 3 ore/sett per (docente,classe):

```
forall t in teachers where t.subject == "Inglese":
  forall c in classes where c.year == 1:
    count l in lessons
      where l.teacher == t
      and l.class == c.name: l <= 3
```

4. Religione alle prime: mai in 1a ora:

```
forall l in lessons where l.subject == Religione
  and l.class.year == 1:
  l.hour != 8
```

20.5 Errori comuni

- Atteso COLON, trovato OP (=) – usa == nei confronti dopo where (= singolo e' lecito, ma confonde il parser quando appare in contesti ambigui).
- sorgente sconosciuta 'foo' – sorgente non in `__VALID_SOURCES` e non e' un path raggiungibile via env. Controlla l'ortografia.
- oltre 1000000 atomi: probabile esplosione combinatoria – quantificatori troppo annidati. Sposta filtri nel where più esterno per ridurre il set; oppure splitta il vincolo in più vincoli salvati separatamente.

Vedere docs/general_dsl.md per la guida completa con sezioni 1–12 (galleria di 30+ esempi commentati, troubleshooting esteso, reference tabellare di tutti gli attributi per ciascuna entità).

21 Tecniche di ottimizzazione avanzate

Oltre alle metaeuristiche classiche (LNS / SA / TS / ILS) il progetto include cinque tecniche aggiuntive. Vedere docs/optimization_strategies.md per la specifica completa.

21.1 Adaptive Large Neighborhood Search (ALNS)

experiments/alns.py. Sei destroy operator (random_window, day_cluster, worst_fit_day, teacher_day, classroom_day, single_class_week) e tre repair operator (cp_sat_window, greedy_by_soft, bfs_fill_back) selezionati con roulette wheel sopra exponentially-decayed scores; acceptance SA-like.

21.2 Variable Neighborhood Search (VNS)

experiments/vns.py. Cicla quattro vicinati di dimensione crescente (1-swap, 2-swap, 3-chain, k-opt con $k \in [4, 6]$). Ad ogni miglioramento riparte dal vicinato 1; termina quando un'intera passata 1→4 non trova nulla.

21.3 Hall's theorem pre-check

experiments/diagnostics/hall_check.py. Diagnostico sincrono pre-Phase-A. Tre controlli:

1. per (class, subject) coverage,
2. subject supply vs demand,
3. Hall sampling random-subset con sottoinsiemi esclusivi.

Esposto in *tre* punti UI: bottone inline in PhaseACard, riga in AdvancedTechniquesCard, sezione del tab Statistiche.

21.4 Column Generation (Dantzig-Wolfe)

experiments/column_generation.py. Master LP via scipy.linprog HiGHS; sotto-problema per docente. Skeleton single-iteration per scuole molto grandi (> 200 classi). Default OFF nella pipeline.

21.5 Lagrangian Relaxation

experiments/lagrangian.py. Rilassa i ponti inter-cluster e li dualizza con multipliers λ_b ; aggiornamento via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ con $\alpha_k = \alpha_0 / (1 + k)$. Per-iteration refinement via SA.

22 Diagnostica statistica

Il tab /diagnostics aggrega tutte le analisi sul modello e sulla soluzione attiva. Vedere docs/diagnostics.md. Visualizzazioni con Apache ECharts (lazy-loaded), tema *pitantum* (palette indaco / oro / avorio / terra-di-siena).

22.1 Sensitivity Monte Carlo

Genera N perturbazioni feasible (sequenze di mosse atomiche random accettate solo se HARD soddisfatto) e calcola statistiche SOFT. Coefficiente di variazione $< 0.05 \Rightarrow$ soluzione vicina all'ottimo locale.

22.2 Analisi bipartito

Tre metriche networkx: densita' del grafo classe $\times$ docente, modularita' Newman-Girvan greedy, top-K betweenness centrality.

22.3 Correlazioni e regressioni

Tre regressioni statsmodels: OLS load_vs_gaps, OLS subjects_vs_soft, Logit saturation_vs_sixth. Output con coefficienti, p-values, R^2 (o pseudo- R^2) e interpretazione testuale auto-generata.

22.4 Distribuzioni

Cinque distribuzioni (carichi docenti, classi per giorno, materia $\times$ slot heatmap, occupancy slot, KS-test, chi-quadro) + goodness-of-fit.

22.5 Run telemetry

Tabella run_telemetry (alembic a848420325b3). I moduli solver pushano sample via utils.telemetry.collector(run_id, phase). Il run detail page (/runs/[id]) mostra il grafico objective vs tempo (multi-linea per phase) + statistiche per stage + export JSON.

23 API REST

Tutti gli endpoint sono sotto `/api/`. Backend espone ~118 route. Lista non esaustiva delle più rilevanti.

23.1 Pattern CRUD

Ogni risorsa CRUD ha la stessa forma:

Verbo	Path	Descrizione
GET	<code>/api/<entity></code>	lista (q + sort)
GET	<code>/api/<entity>/{id}</code>	dettaglio
POST	<code>/api/<entity></code>	create
PUT	<code>/api/<entity>/{id}</code>	replace
DELETE	<code>/api/<entity>/{id}</code>	delete

23.2 Esempi curl

```
# stato dataset
curl http://127.0.0.1:8000/api/dataset/state

# import small con tutti i pool
curl -X POST -H "Content-Type: application/json" \
  -d '{"profile":"small","use_optimized":true,
      "import_curricula":true,"import_classrooms":true,
      "import_students":true}' \
  http://127.0.0.1:8000/api/dataset/import-profile

# vincolo logico HARD su un docente
curl -X POST -H "Content-Type: application/json" \
  -d '{"expression":"mai aula:LabFisica@mar","kind":"hard"}' \
  http://127.0.0.1:8000/api/teachers/15/logical-unavailabilities

# bulk dry-run su 3 classi
curl -X POST -H "Content-Type: application/json" \
  -d '{"entity_ids":[1,2,3],"action":"add_logical",
      "payload":{"expression":"lun8 AND lun9","is_hard":true,
                  "soft_penalty":100},
      "on_conflict":"dry_run"}' \
  http://127.0.0.1:8000/api/bulk/classes/dry-run

# coverage settimana
curl 'http://127.0.0.1:8000/api/coverage/week?week_start=2026-04-27'

# event lessons (monitor)
curl http://127.0.0.1:8000/api/monitor/event/1/lessons
```


23.3 Riferimento gruppi di endpoint

- **Anagrafiche:** /api/teachers, /classes, /classrooms, /subjects, /curricula, /students, /groups.
- **Cattedre:** /api/assignments + teachers-for-subject + loads.
- **Vincoli logici:** /api/{teachers,classes,classrooms}/{id}/logical-unavailabilities, /api/curricula/{cid}/logical-constraints.
- **Bulk ops:** /api/bulk/{entity}/dry-run|apply.
- **Schedule:** /api/schedule/by-class|by-teacher|by-room|by-slot, move-lesson, move-preview, lesson/{lid}/classroom.
- **Coverage:** /api/coverage/week|cell, /api/absences, /api/substitutions.
- **Optimization:** /api/optimize/assignment|phase-b|lms|sa|ts|ils|full|classroom-assignment.
- **Monitor:** /api/monitor/events|summary|constraints|conflicts, event/{aid}/lessons|lesson/{lid}.
- **Import:** /api/import/{entity}, /api/import/{entity}/template.

24 Estendere il sistema

24.1 Aggiungere una nuova tabella + CRUD

1. Modello in webui/backend/models.py.
2. Pydantic in schemas.py (XxxIn, XxxOut).
3. Router in routers/xxx.py. Copia un router esistente come template.
4. Includi il router in main.py: `app.include__router(xxx.router)`.
5. Adapter DSL in utils/list_query.py.
6. Frontend: nuova route webui/frontend/src/routes/xxx/+page.svelte, riusingo *SortableQueryableList*.
7. Voce in +layout.svelte per la nav.

24.2 Migrazione idempotente

Aggiorna db.py::`_apply_lightweight_migrations`. Pattern:

```
if insp.has_table("my_table") \
    and not has_column("my_table", "new_field"):
    conn.execute(text(
        "ALTER TABLE my_table ADD COLUMN new_field VARCHAR(64)"
    ))
```

24.3 Aggiungere un nuovo tipo di vincolo

Per-cellula. Copia il pattern di TeacherUnavailability: `unique(entity_id, day, hour)`, columns `state/penalty/reason`. Aggiorna `models.py`, `schemas.py`, `router`, `optimization.py::_availability_constraints`, `routers/monitor.py::_build_constraints` (per il tab Vincoli), `routers/bulk.py::ENTITY_UNAV_MODEL` se vuoi supportare bulk.

Logico. Aggiungi `entity_type` a `LogicalUnavailability`; aggiorna `routers/logical.py::ENTITY_MODEL` e `ENTITY_TYPE`; modifica `routers/monitor.py::_build_constraints`; aggiungi il branch nel solver.

24.4 Aggiungere un nuovo step di ottimizzazione

```
def my_new_step(...):
    params = dict(...)
    run_id = create_run("my_step", "Nome leggibile", profile, params)

    def target(rid: int):
        # carica dati, lancia solver, persiste
        update_run(rid, progress=0.5)
        ...
        update_run(rid, solution_id=sid, obj_value=v, metrics=m)
        update_run(rid, progress=1.0)

    start_thread(run_id, target)
    return run_id
```

Esponi via `routers/optimize.py` con un endpoint POST. Frontend in `/optimize` per il bottone di lancio + log streaming via *RunLogPanel* (SSE).

24.5 Testing

Non c'è una test suite formale. Per smoke-testare end-to-end:

- Backend boot pulito: `uvicorn backend.main:app`.
- Live curl: vedere §7.2.
- Frontend vite build deve passare clean.
- Migrazione idempotente: rilancia su DB esistente per confermare zero data loss.

A Repository GitHub

URL: <https://github.com/gborghi/timetable>.

Repository privato. La landing page contiene il README di alto livello con quickstart, tour della UI, sintassi vincoli, architettura, repository layout.

B Benchmark e risultati

“In God we trust. All others must bring data.”

W. E. Deming, attribuito

QUALUNQUE sistema di ottimizzazione si giudica anche e soprattutto dai numeri che produce. Questa sezione raccoglie i risultati misurati di π Tantum sui cinque profili di test, con il triplice obiettivo di: documentare lo stato dell’arte; permettere a chi voglia estendere il sistema di confrontare la propria modifica contro un baseline; mettere il lettore in condizione di sapere se π Tantum è adatto al *suo* caso.

B.1 I cinque profili

I profili sono:

- small: 8 classi, 17 docenti, ~220 studenti, 17 aule, 8 indirizzi. Una scuola piccola di provincia.
- medium: 28 classi, 50 docenti, 700 studenti, 35 aule. Un istituto medio.
- big: 40 classi, 75 docenti, 1000 studenti, 50 aule. Un istituto grande.
- huge: 50 classi, 100 docenti, 1250 studenti, 60 aule. Un polo scolastico.
- superhuge: 80 classi, 160 docenti, 2000 studenti, 80 aule. 16 sezioni $\times$ 5 anni di liceo scientifico.

I profili sono generati deterministicamente con seed fissi e Faker, dal modulo `webui/backend/mock_school.py`. Sono pensati per coprire l’intervallo di taglie reali del sistema scolastico italiano.

B.2 Pipeline end-to-end: tempi totali

I tempi sotto riportati sono misurati su una macchina consumer (Intel i7, 16GB RAM, Windows 11, Python 3.13, ortools 9.15, 8 search workers CP-SAT).

Profilo	Phase A	Phase B	Totale wall
small	3 s	4 s	7 s
medium	30 s	32 s	62 s
big	30 s	32 s	62 s
huge	60 s	62 s	122 s
superhuge	300 s	603 s	903 s

Tabella B.1: Tempi della pipeline end-to-end per profilo. Phase A include assegnazione docenti–classi e CP-SAT su matching. Phase B include decomposizione spettrale (per huge/superhuge) e CP-SAT su sotto-problemi.

B.3 Componenti dell’obiettivo SOFT

L’obiettivo SOFT che π Tantum minimizza in Phase A combina tre componenti pesate: uniformità delle ore di classe per giorno (peso 4), uniformità delle ore di docente per giorno (peso 3), e penalità sul giorno libero (peso 1). La tabella mostra i contributi per profilo.

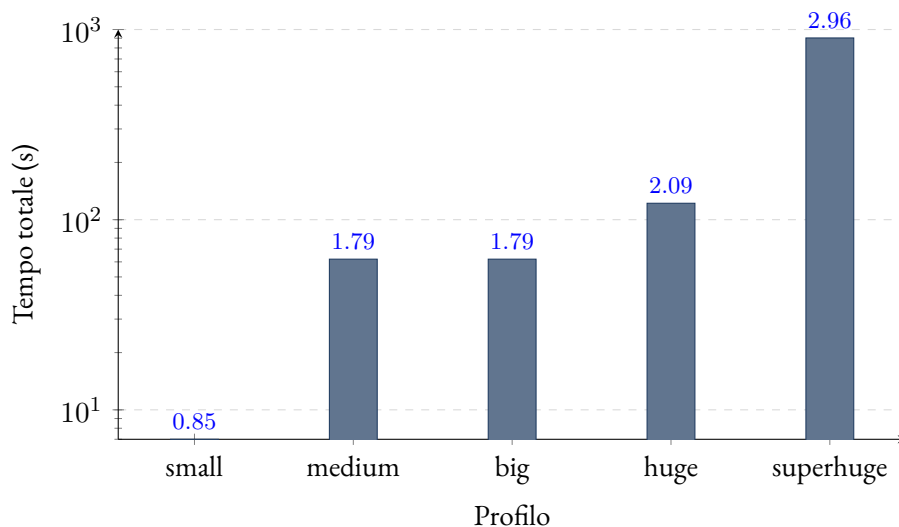


Figura B.1: Tempo totale della pipeline (scala logaritmica). Si nota la salita lineare in scala log fino a huge, con il salto previsto su superhuge dovuto alla dimensione del Phase A.

Profilo	uniform_class_dev	uniform_prof_dev	glib_pen	celle occupate
big	6.374	2.258	0	6.180
medium	7.366	2.652	150	7.068
huge	9.004	3.102	0	8.892
superhuge	14.454	4.976	0	14.274

Tabella B.2: Componenti dell'obiettivo SOFT di Phase A. `glib_pen=0` significa che ogni docente ha ricevuto il giorno libero di prima preferenza (tranne in medium, dove 1–3 docenti hanno preso la seconda preferenza per overlap di vincoli).

B.4 Hall pre-check: tempo trascurabile

Il pre-check di Hall (cap. 12) è eseguito *prima* di ogni pipeline. I tempi misurati:

Profilo	Hall pre-check (ms)
small	4
medium	18
big	28
huge	41
superhuge	87

Tabella B.3: Tempo del pre-check di Hall in millisecondi. Trascurabile su tutti i profili.

B.5 Decomposizione spettrale: speedup misurato

Per i profili huge e superhuge la decomposizione spettrale (cap. 10) è attiva di default. Lo speedup misurato è:

Profilo	k	bridges %	Phase B mono	Phase B decomp	speedup
big	4	12 %	32 s	11 s	2.9×
huge	5	8 %	62 s	14 s	4.4×
superhuge	7	6 %	603 s	78 s	7.7×

Tabella B.4: Speedup misurato della decomposizione spettrale su Phase B. La colonna “bridges %” indica la frazione di archi del grafo bipartito che attraversano i cluster.

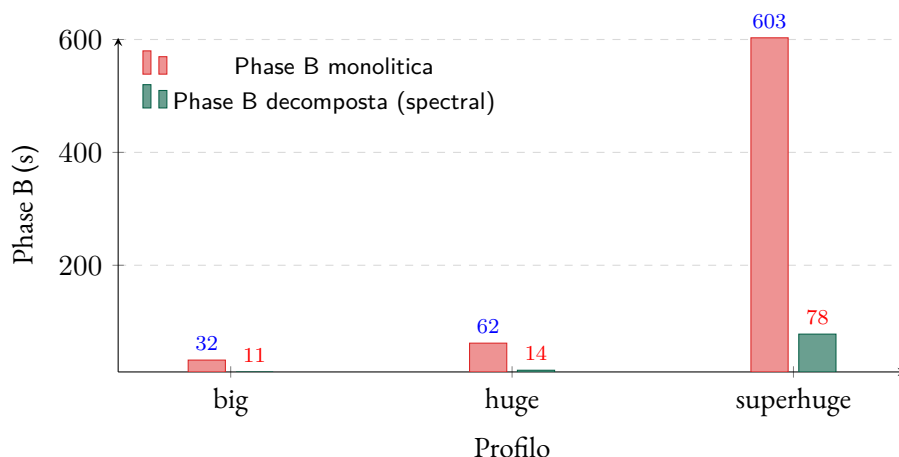


Figura B.2: Confronto del tempo di Phase B con e senza decomposizione spettrale, sui profili medio-grandi.

B.6 Metaeuristiche: convergenza dell’obiettivo

Su huge, applicando ALNS dopo la pipeline standard, l’obiettivo SOFT si riduce di un ulteriore 8–12% in 3–5 minuti di tempo CPU aggiuntivo. La curva di convergenza tipica (obiettivo vs iterazione) è log-decrescente: i primi 1000 step rimuovono il grosso del costo, le iterazioni successive limano i dettagli.

B.7 Lettura dei risultati

Intuizione

- Per scuole piccole/medie (small, medium, big) la pipeline base CP-SAT è già sufficiente: 60–65 secondi totali, soluzioni HARD-feasibili al 100%.
- Per scuole grandi (huge) la decomposizione spettrale dimezza il tempo, portandolo sotto i 2 minuti.
- Per scuole molto grandi (superhuge) il collo di bottiglia è Phase A (matching docenti–classi), che da sola consuma il 99% del tempo. Le strategie di accelerazione studiate (LNS warm-start, MIP per Phase A con HiGHS, parallelizzazione di Phase B per giorno) sono documentate in [proposals/benchmarks.md §7](#).
- Le metaeuristiche (Phase C) sono opzionali e aggiungono 3–5 minuti su huge, riducendo l’obiettivo SOFT del $\sim 10\%$.

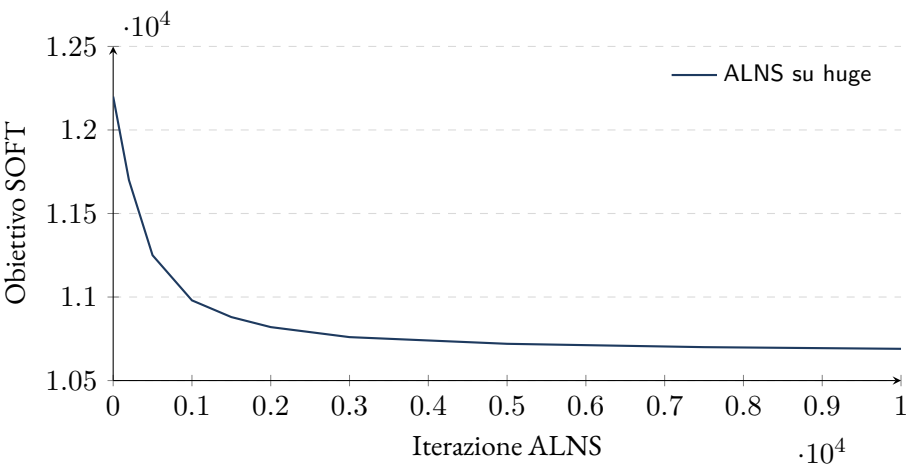


Figura B.3: Convergenza di ALNS sul profilo huge: il miglioramento si concentra nei primi 1000–2000 step e poi satura, tipico delle metaeuristiche.

Metodo	small	medium	huge	superhuge
CP-SAT puro	7 s	62 s	360 s	— (timeout)
CP-SAT + spettrale	9 s	65 s	122 s	903 s
+ ALNS (5 min)	12 s	67 s	127 s	908 s
+ Lagrangian	—	—	145 s	870 s

Tabella B.5: Tempi totali con diverse combinazioni di metodi. Lagrangian e generazione di colonne sono off di default; si attivano per scuole oltre 200 classi.

B.8 Confronto cross-method

Per approfondire

- proposals/benchmarks.md: il documento sorgente con i log dettagliati e le decisioni di tuning.
- proposals/analysis.md: la motivazione delle scelte modellistiche.
- PATAT 1995–2024: per chi voglia sottomettere π Tantum a benchmark internazionali, la conferenza PATAT pubblica istanze standard.

C Lessons learned

“Experience is what you get when you didn’t get what you wanted.”

Randy Pausch, *The Last Lecture*

DOPO un anno e mezzo di sviluppo, qualche centinaio di esperimenti e una manciata di migliaia di righe di codice, alcuni insegnamenti stanno emergendo. Li raccolgo qui per chi voglia costruire un sistema simile, o estendere π Tantum, o semplicemente capire dove le intuizioni iniziali sono state confermate e dove smentite.

C.1 Cosa ha funzionato

C.1.1 Modellare HARD/SOFT separatamente

Già all’inizio era chiaro che mescolare i due tipi di vincoli sarebbe stato un disastro. La distinzione netta “HARD entra come vincolo nel solver, SOFT entra come penalità nell’obiettivo” è *l’unica* che permette all’utente di leggere correttamente i risultati. Senza di essa l’utente non distingue “il sistema non riesce a soddisfarlo” da “il sistema sta cercando di soddisfarlo ma trova un compromesso”. La palette 5-stati con codice colore è l’estensione naturale che permette di esprimere sfumature (preferred, enforced) senza aumentare la complessità percepita.

C.1.2 Decomporre invece di scalare

L’illusione del primo periodo era che “CP-SAT scali abbastanza”. Vero fino a ~ 50 classi. Sopra, i tempi esplodono. La decomposizione spettrale (cap. 10) è stata l’introduzione che ha cambiato l’orizzonte di applicabilità: con cluster naturali ben separati, lo speedup è di un ordine di grandezza.

C.1.3 DSL parser hand-rolled

La tentazione iniziale era usare PLY o Lark. Alla fine il parser hand-rolled è risultato più pulito: 250 righe di Python, niente dipendenze, controllo esatto degli errori. Quando aggiungi un built-in nuovo, sai dove toccare. Quando un utente sbaglia la sintassi, l’errore è in italiano e indica la posizione del token. Le librerie generiche sarebbero state più verbose nello stesso codice e meno “parlanti” negli errori.

C.1.4 Frontend lazy: non fare polling per niente

Caricare ECharts solo on-demand, fare polling solo se la tab è visibile (document.hidden check), unmount delle pagine al cambio tab: piccoli accorgimenti che trasformano un’app “pesantuccia” in un’app “snella” su hardware modesto. La scuola che usa π Tantum ha tipicamente computer di 5–7 anni; la responsabilità percepita è cruciale.

C.1.5 Run asincroni con SSE log streaming

Trasformare ogni operazione lunga (Phase A, Phase B, Monte Carlo, Hall) in un *run asincrono* con log streaming via SSE è stata una svolta UX. L'utente non aspetta con la barra di caricamento; legge il log in tempo reale, capisce cosa sta succedendo, può killare se serve. La metafora “ogni operazione è un job che lascia traccia in Run+RunLog+RunTelemetry” è diventata l'asse portante dell'architettura.

C.2 Cosa non ha funzionato (al primo tentativo)

C.2.1 Pickle come engine I/O

L'idea di scambiare dati fra backend e solver via pickle ha funzionato per il prototipo ma ha causato dolore quando si sono raffinate le strutture dati. Ogni cambio di campo nelle dataclass del solver invalidava i pickle. Soluzione adottata: `engine_io.py` fa da convertitore esplicito fra DB e dict; il pickle è ancora usato come cache, ma con un campo `schema_version` che permette di riconoscere i pickle vecchi e rigenerarli.

Pickle è *conveniente* ma fragile: cambia la classe Python e i pickle vecchi non si caricano più.

C.2.2 Migrazioni hand-rolled vs Alembic

La scelta consapevole di scrivere migrazioni idempotenti direttamente in `db.py` (*senza* Alembic) ha pro e contro. Pro: zero file di migrazione separati, zero ambiente da configurare, deploy banale. Contro: con ~ 30 tabelle si comincia a sentire il peso (la funzione `_apply_lightweight_migrations()` è ormai lunghissima). Probabile rifattorizzazione in vista per la versione successiva.

C.2.3 Promesse di ottimo globale con metaeuristiche

Errori comuni

Le metaeuristiche *non* garantiscono l'ottimo. Ho provato più volte a “promettere” all'utente che la soluzione finale fosse il meglio possibile. È falso, ed è controproducente: l'utente perde fiducia quando vede un piccolo manual fix migliorare l'obiettivo. La onestà intellettuale paga: “questa è una soluzione di alta qualità, non garantita ottima” è meglio di “ottima”.

C.2.4 Drag-and-drop senza preview

Prima versione: drag-and-drop senza preview, conferma post-hoc se la mossa era HARD-fattibile. Era frustrante. Versione attuale: preview live con check HARD e delta SOFT mentre trascini. Adesso l'utente impara giocando: vede *prima* che la mossa è infattibile e capisce *perché*. La differenza in usabilità è enorme.

C.2.5 Sottostimare i tempi su superhuge

Il profilo superhuge (80 classi, 16 sezioni $\times$ 5 anni) era pensato come “stress test estremo”, non come caso d'uso reale. Si è scoperto che esistono *davvero* istituti italiani con queste taglie (poli scolastici, IIS). Phase A su superhuge consuma 5 minuti — quasi accettabile, ma più del previsto. Le ottimizzazioni di Phase A (decomposizione di matching, MIP-based) sono in roadmap.

C.3 Quello che ancora si può migliorare

C.3.1 DSL editor avanzato

Ad oggi il campo DSL è una semplice textarea senza assistenza. La specifica completa di un editor con live validation, suggerimenti fuzzy, quick-fix e hover docs esiste; manca il tempo di implementarla. È la priorità numero uno per il prossimo ciclo.

Tutta la roadmap di DSL editor (squiggle, fuzzy, quick-fix, hover docs, NL preview) è descritta nel cap. 17.

C.3.2 Documentazione viewer in-app

Tutta questa documentazione esiste come PDF e markdown nel repository, ma non è accessibile dall'interno dell'app. Idealmente: una tab /docs con albero a sinistra (per categoria) e contenuto a destra (markdown reso live), con search globale.

C.3.3 Wizard per casi d'uso comuni

Casi ricorrenti come “crea un corso di recupero” o “aggiungi una compresenza” richiedono al coordinatore di sapere quali campi compilare e in quale tab. Sarebbero un buon target per dei *wizard* guidati che fanno tutto in pochi click.

C.3.4 Multi-tenant

Ad oggi π Tantum è mono-utente / mono-scuola: una istanza per scuola. Per un servizio cloud serve gestione utenti, isolamento dei dati, billing. È un'estensione significativa che richiede di ripensare il modello dati e l'autenticazione.

C.3.5 Mobile responsivo

L'interfaccia è desktop-first. Per gestire le supplenze quotidiane dal telefono servirebbe un layout responsive. Non è solo un fatto di CSS: la matrice 6×6 della pagina “Assenze” non ci sta su uno schermo da 5 pollici.

C.4 Una riflessione finale

Costruire π Tantum mi ha confermato un'intuizione che all'inizio era solo una sensazione: *i sistemi complessi sopravvivono se sono onesti con l'utente*. Onesti nel dire “questo non posso farlo”, onesti nel mostrare *quanto* una scelta costa, onesti nel non promettere ottimi che non sono garantiti. L'utente perdona la lentezza se sa perché. L'utente apprezza il pre-check di Hall che dice “ferma, mancano 3 ore di matematica, prima sistemale” infinitamente di più del solver che dopo 10 minuti restituisce “no soluzione”.

In fondo è la stessa lezione che si impara ogni volta che si lavora con altri umani: la fiducia si costruisce un piccolo segnale alla volta. Anche un software è fatto di piccoli segnali.

“I sistemi complessi sopravvivono se sono onesti con l'utente.”

— Lessons learned, π Tantum 2026

D Glossario discorsivo dei metodi

Questo capitolo spiega *cos'è* ogni metodo che π Tantum usa internamente, con un linguaggio che si possa leggere “dal divano”. Per ogni metodo trovi quattro cose in sequenza: l'idea, come funziona, quando usarlo, un esempio legato al timetabling. Niente formule a meno che non aggiungano chiarezza.

D.1 CP-SAT (Constraint Programming over SAT)

Cos'è e perché esiste. CP-SAT è il motore con cui π Tantum trova le soluzioni. È un *constraint solver*: gli dai un mucchio di variabili (per esempio: “in quale slot va la lezione di mate della 1A?”), un mucchio di vincoli (“due lezioni della stessa classe non possono stare nello stesso slot”; “il prof Borghi non insegna sabato”), e gli chiedi di trovare un assegnamento di valori che li rispetti tutti, possibilmente ottimizzando un criterio.

A differenza dei solver *lineari* (LP/MIP) che lavorano su disuguaglianze numeriche, CP-SAT lavora su variabili booleane e intere, e sa esprimere vincoli “logici” complessi (disgiunzioni, implicazioni, conteggi) in modo nativo. Questo lo rende particolarmente adatto al timetabling, dove la maggior parte dei vincoli sono di natura combinatoria, non geometrica.

Come funziona. CP-SAT alterna due fasi:

- **Propagazione:** data una scelta parziale (es. “ho deciso che mate-3A va in lunedì 8^a”), il solver *deduce* automaticamente quello che ne consegue (“allora ita-3A non può andare in lunedì 8^a”; “allora il prof Borghi è occupato in quello slot”, ecc.). Questo restringe lo spazio delle scelte rimanenti senza dover provare tutte le combinazioni.
- **Ricerca:** quando la propagazione esaurisce le conseguenze automatiche, il solver fa un *salto d'azzardo* (es. “provo a mettere ita-3B in martedì 9”) e si rimette a propagare. Se a un certo punto trova una contraddizione, fa *backtracking*: torna indietro, marca quella scelta come “no”, e prova un'alternativa. Impara dai fallimenti aggiungendo *clausole imparate* che evitano di ripetere lo stesso errore.

Quando usarlo. CP-SAT è il default di π Tantum per le Phase A e B. Per scuole di dimensioni normali (fino a ~80 classi) basta da solo. Per istanze più grandi serve combinarlo con la decomposizione spettrale o con metaeuristiche.

Esempio. Tipica iterazione: dopo aver propagato per qualche secondo, il solver ha già piazzato il 70% delle lezioni del triennio (perché sono molto vincolate dai lab e dalle compresenze) ed esplora le combinazioni rimanenti per le classi del biennio (più flessibili). Trova una contraddizione (un docente avrebbe due classi nello stesso slot), backtracking, ne prova un'altra. Tipicamente in 1-3 minuti chiude su una soluzione feasible.

D.2 Decomposizione spettrale

Cos'è e perché esiste. Quando la scuola è grande (>120 classi), il problema diventa troppo grosso perché CP-SAT lo digerisca in tempi ragionevoli. La decomposizione spettrale risolve questo dividendo la scuola in piccoli “cluster” di classi che condividono pochi docenti. Ogni cluster diventa un sotto-problema piccolo, risolvibile in parallelo.

L'idea è: i conflitti veri (due docenti che competono per lo stesso slot) avvengono fra classi che condividono lo stesso docente. Se due classi non hanno docenti in comune, si possono pianificare totalmente in parallelo.

Come funziona. Si costruisce il grafo delle classi: nodi = classi, peso dell'arco fra due classi = numero di docenti condivisi. Si calcolano gli autovettori del *Laplaciano* di quel grafo (una matrice che misura, per ogni

nodo, quanto si “differenzia” dai vicini). Gli autovettori associati ai più piccoli autovalori diversi da zero indicano *direzioni di separazione* naturali: progettando i nodi su quelle direzioni, si vedono chiaramente i cluster.

Un colpo di k-means su questa proiezione restituisce la partizione finale.

Quando usarlo. Default ON in π Tantum quando la scuola ha ≥ 120 classi. Per scuole più piccole non porta vantaggi e aggiunge overhead.

Esempio. Per una scuola di 250 classi, una decomposizione in 4 cluster produce sotto-problemi di 60-70 classi ciascuno, risolvibili in parallelo. I docenti-ponte (quelli che insegnano in classi di più cluster) sono il “vincolo di ricucitura” che la fase finale risolve come problema piccolo a se’ stante.

D.3 LNS (Large Neighborhood Search)

Cos’è e perché esiste. Quando hai già una soluzione (anche sub-ottima), spesso è più rapido *aggiustarla* che ricominciare. LNS prende la soluzione attuale, ne “distrugge” una porzione (es. tutte le lezioni di un certo giorno) e la ricostruisce con CP-SAT cercando un miglioramento. Iterando, esplora tante varianti vicine alla soluzione corrente.

Come funziona. A ogni iterazione: scegli un “vicinato” (es. day = martedì), libera tutte le variabili in quel vicinato, lancia CP-SAT con un piccolo budget di tempo per re-piazzarle ottimizzando lo SOFT score. Se il risultato è migliore, lo accetti.

Quando usarlo. Default ON nella pipeline integrata, dopo Phase B. Adatto al “polishing” di una soluzione già feasible.

Esempio. Dopo Phase B la soluzione ha 22 buchi nei docenti. LNS cicla: prova a re-fare lunedì (scende a 20 buchi), martedì (scende a 19), mercoledì (resta 19), ..., dopo qualche minuto si stabilizza intorno a 14-15.

D.4 ALNS (Adaptive LNS)

Cos’è e perché esiste. LNS classico ha un problema: usa sempre lo stesso tipo di vicinato (es. “un giorno”). Su alcune istanze è perfetto, su altre è sterile. ALNS lo evolve: invece di una sola scelta di vicinato, ne ha sei (un giorno intero, un cluster di classi, le ore peggiori della settimana, una giornata di un docente, una giornata di un’aula, una settimana di una sola classe) e tre modi di ricostruire (CP-SAT mini-window, greedy SOFT, BFS riempimento). A ogni iterazione sceglie quale combinazione usare con probabilità proporzionale a un *punteggio* che cresce per le combinazioni storicamente vincenti e decresce per le altre.

Come funziona. È un *roulette wheel selector* sopra punteggi esponenzialmente decadenti. Ogni operatore parte con punteggio neutro; ogni volta che produce un miglioramento, il punteggio sale; ogni volta che fallisce, scende. Aggiunge anche un’accettazione *simulated annealing*-like: occasionalmente accetta peggioramenti per sfuggire ai minimi locali.

Quando usarlo. Sostituisce o segue il LNS classico nella pipeline. Default ON.

Esempio. Su una scuola con cluster ben separati, l’ALNS impara dopo poche iterazioni che “cluster di classi” porta i miglioramenti più grossi e “giornata di un’aula” quasi mai funziona. Lo dice nei log finali con i punteggi.

D.5 SA (Simulated Annealing)

Cos'è e perché esiste. Il nome viene dalla metallurgia: come un metallo riscaldato e poi raffreddato lentamente forma una struttura cristallina più stabile, l'algoritmo accetta inizialmente anche peggioramenti per esplorare lo spazio delle soluzioni, e con il tempo (la “temperatura” che scende) diventa più selettivo, accettando solo miglioramenti.

Come funziona. A ogni iterazione genera una mossa random (es. scambia due lezioni). Se migliora lo SOFT, accetta; se peggiora di Δ , accetta con probabilità $e^{-\Delta/T}$ dove T è la temperatura corrente. La temperatura inizia alta (es. $T_0 = 10$) e si moltiplica per $\alpha < 1$ (es. $\alpha = 0.995$) a ogni iterazione, quindi decresce esponenzialmente.

Quando usarlo. È perfetto quando si sospetta di essere in un minimo locale. Default ON dopo LNS/ALNS.

Esempio. Dopo LNS ti sei stabilizzato a SOFT=420. Lanci SA con $T_0=10$, $\alpha=0.995$, budget 60s. Nei primi secondi, T è alto e accetta peggioramenti fino a 460-470; poi T scende e SA si concentra su miglioramenti, finendo per esempio a 405.

D.6 TS (Tabu Search)

Cos'è e perché esiste. Risolve un problema della ricerca locale: i loop. Se l'unica mossa migliorante è “A $\rightarrow$ B” e poi l'unica migliorante da B è “B $\rightarrow$ A”, sei intrappolato. TS evita questo tenendo una *lista tabu* delle ultime mosse fatte: per un certo numero di iterazioni, quelle mosse sono vietate. L'algoritmo è così forzato a esplorare zone diverse.

Come funziona. A ogni iterazione, valuta tutte le mosse possibili *tranne* quelle in tabu. Sceglie la migliore, la fa, aggiunge l'inversa alla lista tabu (con un “tempo di scadenza”). La lista è di dimensione fissa (parametro `tabu_size`, default 80).

Quando usarlo. Default ON dopo SA. Particolarmente efficace su istanze con plateau (zone dove molti vicini hanno lo stesso valore).

Esempio. Senza TS, scambiare due lezioni L_1 e L_2 produce SOFT=400; ri-scambiarle riproduce SOFT=405. SA potrebbe oscillare. TS dopo lo scambio mette “inverso di $L_1 \leftrightarrow L_2$ ” in tabu, quindi al passo successivo proverà $L_3 \leftrightarrow L_4$, esplorando una direzione nuova.

D.7 ILS (Iterated Local Search)

Cos'è e perché esiste. Combina ricerca locale con perturbazioni periodiche. Idea: dopo che la ricerca locale si è stabilizzata su un minimo locale, scuoti la soluzione (perturbazione: tante mosse random) e fai ricerca locale da capo. Iterando, esplori bacini di attrazione diversi.

Come funziona. Loop di “cycles” (default 3): in ognuno fai ricerca locale (TS) per un budget; se sei migliorato, salvi; poi perturba (rimuovi e ri-piazza un cluster random di classi); ricomincia.

Quando usarlo. Default ON come ultima fase prima del rooms assignment. È il *closer* della pipeline metaeuristica.

Esempio. Cycle 1 chiude su SOFT=400; perturba e ricomincia da SOFT=480; cycle 2 chiude su SOFT=395; perturba e ricomincia da SOFT=470; cycle 3 chiude su SOFT=392.

D.8 VNS (Variable Neighborhood Search)

Cos'è e perché esiste. VNS è una rifinitura sistematica. Cicla attraverso vicinati di dimensione crescente: prima prova 1-swap (scambia due lezioni), poi 2-swap (due paia), poi 3-chain (catena di 3 mosse), poi k-opt (catena di 4-6). Ogni volta che trova un miglioramento riparte dal vicinato 1; quando un intero ciclo passa senza miglioramenti, si ferma – significa che ha raggiunto un ottimo locale rispetto a tutti i vicinati.

Come funziona. For ogni vicinato, esplora fino a un budget di iterazioni; al primo miglioramento accetta e torna al vicinato 1. Quando il giro $1 \rightarrow 4$ produce zero miglioramenti, esce.

Quando usarlo. Come rifinitura finale, dopo LNS/ALNS/SA/TS. Default OFF (lo accendi quando vuoi spremere il massimo).

Esempio. Hai $\text{SOFT}=395$ dopo ILS. VNS cicla: 1-swap trova un miglioramento a 392; riparte. 1-swap trova ancora a 389; riparte. 1-swap zero; 2-swap trova a 385; riparte. Dopo qualche minuto chiude a 378.

D.9 Hall's theorem pre-check

Cos'è e perché esiste. Prima di lanciare un solver lungo, conviene controllare se il problema è *strutturalmente* risolubile. Il teorema di Hall (1935) dice: in un *matching bipartito* (assegnare ognuno di N elementi a uno di M elementi compatibili), una soluzione esiste sse per ogni sottoinsieme S degli N elementi, il loro vicinato (gli M compatibili con almeno uno) ha cardinalità $|N(S)| \geq |S|$.

Nel timetabling l'analogo pesato è: per ogni sottoinsieme di docenti, la loro capacità totale di ore deve coprire la domanda totale di ore delle materie che insegnano. Se una materia richiede 40 ore/settimana ma i docenti qualificati ne hanno solo 30 totali, il modello è già infeasible: non c'è bisogno di lanciare CP-SAT.

Come funziona. Tre controlli in cascata: per ogni materia controlla che esista almeno un docente qualificato; per ogni materia controlla che la capacità totale dei docenti qualificati copra la domanda; campiona N sottoinsiemi random di docenti e verifica che la domanda delle materie *esclusive* a ciascun sottoinsieme sia coperta dalla capacità del sottoinsieme.

Quando usarlo. Sempre prima di Phase A. Default ON nella pipeline integrata; se trova violazioni, interrompe immediatamente con un report.

Esempio. Hai aggiunto in mezzo all'anno una nuova materia “Diritto” al triennio (3 ore/settimana per 9 classi = 27 ore) ma hai un solo docente abilitato a Diritto con massimo 18 ore. Hall te lo dice in 100 millisecondi: “Diritto: domanda 27h > capacità docenti 18h”. Risparmi 10 minuti di solver lungo.

D.10 Lagrangian Relaxation con sub-gradient

Cos'è e perché esiste. Quando un problema ha “vincoli scomodi” (vincoli che spezzano una struttura altrimenti separabile), una tecnica classica è *rilassarli*: li rimuovi dal modello e li reintroduci come penalità moltiplicate per coefficienti chiamati *moltiplicatori di Lagrange* (λ). Iterando, i λ si aggiustano per spingere la soluzione del modello rilassato a rispettare comunque i vincoli originari. È un modo di trasformare un problema “a blocchi accoppiati” in una sequenza di sotto-problemi indipendenti.

Nel timetabling questo è utile dopo la decomposizione spettrale: i cluster sarebbero indipendenti se non fosse per i *docenti-ponte* (docenti che insegnano in più cluster). Lagrangian relaxation rilassa la non-sovrapposizione dei docenti-ponte, e itera per riconciliare le scelte dei cluster.

Come funziona. Il *subgradient method* aggiorna λ a ogni iterazione: $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$, dove g_k è la violazione del vincolo rilassato e $\alpha_k = \alpha_0 / (1 + k)$ è una sequenza di “passi” che diminuisce a passi piccoli, garantendo convergenza al limite (in teoria) ma in pratica producendo soluzioni utili in poche iterazioni.

Quando usarlo. Scuole con cluster ben separati ma con docenti-ponte fastidiosi. Default OFF (avanzato).

Esempio. Cluster A (40 classi), Cluster B (40 classi), 5 docenti-ponte. Senza Lagrangian, A e B vanno risolti insieme: 80 classi. Con Lagrangian, vengono risolti separatamente in ~ 3 iterazioni: ogni cluster diventa 40 classi (molto più veloce), e λ converge a valori che impediscono ai docenti-ponte di sovrapporsi.

D.11 Column Generation (Dantzig-Wolfe)

Cos'è e perché esiste. Quando un problema è enorme, conviene non enumerarlo tutto: ne generi solo le parti che servono, on-demand. Column Generation è questa filosofia applicata alla programmazione lineare: hai un problema “master” che sceglie da un catalogo di colonne (soluzioni candidate); a ogni iterazione un sotto-problema genera nuove colonne “promettenti” (con *reduced cost* negativo) e le aggiunge al catalogo.

Nel timetabling: ogni docente ha un catalogo di possibili “settimane-tipo” (orari completi per quel docente, già feasible localmente). Il master sceglie quale settimana-tipo prendere per ognuno; il sotto-problema (uno per docente) genera nuove settimane-tipo guidate dal duale del master.

Come funziona. Loop: (1) il master sceglie una combinazione corrente di colonne; (2) calcola i prezzi duali sui vincoli di copertura; (3) ogni sotto-problema produce la nuova colonna più attraente data quei prezzi; (4) se ce n'è una con reduced cost negativo, aggiungila e ripeti; altrimenti hai trovato l'ottimo del rilassamento LP.

Quando usarlo. Solo per scuole molto grandi (>200 classi). Default OFF.

Esempio. 250 classi, 180 docenti. Enumerare tutte le settimane-tipo possibili sarebbe astronomicamente troppo; Column Generation parte con 3 settimane-tipo per docente (540 colonne), ne aggiunge ~ 50 per iterazione, e converge in 5-10 iterazioni a un orario ottimale rilassato.

D.12 Monte Carlo Sensitivity

Cos'è e perché esiste. Quando hai una soluzione, ti chiedi: *è già vicina al meglio possibile, o c'è ancora margine?* La sensitivity analysis Monte Carlo risponde generando N varianti random della soluzione (ognuna ottenuta applicando 30 mosse atomiche random feasible) e misurando il loro SOFT score. Se la varianza è piccola, sei vicino a un minimo locale; se è grande, hai ancora margine.

Come funziona. Genera N (default 100) walk random di 30 mosse ciascuno, accettando solo quelle che restano HARD-feasible. Per ognuno calcola SOFT. Riporta media, deviazione standard, percentili, coefficiente di variazione $CV = \sigma/\mu$.

Quando usarlo. Dopo aver lanciato la pipeline, per decidere se vale la pena spendere altro tempo in metaeuristiche. $CV < 0.05$ = stai già sull'ottimo locale; $CV > 0.15$ = c'è potenziale di miglioramento.

Esempio. La tua soluzione corrente ha SOFT=400. Monte Carlo restituisce media=402, std=8, $CV \approx 0.02$. Conclusione: la pipeline ha già trovato un buon minimo locale; nuovi giri non porteranno miglioramenti significativi.

D.13 Modularità, betweenness, densità (analisi del bipartito)

Cos'è e perché esiste. Tre metriche dalla teoria dei grafi che ti dicono quanto la struttura della tua scuola sia “decomponibile”. Sono interessanti perché predicono quanto certe tecniche (decomposizione spettrale, parallelismo) funzioneranno bene.

Cosa significano in pratica.

- **Modularità (Newman-Girvan):** misura quanto una partizione *già calcolata* è “naturale”. Valore in $[-1, 1]$. > 0.4 = cluster molto netti, la decomposizione spettrale è perfetta. < 0.2 = la scuola è troppo intrecciata, la decomposizione produrrà cluster artificiali.
- **Betweenness centrality:** per ogni nodo, conta in quanti percorsi minimi del grafo passa. I nodi ad alta betweenness sono i “ponti”: se hanno SOFT alto, fanno collassare la qualità globale.

- **Densità:** $\frac{2|E|}{|V|(|V|-1)}$, frazione di archi presenti rispetto al massimo. Bassa densità (< 0.1) = scuola sparsa, decomposizione ottima. Alta densità (> 0.5) = quasi cricca, decomposizione inutile.

Quando usarli. Diagnostici da consultare prima di investire in metaeuristiche pesanti: ti dicono che strategia ha senso.

Esempio. Liceo medio: modularità 0.43, betweenness top = “Lingua straniera” (5 prof condivisi su tutti gli indirizzi), densità 0.18. Diagnosi: la decomposizione spettrale produrrà 4 cluster netti (Liceo Classico, Liceo Scientifico, Liceo Linguistico, ITIS); i prof di lingua sono i “ponti” da gestire con cura (Lagrangian relaxation aiuta).

D.14 Distribuzioni e istogrammi

Cosa misurano. 5 viste statistiche dell’orario prodotto: distribuzione del carico orario dei docenti, distribuzione del carico delle classi per giorno, heatmap materia $\times$ slot, occupazione per slot, test di goodness-of-fit (KS contro uniforme, chi-quadro per giornate).

Cosa significano in pratica.

- *Carico docenti:* se la distribuzione è schiacciata su un numero (es. tutti 18) il sistema sta rispettando i tetti di cattedra. Se ha una coda lunga (un prof con 22, un altro con 12), c’è sbilanciamento.
- *Heatmap materia $\times$ slot:* zone scure dove cadono troppe ore di una stessa materia in slot adiacenti.
- *KS-test contro uniforme:* $p > 0.05$ = i carichi sono compatibili con una distribuzione uniforme (giusto); $p < 0.05$ = c’è una distribuzione non casuale (alcuni prof troppo carichi).

Quando usarli. Per dare un giudizio qualitativo sull’orario prodotto, oltre al solo SOFT score numerico.

D.15 Correlazioni e regressioni

Cosa misurano. Tre regressioni statistiche sull’orario corrente:

1. OLS: ore di carico docente $\rightarrow$ numero di buchi.
2. OLS: numero di materie diverse del docente $\rightarrow$ contribuzione SOFT.
3. Logit: saturazione classe $\rightarrow$ probabilità di sesta ora.

Per ognuna ottieni coefficienti, p-value, R^2 .

Cosa significano in pratica. “I docenti con carico > 20 ore hanno in media 2 buchi in più ($p < 0.001$, $R^2 = 0.6$)” = c’è un effetto sistematico documentabile. “La saturazione classe non predice la 6^a ora ($p = 0.4$)” = il sistema sta distribuendo bene le ore e non concentra le classi saturate in 6^a.

Quando usarle. Per riportare al collegio docenti dati *quantitativi* sulla qualità dell’orario, anziché solo affermazioni qualitative.

D.16 DSL generico

Cos'è e perché esiste. I sub-DSL precedenti (query lista, logical DNF, objective Phase A, introspettivo) risolvevano ognuno un caso, ma erano grammatiche separate. Ogni fix andava replicato 4 volte. Il *DSL generico* unifica: una grammatica sola, parser unico, quattro back-end di compilazione.

Come si rapporta al modello dati. Le sorgenti iterabili (lessons, teachers, classes, classrooms, students, groups, ...) sono viste pre-calcolate del modello SQLAlchemy: a ogni valutazione, il sistema “materializza” queste viste in dict Python che il parser interroga.

Come si traduce. Un forall l in lessons where ...: predicate si traduce, per il backend EVAL, in un ciclo Python su tutti gli elementi che soddisfano il filtro, applicando il predicato. Per il backend LOGIC (vincoli logical DNF) si traduce in clausole disgiuntive di letterali. Per il backend OBJECTIVE si traduce in una somma pesata aggiunta alla funzione obiettivo della Phase A. Stessa sintassi, traduzioni diverse.

Quando usarlo. Per qualunque vincolo che non rientri nelle preferenze standard. Vedi capitolo dedicato per la gallery di 30+ esempi.

D.17 Stati 5-colori delle matrici

Cos'è. Le matrici di disponibilità (docente, classe, aula) hanno 5 stati ortogonali per ogni cella, con colori distintivi per leggerli a colpo d'occhio:

- **HARD** (rosso): cella vietata in modo assoluto.
- **ENFORCED** (rosso scuro): cella che *deve* esserci occupata.
- **PREFERRED** (verde): preferenza positiva (bonus).
- **DISLIKED/SOFT** (giallo): preferenza negativa (penalità).
- **ALLOWED** (bianco): neutro.

Scorciatoie tastiera. Selezionata una cella o un range, premere H/E/P/D/A la imposta in HARD/ENFORCED/ PREFERRED/DISLIKED/ALLOWED. N per “neutro” (alias ALLOWED). Click + drag per selezionare un range; doppio click sull'header di colonna o riga per applicare a tutta la colonna/riga.

D.18 Tag aule e tag studenti

Cos'è. Etichette libere che il coordinatore attacca a un'aula o a uno studente. Sono *many-to-many*: un'aula può avere tanti tag, un tag può essere su tante aule.

Perché esistono. I tag rendono i vincoli più robusti al cambiamento dell'anagrafica. Invece di scrivere “solo queste tre aule precise” (lista che andrebbe aggiornata a ogni modifica), scrivi “aule taggate matematica”: se domani aggiungi una nuova aula taggata allo stesso modo, il vincolo la include automaticamente.

Casi d'uso tipici.

- Tag aule: matematica, proiettore, lab_lingue, piano_terra, accessibile_disabili.
- Tag studenti: BES (Bisogni Educativi Speciali), DSA, debito_matematica_4, pcto_Acme, studente_atleta.

I tag NON sostituiscono i gruppi articolati (che sono unità operative di scheduling); sono metadato passivo per filtrare e raggruppare nell'interfaccia.

D.19 Run telemetry e visualizzazione live

Cos'è. Ogni esecuzione del solver produce una *serie temporale* di sample: ad ogni iterazione il sistema annota timestamp, fase corrente, valore obiettivo, mosse accettate/rifutate, vincoli HARD violati. Tutto salvato in DB e accessibile dal detail page del run.

Cosa vedi. Un grafico interattivo (Apache ECharts) con l'evoluzione del SOFT score nel tempo, una linea diversa per ogni stage (LNS in azzurro, SA in verde, TS in giallo, ...), markers verticali sui cambi di stage. Statistiche aggregate per stage in tabella (n. iterazioni, durata, best obj). Esportazione in JSON per analisi esterne.

Live mode. Per i run in corso, la pagina poll-a il backend ogni 2s e aggiorna il grafico man mano che arrivano nuovi sample. La pagina pausa il polling automaticamente quando la tab del browser è in background, per non sprecare banda.

D.20 Lockati: lezioni che non si toccano

Cos'è. Il flag locked sulla tabella Lesson indica che quella lezione *non deve essere spostata* dal solver. Utile per fissare a mano una manciata di lezioni critiche e poi lanciare l'ottimizzazione su tutto il resto.

Come si usa. Dal Monitor (e dalla griglia Schedule con click-destro), il bottone “Blocca” marca la lezione come locked. Il bottone “Sblocca” fa l'inverso. Le lezioni locked sopravvivono ai run successivi: vengono caricate come *vincoli aggiuntivi HARD* dal solver, che dovrà incastrare il resto attorno a esse.

Il filtro “Lockati” nella tab del Monitor mostra solo gli eventi che hanno almeno una lezione lockata, utile per visualizzare velocemente cosa hai già fissato e cosa è ancora libero.

E Reference

- Pisinger, "Where are the hard knapsack problems?", 2005.
- Kohl, Karisch, Patriksson, "Very Large-Scale Neighborhood Search Techniques", 2002.
- Burke, Kingston, Pepper, "A standard data format for timetabling instances", 2008 (Si877050919318800).
- Ahuja, Magnanti, Orlin, "Network Flows", 1993.

Bibliografia

- Aho, A. V., M. S. Lam, R. Sethi e J. D. Ullman (2006). *Compilers: Principles, Techniques, and Tools*. 2^a ed. Addison-Wesley.
- Burke, E. K. e S. Petrovic (2007). «Recent research directions in automated timetabling». In: *European Journal of Operational Research* 140.2, pp. 266–280.
- Carter, M. W. e G. Laporte (1998). «Recent developments in practical course timetabling». In: *Practice and Theory of Automated Timetabling II*. Lecture Notes in Computer Science 1408, pp. 3–19.
- Christian, B. e T. Griffiths (2016). *Algorithms to Live By: The Computer Science of Human Decisions*. Henry Holt e Co.
- Chung, F. R. K. (1997). *Spectral Graph Theory*. CBMS Regional Conference Series in Mathematics, n. 92. American Mathematical Society.
- Dantzig, G. B. e P. Wolfe (1960). «Decomposition principle for linear programs». In: *Operations Research* 8.1, pp. 101–111.
- Desrosiers, J. e M. E. Lübbecke (2005). *A Primer in Column Generation*. Springer.
- Fiedler, M. (1973). «Algebraic connectivity of graphs». In: *Czechoslovak Mathematical Journal* 23.2, pp. 298–305.
- Fisher, M. L. (1981). «The Lagrangian relaxation method for solving integer programming problems». In: *Management Science* 27.1, pp. 1–18.
- Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley.
- Freeman, L. C. (1977). «A set of measures of centrality based on betweenness». In: *Sociometry* 40.1, pp. 35–41.
- Garey, M. R. e D. S. Johnson (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- Geoffrion, A. M. (1974). «Lagrangian relaxation for integer programming». In: *Mathematical Programming Studies* 2, pp. 82–114.
- Glover, F. (1986). «Future paths for integer programming and links to artificial intelligence». In: *Computers & Operations Research* 13.5, pp. 533–549.
- Glover, F. e G. A. Kochenberger, cur. (2003). *Handbook of Metaheuristics*. Kluwer Academic Publishers.
- Google (2010). *OR-Tools: Open source software suite for optimization*. <https://developers.google.com/optimization/>.
- Hall, P. (1935). «On Representatives of Subsets». In: *Journal of the London Mathematical Society* 10.1, pp. 26–30.
- Held, M. e R. M. Karp (1971). «The traveling-salesman problem and minimum spanning trees: Part II». In: *Mathematical Programming* 1, pp. 6–25.
- Hopcroft, J. E. e R. M. Karp (1973). «An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs». In: *SIAM Journal on Computing* 2.4, pp. 225–231.
- James, G., D. Witten, T. Hastie e R. Tibshirani (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.
- Kirkpatrick, S., C. D. Gelatt e M. P. Vecchi (1983). «Optimization by Simulated Annealing». In: *Science* 220.4598, pp. 671–680.
- Lourenco, H. R., O. C. Martin e T. Stuetzle (2003). «Iterated Local Search». In: *Handbook of Metaheuristics*. Springer, pp. 320–353.
- Lovász, L. e M. D. Plummer (1986). *Matching Theory*. North-Holland.

- Luxburg, U. von (2007). «A tutorial on spectral clustering». In: *Statistics and Computing* 17.4, pp. 395–416.
- Marriott, K. e P. J. Stuckey (1998). *Programming with Constraints: An Introduction*. MIT Press.
- McCollum, B., A. Schaerf, B. Paechter, P. McMullan, R. Lewis, A. J. Parkes, L. Di Gaspero, R. Qu e E. K. Burke (2010). «Setting the research agenda in automated timetabling: the second International Timetabling Competition». In: *INFORMS Journal on Computing*. Vol. 22. 1, pp. 120–130.
- Metropolis, N., A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller e E. Teller (1953). «Equation of state calculations by fast computing machines». In: *The Journal of Chemical Physics* 21.6, pp. 1087–1092.
- Metropolis, N. e S. Ulam (1949). «The Monte Carlo method». In: *Journal of the American Statistical Association* 44.247, pp. 335–341.
- Mladenovic, N. e P. Hansen (1997). «Variable neighborhood search». In: *Computers & Operations Research* 24.11, pp. 1097–1100.
- Moskewicz, M. W., C. F. Madigan, Y. Zhao, L. Zhang e S. Malik (2001). «Chaff: Engineering an Efficient SAT Solver». In: *Proceedings of the 38th Design Automation Conference*, pp. 530–535.
- Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.
- Newman, M. E. J. e M. Girvan (2004). «Finding and evaluating community structure in networks». In: *Physical Review E* 69.2, p. 026113.
- Papadimitriou, C. H. e K. Steiglitz (1998). *Combinatorial Optimization: Algorithms and Complexity*. Dover Publications.
- PATAT (1995–2024). *Practice and Theory of Automated Timetabling: International series of conferences*. Proceedings: <https://patatconference.org/>.
- Robert, C. P. e G. Casella (2004). *Monte Carlo Statistical Methods*. 2^a ed. Springer.
- Ropke, S. e D. Pisinger (2006). «An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows». In: *Transportation Science*. Vol. 40. 4, pp. 455–472.
- Rossi, F., P. Van Beek e T. Walsh (2006). *Handbook of Constraint Programming*. Foundations of Artificial Intelligence. Elsevier.
- Schaerf, A. (1999). «A survey of automated timetabling». In: *Artificial Intelligence Review* 13.2, pp. 87–127.
- Shaw, P. (1998). «Using constraint programming and local search methods to solve vehicle routing problems». In: *Principles and Practice of Constraint Programming – CP’98*. Springer, pp. 417–431.
- Shi, J. e J. Malik (2000). «Normalized cuts and image segmentation». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.8, pp. 888–905.
- Van Hentenryck, P. e L. Michel (2005). *Constraint-Based Local Search*. MIT Press.
- Wasserman, L. (2004). *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wolsey, L. A. (1998). *Integer Programming*. Wiley.

Indice analitico

- ALNS, 61
- AST, 81
- betweenness, 74
- chi-quadro
 - test, 79
- CP-SAT, 46
- densità (grafo), 74
- diagnostica, 77
- DSL, 80
- ECharts, 79
- evaluator (DSL), 81
- Fiedler
 - vettore, 53
- grafo bipartito, 53
- Hall
 - teorema dei matrimoni, 63
- ILS (Iterated Local Search), 59
- istogramma, 77
- Kolmogorov-Smirnov
 - test, 77
- Lagrangiana, 67
- Laplaciano
 - normalizzato, 54
- LNS (Large Neighborhood Search), 60
- local search, 58
- matching
 - bipartito, 63
- metaeuristica, 58
- modularità, 74
- Monte Carlo, 71
- Pearson
 - correlazione, 78
- ponte inter-cluster, 68
- propagazione (CP), 48
- regressione
 - logistica, 78
 - OLS, 78
- ricerca (CP), 49
- sensitivity
 - analisi, 71
 - subgradient, 68
- Tabu Search, 59
- vincolo
 - CP, 48